

Appendix L: ResNet50V2 Tests 7

In [1]: `!nvidia-smi`

```
Tue Mar 25 15:47:13 2025
+-----+
+-----+
| NVIDIA-SMI 550.54.15                  Driver Version: 550.54.15          CUDA Version:
12.4    |
|-----+-----+-----+-----+
+-----+
| GPU   Name                               Persistence-M | Bus-Id        Disp.A | Volatile Unc
orr. ECC |
| Fan   Temp   Perf           Pwr:Usage/Cap |      Memory-Usage | GPU-Util  Co
mpute M. |
|              |                    |                      |
MIG M. |
|=====+=====+=====+=====+
=====|
|    0   NVIDIA A100-SXM4-40GB                Off | 00000000:00:04.0 Off |
0|
| N/A    29C    P0              46W / 400W |      0MiB / 40960MiB |      0%
Default |
|              |                    |                      |
Disabled |
+-----+-----+-----+-----+
+-----+
+-----+
| Processes:
|
| GPU   GI    CI          PID    Type    Process name                        GP
U Memory |
|       ID    ID                                   Us
age      |
|=====+=====+=====+=====+
=====|
| No running processes found
|
+-----+-----+-----+-----+
+-----+
```

In [2]: `#!/pip install tensorflow`
`import tensorflow as tf`
`print(tf.config.list_physical_devices('GPU'))`

[PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]

Import libraries

In [3]: `import tensorflow as tf # Access to models, datasets and training`
`from tensorflow.keras.datasets import cifar100 # Access to CIFAR-100`
`from tensorflow.keras.applications import ResNet50 # Access to pre-trained model`
`from tensorflow.keras import layers, models, optimizers # Access to building blo`

```

from tensorflow.keras.models import load_model # Ability to load best_saved mode
from tensorflow import keras # Access to stuff for model
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint, ReduceLROnPlateau
from sklearn.model_selection import train_test_split # To help split dataset up
from sklearn.metrics import confusion_matrix, classification_report # Analysis of model
import numpy as np # Manipulate data
import pandas as pd # Statistical analysis of data
import seaborn as sns
import matplotlib.pyplot as plt # Plot data

```

Function definitions

```

In [4]: def display_imgs(imgs, labels):
        plt.subplots(figsize=(10,10))
        for i in range(16):
            plt.subplot(4, 4, i+1)
            k = np.random.randint(0, imgs.shape[0])
            if i == 0:
                print(f"labels[{k}].shape: {labels[k].shape}")
                print(f"imgs[{k}].shape: {imgs[k].shape}")
            plt.imshow(imgs[k])
            #plt.title(labels[k])
            plt.axis('off')
        plt.tight_layout()
        plt.show()

def class_distrib(y, labels_names, dataset_name):
    counts = pd.DataFrame(data=y).value_counts().sort_index()
    #print(f"counts:\n{counts}")
    fig, ax = plt.subplots(figsize=(20,10))
    ax.bar(labels_names, counts)
    ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
    plt.title(f"Distribution of '{dataset_name}' Dataset", fontsize=25)
    plt.grid()
    plt.tight_layout()
    plt.show()

def augment_dataset(x, y):
    x = tf.image.resize(x, (224, 224)) # Resize images
    x = tf.image.random_flip_left_right(x) # Random horizontal flip
    x = tf.image.random_brightness(x, max_delta=0.2) # Adjust brightness
    x = tf.image.random_contrast(x, lower=0.8, upper=1.2) # Adjust contrast
    y = tf.squeeze(tf.one_hot(y, depth=100, dtype=tf.float32)) # One-hot encoding
    return x, y

def plot_evidence(epochs, train_loss, val_loss, train_acc, val_acc):
    # Plot Training and Validation Loss
    plt.figure(figsize=(12, 5))
    plt.subplot(1, 2, 1)
    plt.plot(epochs, train_loss, label='Training Loss')
    plt.plot(epochs, val_loss, label='Validation Loss')
    plt.xlabel('Epoch')
    plt.ylabel('Loss')
    plt.title('Training and Validation Loss')
    plt.legend()
    plt.grid()

```



```

        'caterpillar', 'cockroach', 'bear', 'leopard', 'lion', 'tiger', 'wo
        'road', 'skyscraper', 'cloud', 'forest', 'mountain', 'plain', 'sea'
        'elephant', 'kangaroo', 'fox', 'porcupine', 'possum', 'raccoon', 's
        'spider', 'worm', 'baby', 'boy', 'girl', 'man', 'woman', 'crocodile'
        'turtle', 'hamster', 'mouse', 'rabbit', 'shrew', 'squirrel', 'maple
        'bicycle', 'bus', 'motorcycle', 'pickup truck', 'train', 'lawn-mow
        'tractor']

class_distrib(y_train, labels_names, "Training")
class_distrib(y_val, labels_names, "Validating")
class_distrib(y_test, labels_names, "Testing")

# Create TensorFlow datasets

batch_size = 64

train_dataset_ = (tf.data.Dataset.from_tensor_slices((X_train, y_train))
                  .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                     tf.squeeze(tf.one_hot(y, depth=100, dtype=
                  .batch(batch_size)
                  .prefetch(tf.data.experimental.AUTOTUNE)))

# DEBUG augment
#train_dataset_aug = (tf.data.Dataset.from_tensor_slices((X_train, y_train))
#                    .map(augment_dataset, num_parallel_calls=tf.data.experimen
#                    .batch(batch_size)
#                    .prefetch(tf.data.experimental.AUTOTUNE)))

# Combine the original dataset and the augmented dataset
train_dataset = train_dataset_.concatenate(train_dataset_aug)

val_dataset = (tf.data.Dataset.from_tensor_slices((X_val, y_val))
              .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                               tf.squeeze(tf.one_hot(y, depth=100, dtype=
              .batch(batch_size)
              .prefetch(tf.data.experimental.AUTOTUNE)))

test_dataset = (tf.data.Dataset.from_tensor_slices((X_test, y_test))
               .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                tf.squeeze(tf.one_hot(y, depth=100, dtype=
               .batch(batch_size)
               .prefetch(tf.data.experimental.AUTOTUNE)))

print(f"Training dataset:\n {train_dataset}")
for img, lbl in train_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}") # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}") # Should be (batch_size, 10)
    del img, lbl
print(f"\nValidation dataset:\n {val_dataset}")
for img, lbl in val_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}") # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}") # Should be (batch_size, 10)
    del img, lbl
print(f"\nTesting dataset:\n {test_dataset}")
for img, lbl in test_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}") # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}") # Should be (batch_size, 10)

```

```

del img, lbl

#### <<<<<<<<Pre-trained model>>>>>>>>>
# Load ResNet50 pre-trained on ImageNet (w/out the top classification layer
resnet_50_base = tf.keras.applications.ResNet152V2(weights='imagenet', inclu

# Freeze the Layers of VGG16 so they don't get updated during training - can
resnet_50_base.trainable = False

for layer in resnet_50_base.layers:
    if isinstance(layer, tf.keras.layers.Conv2D):
        layer.kernel_regularizer = tf.keras.regularizers.l2(3e-1) # DEBUG L2

# Add custom classification layers for CIFAR-100 (100 classes) - adapt model
model = models.Sequential([
    resnet_50_base,
    layers.GlobalAveragePooling2D(), # Better for ResNet than Flatten
    layers.Dense(512, activation='relu'),
    layers.Dense(100, activation='softmax') # CIFAR-100 has 100 classes
])

for sample in test_dataset.take(1):
    print(type(sample)) # Should be <class 'tuple'>
    print(len(sample)) # Should be 2
    print(type(sample[0]), type(sample[1])) # Both should be <class 'tensor'>
    print(sample[0].shape) # Should be (batch_size, 224, 224, 3)
    print(sample[1].shape) # Should be (batch_size, 100)
print(f"Model input shape: {model.input_shape}")
print(f"Model output shape: {model.output_shape}")
sample = next(iter(test_dataset.as_numpy_iterator()))
print(len(sample)) # Should be 2
print(type(sample[0]), type(sample[1])) # Both should be <class 'numpy.ndarray'>
print(sample[0].shape, sample[1].shape) # Should match model input and output
print("\n")
#for x, y in test_dataset.take(1):
#    print(type(x), type(y)) # Both should be <class 'tensorflow.Tensor'>
#for x_batch, y_batch in test_dataset.take(1):
#    test_loss, test_acc = model.evaluate(x_batch, y_batch)
#    print(f"Test Loss: {test_loss}, Test Accuracy: {test_acc}")

# Compile the model
#tensorboard_callback = keras.callbacks.TensorBoard(log_dir="./Logs")
model.compile(optimizer=optimizers.Adam(learning_rate=1e-3, weight_decay=3e-
    loss='categorical_crossentropy',
    metrics=['accuracy', 'precision', 'f1_score'])

model.summary()

#### <<<<<<<<Train Model & Track Training/Validation Error>>>>>>>>>
early_stopping = EarlyStopping(monitor='val_loss', # or val_accuracy
    patience=5, # Num. epochs with no improvement
    restore_best_weights=True)
reduce_lr = ReduceLROnPlateau(monitor='val_loss', # or val_accuracy
    factor=0.5, # Reduce Lr by a factor
    patience=3, # Num epochs w/ no improvement
    min_lr=1e-6, # Min Lr
    verbose=1)
#tensorboard = TensorBoard(log_dir='./Logs', # Logs directory
#    histogram_freq=1, # Logs histograms for weights/acc

```

```

# write_graph=True, # Logs graph of model
# write_images=True) # Log images like weight histog
checkpoint = ModelCheckpoint('best_model_v1.keras', # Path where the best mo
    monitor='val_loss', # What to monitor (validati
    save_best_only=True, # Only save model when val
    mode='min', # Looking to minimise the validation
    verbose=1) # Print out message when saving mode
checkpoint_fine_tune = ModelCheckpoint('best_model_fine_tune_v1.keras', # Pa
    monitor='val_loss', # What to monitor (validati
    save_best_only=True, # Only save model when val
    mode='min', # Looking to minimise the validation
    verbose=1) # Print out message when saving mode
#cv_logger = CSVLogger('training_log.csv', separator=',', append=True) # Sa

# Train the model
history = model.fit(train_dataset, validation_data=val_dataset, epochs=25,
    batch_size=batch_size, callbacks=[checkpoint, early_stop

#### <<<<<<<<<Plot Training & Validation Error>>>>>>>>>>>>

# Extract loss and accuracy
epochs = range(1,len(history.history['loss'])+1)
train_loss = history.history['loss']
val_loss = history.history['val_loss']
train_acc = history.history['accuracy']
val_acc = history.history['val_accuracy']

plot_evidence(epochs, train_loss, val_loss, train_acc, val_acc)

#### <<<<<<<<<Evaluate Model on Test Data>>>>>>>>>>>>

# Evaluate on test data
model = load_model('best_model_v1.keras')
results = model.evaluate(test_dataset)
test_loss = results[0]
test_acc = results[1]
test_precision = results[2]
test_f1_scores = results[3]
print(f"Test Accuracy: {test_acc*100:.2f}%")
print(f"Test Loss: {test_loss}")
print(f"Test Precision: {test_precision*100:.2f}%")
print(f"Test F1 Scores (Per Class): {test_f1_scores.numpy()*100}")
print(f"Average Test F1 Scores:{np.average(test_f1_scores.numpy()*100):.2f}\

#### <<<<<<<<<Generate Confusion Matrix>>>>>>>>>>>>

# Get predictions
X_test_revised = tf.image.resize(X_test, (224, 224))
y_pred = model.predict(X_test_revised)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true_classes = y_test.flatten()

# Compute confusion matrix
conf_matrix = confusion_matrix(y_true_classes, y_pred_classes)

# Plot confusion matrix
plt.figure(figsize=(20, 20))
sns.heatmap(conf_matrix) #cmap='Blues', fmt='d'
plt.xlabel('Predicted Labels')
plt.ylabel('True Labels')

```

```

plt.title('Confusion Matrix')
plt.tight_layout()
plt.show()

print(classification_report(y_true_classes, y_pred_classes, target_names=lab
#tensorboard --logdir==path_to_your_logs

# Create a DataFrame from the history of the training and store the epoch va
hist = pd.DataFrame(history.history)
hist['epoch'] = history.epoch

# Finally, display the hist DataFrame.
hist

#### <<<<<<<<Fine-Tune>>>>>>>>>>>>
# Adapt Model
for layer in resnet_50_base.layers:
    layer.trainable = True # Allow layers to be updated

# Compile again w/ Lower Learning rate (prevents destroying learned features
model.compile(optimizer=optimizers.Adam(learning_rate=1e-5),
              loss='categorical_crossentropy',
              metrics=['accuracy', 'precision', 'f1_score'])

model.summary()

#### <<<<<<<<Modify Dataset>>>>>>>>>>>>

# DEBUG augment
train_dataset_aug_ = (tf.data.Dataset.from_tensor_slices((X_train, y_train))
                      .map(augment_dataset, num_parallel_calls=tf.data.experimental
                          .batch(batch_size)
                          .prefetch(tf.data.experimental.AUTOTUNE))

# Combine the original dataset and the augmented dataset
train_dataset_aug = train_dataset.concatenate(train_dataset_aug_) # Not val

#### <<<<<<<<Train Model & Track Training/Validation Error>>>>>>>>>>>>

# Train the model
history_fine_tune = model.fit(train_dataset_aug, validation_data=val_dataset
                             batch_size=batch_size, callbacks=[checkpoint_f

#### <<<<<<<<Plot Training & Validation Error>>>>>>>>>>>>

# Extract loss and accuracy
epochs = range(1, len(history_fine_tune.history['loss'])+1)
train_loss = history_fine_tune.history['loss']
val_loss = history_fine_tune.history['val_loss']
train_acc = history_fine_tune.history['accuracy']
val_acc = history_fine_tune.history['val_accuracy']

plot_evidence(epochs, train_loss, val_loss, train_acc, val_acc)

#### <<<<<<<<Evaluate Model on Test Data>>>>>>>>>>>>

# Evaluate on test data
model_fine_tune = load_model('best_model_fine_tune_v1.keras')
results = model_fine_tune.evaluate(test_dataset)

```

```

test_loss = results[0]
test_acc = results[1]
test_precision = results[2]
test_f1_scores = results[3]
print(f"Test Accuracy: {test_acc*100:.2f}%")
print(f"Test Loss: {test_loss}")
print(f"Test Precision: {test_precision*100:.2f}%")
print(f"Test F1 Scores (Per Class): {test_f1_scores.numpy()*100}")
print(f"Average Test F1 Scores:{np.average(test_f1_scores.numpy()*100):.2f}\n")

#### <<<<<<<<Generate Confusion Matrix>>>>>>>>>

# Get predictions
X_test_revised = tf.image.resize(X_test, (224, 224))
y_pred = model_fine_tune.predict(X_test_revised)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true_classes = y_test.flatten()

# Compute confusion matrix
conf_matrix = confusion_matrix(y_true_classes, y_pred_classes)

# Plot confusion matrix
plt.figure(figsize=(20, 20))
sns.heatmap(conf_matrix) #cmap='Blues', fmt='d'
plt.xlabel('Predicted Labels')
plt.ylabel('True Labels')
plt.title('Confusion Matrix')
plt.tight_layout()
plt.show()

print(classification_report(y_true_classes, y_pred_classes, target_names=lab

# Create a DataFrame from the history of the training and store the epoch va
hist = pd.DataFrame(history_fine_tune.history)
hist['epoch'] = history_fine_tune.epoch

# Finally, display the hist DataFrame.
hist

```

X_temp.shape: (8000, 32, 32, 3)

X_temp_val.shape: (4000, 32, 32, 3)

y_temp_val.shape: (4000, 1)

X_temp_test.shape: (4000, 32, 32, 3)

y_temp_test.shape: (4000, 1)

X_train.shape: (42000, 32, 32, 3)

y_train.shape: (42000, 1)

X_val.shape: (9000, 32, 32, 3)

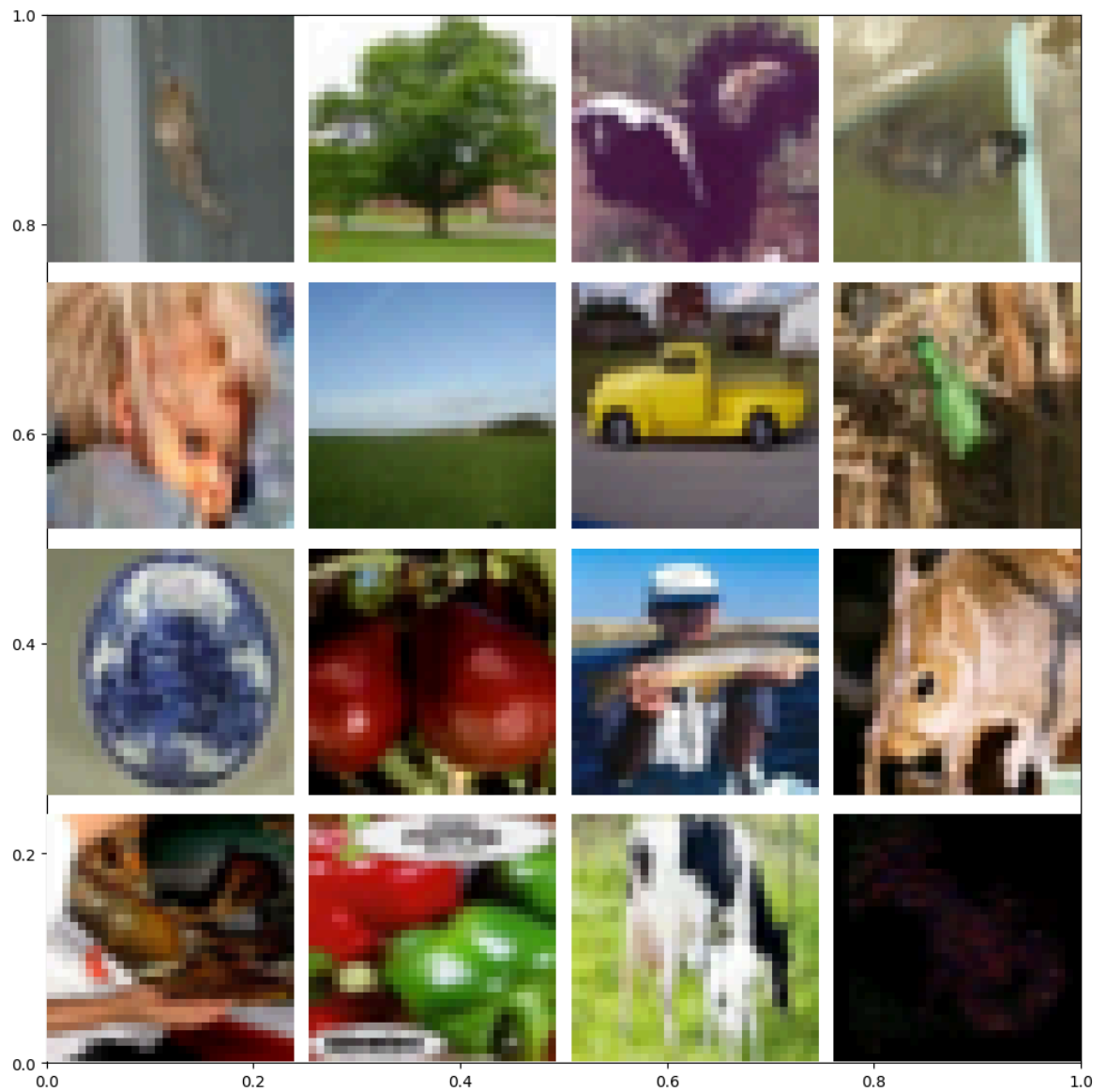
y_val.shape: (9000, 1)

X_test.shape: (9000, 32, 32, 3)

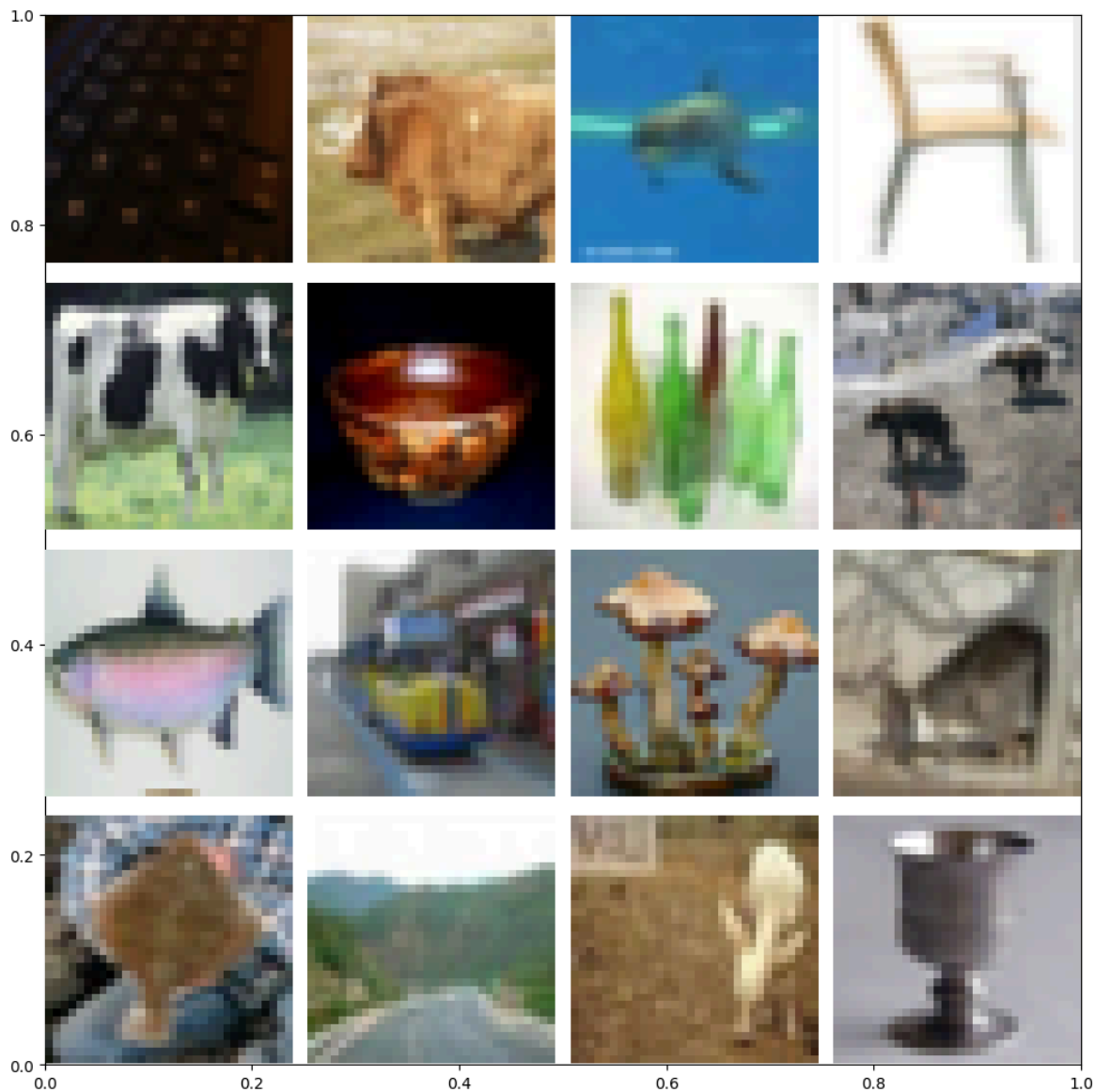
y_test.shape: (9000, 1)

labels[819].shape: (1,)

imgs[819].shape: (32, 32, 3)

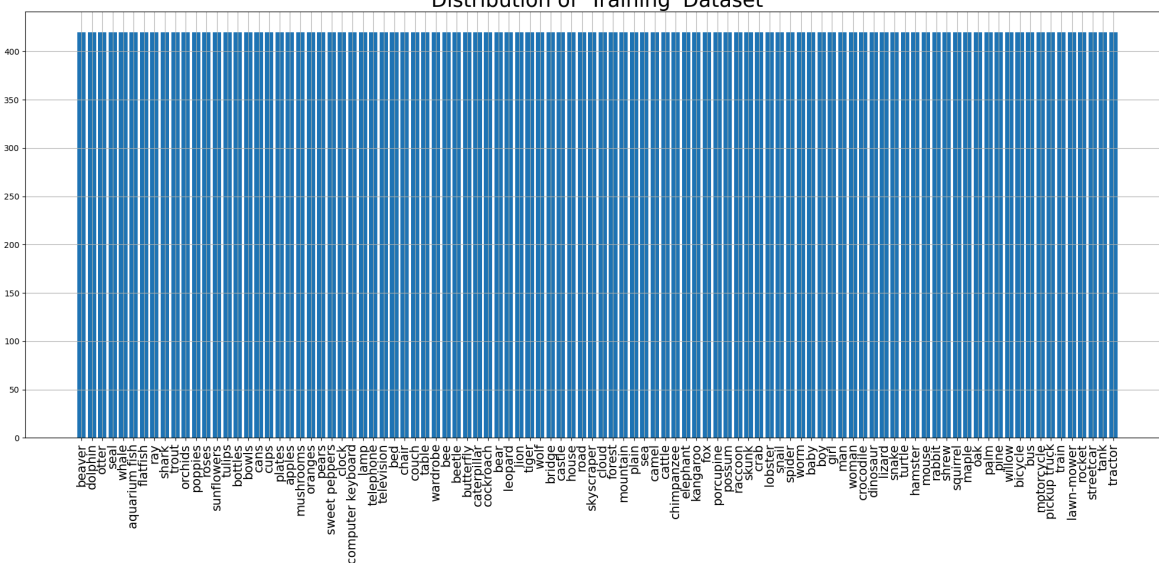


labels[21487].shape: (1,)
imgs[21487].shape: (32, 32, 3)

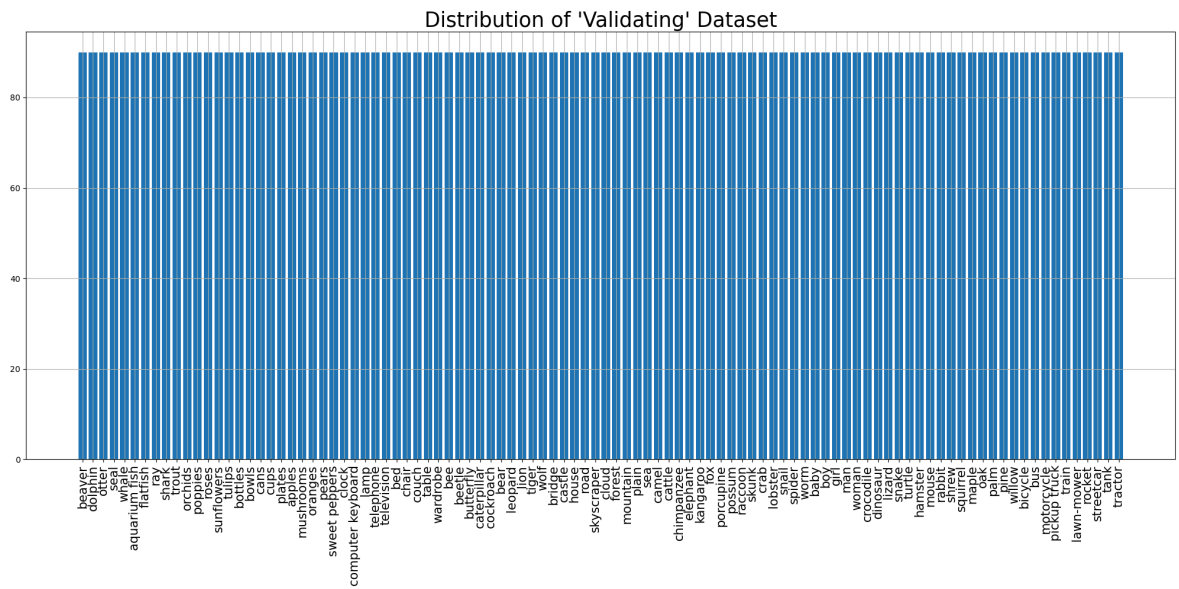


```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```

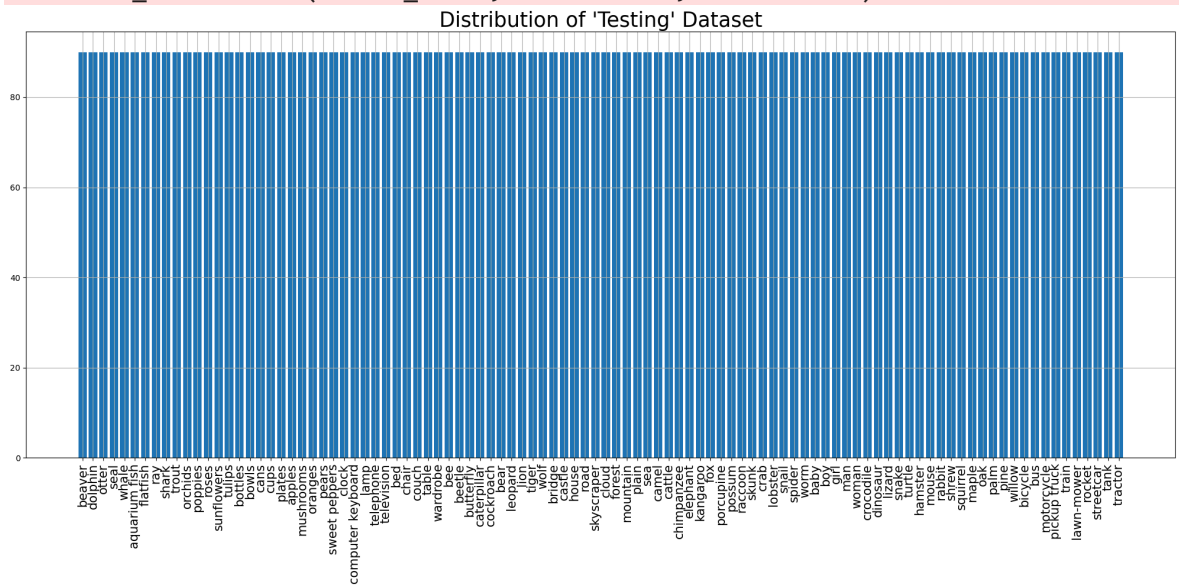
Distribution of 'Training' Dataset



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.  
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



Training dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
```

Validation dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
```

Testing dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
<class 'tuple'>
2
<class 'tensorflow.python.framework.ops.EagerTensor'> <class 'tensorflow.python.framework.ops.EagerTensor'>
(64, 224, 224, 3)
(64, 100)
Model input shape: (None, 224, 224, 3)
Model output shape: (None, 100)
2
<class 'numpy.ndarray'> <class 'numpy.ndarray'>
(64, 224, 224, 3) (64, 100)
```

Model: "sequential_1"

Layer (type)	Output Shape
resnet152v2 (Functional)	(None , 7 , 7 , 2048)
global_average_pooling2d_1 (GlobalAveragePooling2D)	(None , 2048)
dense_2 (Dense)	(None , 512)
dense_3 (Dense)	(None , 100)



Total params: [59,432,036](#) (226.72 MB)

Trainable params: [1,100,388](#) (4.20 MB)

Non-trainable params: [58,331,648](#) (222.52 MB)

Epoch 1/25
657/657 ————— **0s** 62ms/step - accuracy: 0.4782 - f1_score: 0.4758 - loss: 2.0626 - precision: 0.7510
Epoch 1: val_loss improved from inf to 1.35815, saving model to best_model_v1.keras
657/657 ————— **90s** 98ms/step - accuracy: 0.4783 - f1_score: 0.4759 - loss: 2.0619 - precision: 0.7510 - val_accuracy: 0.6156 - val_f1_score: 0.6144 - val_loss: 1.3582 - val_precision: 0.7965 - learning_rate: 0.0010
Epoch 2/25
656/657 ————— **0s** 52ms/step - accuracy: 0.6856 - f1_score: 0.6830 - loss: 1.0891 - precision: 0.8379
Epoch 2: val_loss improved from 1.35815 to 1.29781, saving model to best_model_v1.keras
657/657 ————— **44s** 67ms/step - accuracy: 0.6857 - f1_score: 0.6831 - loss: 1.0889 - precision: 0.8379 - val_accuracy: 0.6308 - val_f1_score: 0.6308 - val_loss: 1.2978 - val_precision: 0.7858 - learning_rate: 0.0010
Epoch 3/25
656/657 ————— **0s** 52ms/step - accuracy: 0.7506 - f1_score: 0.7484 - loss: 0.8669 - precision: 0.8827
Epoch 3: val_loss improved from 1.29781 to 1.29756, saving model to best_model_v1.keras
657/657 ————— **44s** 66ms/step - accuracy: 0.7506 - f1_score: 0.7484 - loss: 0.8667 - precision: 0.8827 - val_accuracy: 0.6332 - val_f1_score: 0.6339 - val_loss: 1.2976 - val_precision: 0.7735 - learning_rate: 0.0010
Epoch 4/25
656/657 ————— **0s** 52ms/step - accuracy: 0.7977 - f1_score: 0.7956 - loss: 0.7100 - precision: 0.9093
Epoch 4: val_loss did not improve from 1.29756
657/657 ————— **41s** 63ms/step - accuracy: 0.7977 - f1_score: 0.7956 - loss: 0.7099 - precision: 0.9093 - val_accuracy: 0.6329 - val_f1_score: 0.6339 - val_loss: 1.3253 - val_precision: 0.7566 - learning_rate: 0.0010
Epoch 5/25
656/657 ————— **0s** 52ms/step - accuracy: 0.8420 - f1_score: 0.8405 - loss: 0.5826 - precision: 0.9327
Epoch 5: val_loss did not improve from 1.29756
657/657 ————— **41s** 63ms/step - accuracy: 0.8420 - f1_score: 0.8405 - loss: 0.5825 - precision: 0.9327 - val_accuracy: 0.6327 - val_f1_score: 0.6331 - val_loss: 1.3629 - val_precision: 0.7480 - learning_rate: 0.0010
Epoch 6/25
656/657 ————— **0s** 52ms/step - accuracy: 0.8759 - f1_score: 0.8745 - loss: 0.4836 - precision: 0.9493
Epoch 6: val_loss did not improve from 1.29756

Epoch 6: ReduceLROnPlateau reducing learning rate to 0.0005000000237487257.
657/657 ————— **41s** 62ms/step - accuracy: 0.8759 - f1_score: 0.8745 - loss: 0.4835 - precision: 0.9493 - val_accuracy: 0.6310 - val_f1_score: 0.6326 - val_loss: 1.4014 - val_precision: 0.7354 - learning_rate: 0.0010
Epoch 7/25
656/657 ————— **0s** 52ms/step - accuracy: 0.8971 - f1_score: 0.8955 - loss: 0.3947 - precision: 0.9536
Epoch 7: val_loss improved from 1.29756 to 1.29246, saving model to best_model_v1.keras
657/657 ————— **44s** 66ms/step - accuracy: 0.8972 - f1_score: 0.8955 - loss: 0.3946 - precision: 0.9536 - val_accuracy: 0.6563 - val_f1_score: 0.6568 - val_loss: 1.2925 - val_precision: 0.7592 - learning_rate: 5.0000e-04
Epoch 8/25
656/657 ————— **0s** 52ms/step - accuracy: 0.9326 - f1_score: 0.9311 - loss: 0.3129 - precision: 0.9757
Epoch 8: val_loss did not improve from 1.29246
657/657 ————— **41s** 62ms/step - accuracy: 0.9327 - f1_score: 0.9311

- loss: 0.3128 - precision: 0.9757 - val_accuracy: 0.6558 - val_f1_score: 0.6564
 - val_loss: 1.2999 - val_precision: 0.7578 - learning_rate: 5.0000e-04

Epoch 9/25

656/657 ————— **0s** 52ms/step - accuracy: 0.9541 - f1_score: 0.9527 -
 loss: 0.2621 - precision: 0.9854

Epoch 9: val_loss did not improve from 1.29246

657/657 ————— **41s** 62ms/step - accuracy: 0.9541 - f1_score: 0.9528
 - loss: 0.2620 - precision: 0.9854 - val_accuracy: 0.6568 - val_f1_score: 0.6571
 - val_loss: 1.3195 - val_precision: 0.7538 - learning_rate: 5.0000e-04

Epoch 10/25

657/657 ————— **0s** 52ms/step - accuracy: 0.9645 - f1_score: 0.9632 -
 loss: 0.2248 - precision: 0.9903

Epoch 10: val_loss did not improve from 1.29246

Epoch 10: ReduceLROnPlateau reducing learning rate to 0.0002500000118743628.

657/657 ————— **41s** 62ms/step - accuracy: 0.9645 - f1_score: 0.9632
 - loss: 0.2248 - precision: 0.9903 - val_accuracy: 0.6539 - val_f1_score: 0.6540
 - val_loss: 1.3372 - val_precision: 0.7493 - learning_rate: 5.0000e-04

Epoch 11/25

656/657 ————— **0s** 52ms/step - accuracy: 0.9721 - f1_score: 0.9706 -
 loss: 0.1988 - precision: 0.9905

Epoch 11: val_loss did not improve from 1.29246

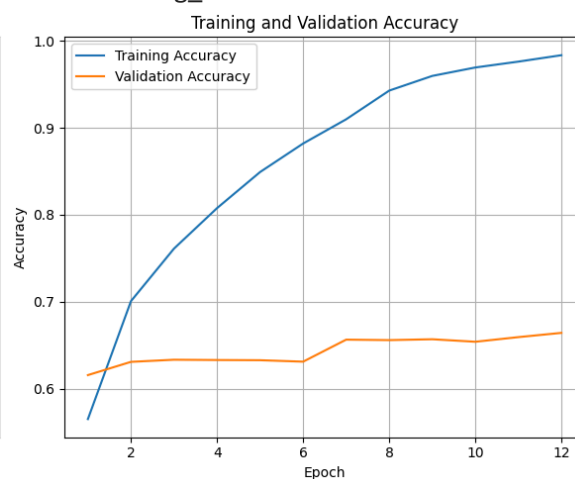
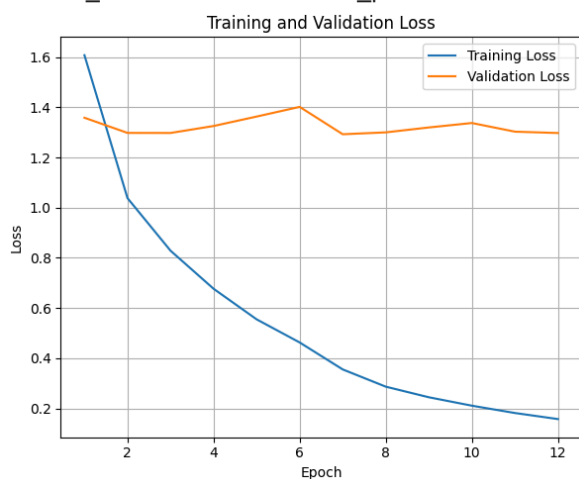
657/657 ————— **41s** 62ms/step - accuracy: 0.9721 - f1_score: 0.9706
 - loss: 0.1987 - precision: 0.9905 - val_accuracy: 0.6592 - val_f1_score: 0.6583
 - val_loss: 1.3027 - val_precision: 0.7547 - learning_rate: 2.5000e-04

Epoch 12/25

656/657 ————— **0s** 52ms/step - accuracy: 0.9798 - f1_score: 0.9782 -
 loss: 0.1728 - precision: 0.9933

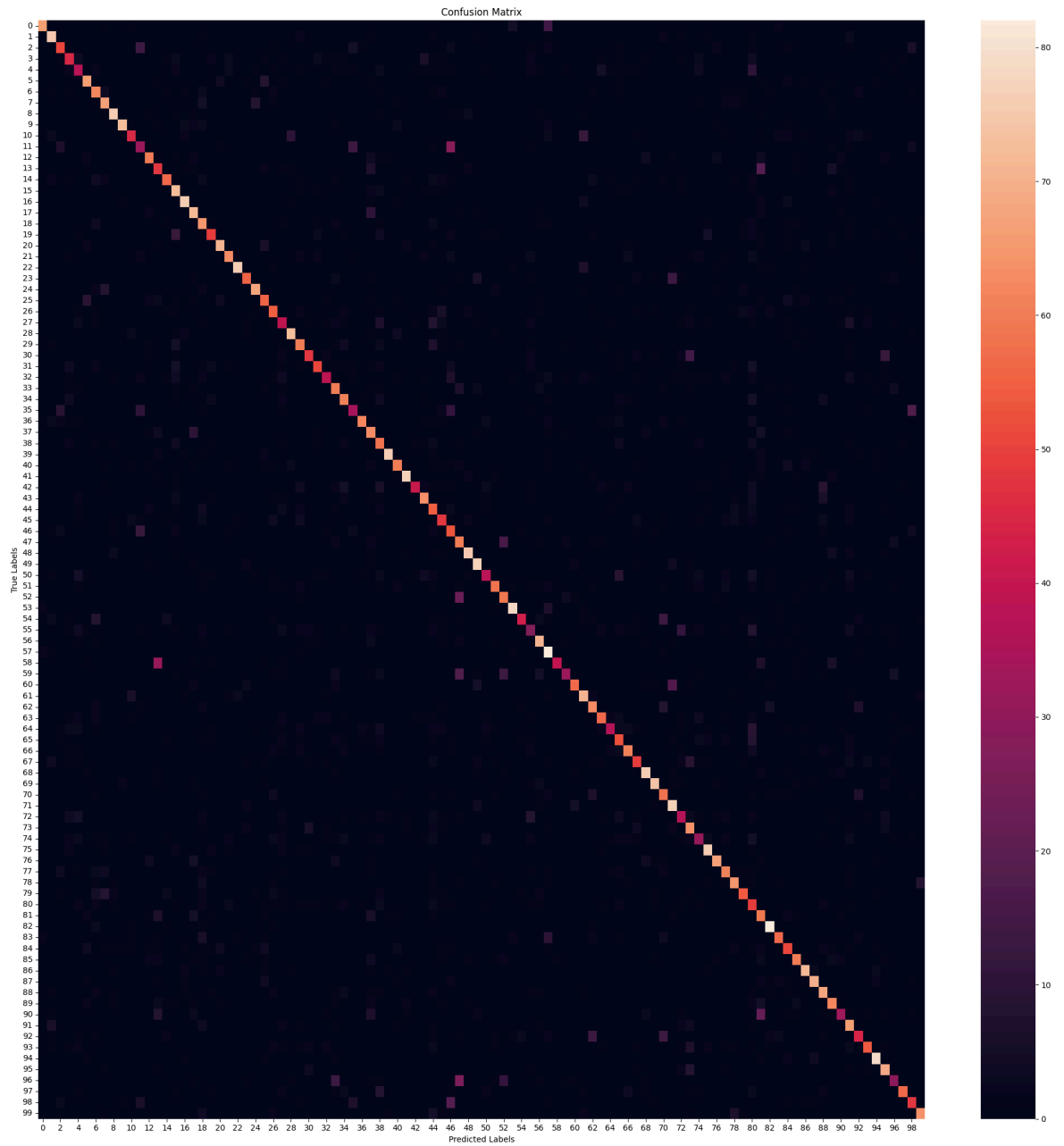
Epoch 12: val_loss did not improve from 1.29246

657/657 ————— **41s** 63ms/step - accuracy: 0.9798 - f1_score: 0.9783
 - loss: 0.1728 - precision: 0.9933 - val_accuracy: 0.6641 - val_f1_score: 0.6631
 - val_loss: 1.2974 - val_precision: 0.7570 - learning_rate: 2.5000e-04



141/141  **25s** 79ms/step - accuracy: 0.6447 - f1_score: 0.6402
- loss: 1.3076 - precision: 0.7553
Test Accuracy: 65.16%
Test Loss: 1.2872965335845947
Test Precision: 75.81%
Test F1 Scores (Per Class): [81.481476 76.923065 58.959538 52.941174 43.181812 71.73912 61.08374
69.791664 83.9779 86.904755 58.441555 37.426895 70.58823 45.933006
68.67469 65.47085 83.060104 70.93597 57.522118 60.869564 80.22598
74.556206 88.888885 69.565216 78.61271 60.869556 59.459454 44.06779
81.11111 64.51612 61.146492 64.15094 53.424656 59.40594 64.893616
43.3121 77.49999 55.752205 51.785706 88.888885 70.17543 86.516846
60.294117 72.72727 48.67256 53.631275 44.53781 50.209206 83.798874
78.57142 51.351345 71.16564 61.13989 86.666664 61.428566 34.3949
73.95833 69.1983 61.068703 47.76119 72.61146 72.82051 65.284966
68.263466 53.237408 53.88601 71.345024 60.24096 82.608696 84.57142
58.291454 72.641495 45.398766 59.259254 39.999996 80.85106 79.28994
68.53932 72.222206 67.92453 37.735847 49.579823 84.97408 70.80744
56.66666 71.428566 79.55801 82.352936 69.74358 66.31015 48.529404
60.550453 54.43037 68.35443 86.33879 71.13402 40.27777 74.99999
53.93258 77.10842]
Average Test F1 Scores:65.29

282/282  **35s** 77ms/step



	precision	recall	f1-score	support
beaver	0.92	0.73	0.81	90
dolphin	0.71	0.83	0.77	90
otter	0.61	0.57	0.59	90
seal	0.56	0.50	0.53	90
whale	0.44	0.42	0.43	90
aquarium fish	0.70	0.73	0.72	90
flatfish	0.55	0.69	0.61	90
ray	0.66	0.74	0.70	90
shark	0.84	0.84	0.84	90
trout	0.94	0.81	0.87	90
orchids	0.70	0.50	0.58	90
poppies	0.40	0.36	0.37	90
roses	0.75	0.67	0.71	90
sunflowers	0.40	0.53	0.46	90
tulips	0.75	0.63	0.69	90
bottles	0.55	0.81	0.65	90
bowls	0.82	0.84	0.83	90
cans	0.64	0.80	0.71	90
cups	0.48	0.72	0.58	90
plates	0.69	0.54	0.61	90
apples	0.82	0.79	0.80	90
mushrooms	0.79	0.70	0.74	90
oranges	0.94	0.84	0.89	90
pears	0.79	0.62	0.70	90
sweet peppers	0.82	0.76	0.79	90
clock	0.60	0.62	0.61	90
computer keyboard	0.58	0.61	0.59	90
lamp	0.45	0.43	0.44	90
telephone	0.81	0.81	0.81	90
television	0.62	0.67	0.65	90
bed	0.72	0.53	0.61	90
chair	0.74	0.57	0.64	90
couch	0.70	0.43	0.53	90
table	0.54	0.67	0.59	90
wardrobe	0.62	0.68	0.65	90
bee	0.51	0.38	0.43	90
beetle	0.89	0.69	0.78	90
butterfly	0.46	0.70	0.56	90
caterpillar	0.43	0.64	0.52	90
cockroach	0.94	0.84	0.89	90
bear	0.74	0.67	0.70	90
leopard	0.88	0.86	0.87	90
lion	0.89	0.46	0.60	90
tiger	0.74	0.71	0.73	90
wolf	0.40	0.61	0.49	90
bridge	0.54	0.53	0.54	90
castle	0.36	0.59	0.45	90
house	0.40	0.67	0.50	90
road	0.84	0.83	0.84	90
skyscraper	0.73	0.86	0.79	90
cloud	0.64	0.42	0.51	90
forest	0.79	0.64	0.71	90
mountain	0.57	0.66	0.61	90
plain	0.87	0.87	0.87	90
sea	0.86	0.48	0.61	90
camel	0.40	0.30	0.34	90
cattle	0.70	0.79	0.74	90
chimpanzee	0.56	0.91	0.69	90

elephant	0.98	0.44	0.61	90
kangaroo	0.73	0.36	0.48	90
fox	0.85	0.63	0.73	90
porcupine	0.68	0.79	0.73	90
possum	0.62	0.70	0.66	90
raccoon	0.74	0.63	0.68	90
skunk	0.76	0.41	0.53	90
crab	0.50	0.58	0.54	90
lobster	0.75	0.68	0.71	90
snail	0.65	0.54	0.59	90
spider	0.81	0.84	0.83	90
worm	0.87	0.82	0.85	90
baby	0.53	0.64	0.58	90
boy	0.63	0.86	0.73	90
girl	0.51	0.41	0.45	90
man	0.51	0.71	0.59	90
woman	0.48	0.34	0.40	90
crocodile	0.78	0.84	0.81	90
dinosaur	0.85	0.74	0.79	90
lizard	0.69	0.68	0.69	90
snake	0.72	0.72	0.72	90
turtle	0.78	0.60	0.68	90
hamster	0.28	0.54	0.37	90
mouse	0.40	0.66	0.50	90
rabbit	0.80	0.91	0.85	90
shrew	0.80	0.63	0.71	90
squirrel	0.57	0.57	0.57	90
maple	0.77	0.67	0.71	90
oak	0.79	0.80	0.80	90
palm	0.88	0.78	0.82	90
pine	0.65	0.76	0.70	90
willow	0.64	0.69	0.66	90
bicycle	0.72	0.37	0.49	90
bus	0.51	0.73	0.60	90
motorcycle	0.64	0.49	0.55	90
pickup truck	0.79	0.60	0.68	90
train	0.85	0.88	0.86	90
lawn-mower	0.66	0.77	0.71	90
rocket	0.54	0.32	0.40	90
streetcar	0.92	0.63	0.75	90
tank	0.55	0.53	0.54	90
tractor	0.84	0.71	0.77	90
accuracy			0.65	9000
macro avg	0.68	0.65	0.65	9000
weighted avg	0.68	0.65	0.65	9000

Model: "sequential_1"

Layer (type)	Output Shape
resnet152v2 (Functional)	(None , 7 , 7 , 2048)
global_average_pooling2d_1 (GlobalAveragePooling2D)	(None , 2048)
dense_2 (Dense)	(None , 512)
dense_3 (Dense)	(None , 100)



Total params: 59,432,036 (226.72 MB)
Trainable params: 1,100,388 (4.20 MB)
Non-trainable params: 58,331,648 (222.52 MB)

Epoch 1/15
1314/1314 ————— **0s** 58ms/step - accuracy: 0.8988 - f1_score: 0.8982
- loss: 0.3707 - precision: 0.9515
Epoch 1: val_loss improved from inf to 1.19345, saving model to best_model_fine_tune_v1.keras
1314/1314 ————— **127s** 77ms/step - accuracy: 0.8988 - f1_score: 0.8982 - loss: 0.3708 - precision: 0.9515 - val_accuracy: 0.6772 - val_f1_score: 0.6760 - val_loss: 1.1934 - val_precision: 0.7836 - learning_rate: 1.0000e-05

Epoch 2/15
1313/1314 ————— **0s** 53ms/step - accuracy: 0.9217 - f1_score: 0.9210
- loss: 0.3152 - precision: 0.9649
Epoch 2: val_loss improved from 1.19345 to 1.18745, saving model to best_model_fine_tune_v1.keras
1314/1314 ————— **80s** 61ms/step - accuracy: 0.9217 - f1_score: 0.9209 - loss: 0.3152 - precision: 0.9649 - val_accuracy: 0.6797 - val_f1_score: 0.6783 - val_loss: 1.1874 - val_precision: 0.7827 - learning_rate: 1.0000e-05

Epoch 3/15
1314/1314 ————— **0s** 53ms/step - accuracy: 0.9290 - f1_score: 0.9282
- loss: 0.2974 - precision: 0.9689
Epoch 3: val_loss improved from 1.18745 to 1.18403, saving model to best_model_fine_tune_v1.keras
1314/1314 ————— **80s** 61ms/step - accuracy: 0.9290 - f1_score: 0.9282 - loss: 0.2975 - precision: 0.9689 - val_accuracy: 0.6798 - val_f1_score: 0.6782 - val_loss: 1.1840 - val_precision: 0.7820 - learning_rate: 1.0000e-05

Epoch 4/15
1313/1314 ————— **0s** 53ms/step - accuracy: 0.9331 - f1_score: 0.9322
- loss: 0.2870 - precision: 0.9712
Epoch 4: val_loss improved from 1.18403 to 1.18097, saving model to best_model_fine_tune_v1.keras
1314/1314 ————— **79s** 60ms/step - accuracy: 0.9330 - f1_score: 0.9322 - loss: 0.2871 - precision: 0.9711 - val_accuracy: 0.6832 - val_f1_score: 0.6818 - val_loss: 1.1810 - val_precision: 0.7826 - learning_rate: 1.0000e-05

Epoch 5/15
1313/1314 ————— **0s** 53ms/step - accuracy: 0.9366 - f1_score: 0.9358
- loss: 0.2789 - precision: 0.9730
Epoch 5: val_loss improved from 1.18097 to 1.18057, saving model to best_model_fine_tune_v1.keras
1314/1314 ————— **79s** 60ms/step - accuracy: 0.9366 - f1_score: 0.9358 - loss: 0.2790 - precision: 0.9730 - val_accuracy: 0.6832 - val_f1_score: 0.6818 - val_loss: 1.1806 - val_precision: 0.7815 - learning_rate: 1.0000e-05

Epoch 6/15
1313/1314 ————— **0s** 53ms/step - accuracy: 0.9394 - f1_score: 0.9386
- loss: 0.2720 - precision: 0.9743
Epoch 6: val_loss improved from 1.18057 to 1.18005, saving model to best_model_fine_tune_v1.keras
1314/1314 ————— **79s** 60ms/step - accuracy: 0.9394 - f1_score: 0.9386 - loss: 0.2721 - precision: 0.9743 - val_accuracy: 0.6829 - val_f1_score: 0.6813 - val_loss: 1.1800 - val_precision: 0.7816 - learning_rate: 1.0000e-05

Epoch 7/15
1313/1314 ————— **0s** 53ms/step - accuracy: 0.9421 - f1_score: 0.9413
- loss: 0.2646 - precision: 0.9753
Epoch 7: val_loss improved from 1.18005 to 1.17902, saving model to best_model_fine_tune_v1.keras
1314/1314 ————— **79s** 60ms/step - accuracy: 0.9420 - f1_score: 0.9413 - loss: 0.2647 - precision: 0.9752 - val_accuracy: 0.6844 - val_f1_score: 0.6829 - val_loss: 1.1790 - val_precision: 0.7797 - learning_rate: 1.0000e-05

Epoch 8/15
1313/1314 ————— **0s** 53ms/step - accuracy: 0.9444 - f1_score: 0.9436
- loss: 0.2591 - precision: 0.9762
Epoch 8: val_loss did not improve from 1.17902

1314/1314 ————— **77s** 58ms/step - accuracy: 0.9443 - f1_score: 0.9436 - loss: 0.2592 - precision: 0.9762 - val_accuracy: 0.6833 - val_f1_score: 0.6819 - val_loss: 1.1794 - val_precision: 0.7805 - learning_rate: 1.0000e-05
Epoch 9/15

1313/1314 ————— **0s** 52ms/step - accuracy: 0.9459 - f1_score: 0.9451 - loss: 0.2549 - precision: 0.9763
Epoch 9: val_loss improved from 1.17902 to 1.17870, saving model to best_model_fine_tune_v1.keras

1314/1314 ————— **77s** 59ms/step - accuracy: 0.9459 - f1_score: 0.9451 - loss: 0.2550 - precision: 0.9762 - val_accuracy: 0.6846 - val_f1_score: 0.6833 - val_loss: 1.1787 - val_precision: 0.7810 - learning_rate: 1.0000e-05
Epoch 10/15

1314/1314 ————— **0s** 52ms/step - accuracy: 0.9472 - f1_score: 0.9465 - loss: 0.2498 - precision: 0.9774
Epoch 10: val_loss improved from 1.17870 to 1.17862, saving model to best_model_fine_tune_v1.keras

1314/1314 ————— **79s** 60ms/step - accuracy: 0.9472 - f1_score: 0.9465 - loss: 0.2498 - precision: 0.9774 - val_accuracy: 0.6852 - val_f1_score: 0.6838 - val_loss: 1.1786 - val_precision: 0.7811 - learning_rate: 1.0000e-05
Epoch 11/15

1313/1314 ————— **0s** 52ms/step - accuracy: 0.9494 - f1_score: 0.9487 - loss: 0.2442 - precision: 0.9784
Epoch 11: val_loss did not improve from 1.17862

1314/1314 ————— **75s** 57ms/step - accuracy: 0.9494 - f1_score: 0.9486 - loss: 0.2443 - precision: 0.9784 - val_accuracy: 0.6864 - val_f1_score: 0.6849 - val_loss: 1.1788 - val_precision: 0.7808 - learning_rate: 1.0000e-05
Epoch 12/15

1314/1314 ————— **0s** 52ms/step - accuracy: 0.9510 - f1_score: 0.9503 - loss: 0.2395 - precision: 0.9786
Epoch 12: val_loss did not improve from 1.17862

Epoch 12: ReduceLROnPlateau reducing learning rate to 4.999999873689376e-06.

1314/1314 ————— **75s** 57ms/step - accuracy: 0.9510 - f1_score: 0.9503 - loss: 0.2395 - precision: 0.9786 - val_accuracy: 0.6874 - val_f1_score: 0.6860 - val_loss: 1.1800 - val_precision: 0.7808 - learning_rate: 1.0000e-05
Epoch 13/15

1314/1314 ————— **0s** 52ms/step - accuracy: 0.9525 - f1_score: 0.9517 - loss: 0.2350 - precision: 0.9800
Epoch 13: val_loss did not improve from 1.17862

1314/1314 ————— **75s** 57ms/step - accuracy: 0.9525 - f1_score: 0.9517 - loss: 0.2351 - precision: 0.9800 - val_accuracy: 0.6873 - val_f1_score: 0.6861 - val_loss: 1.1791 - val_precision: 0.7814 - learning_rate: 5.0000e-06
Epoch 14/15

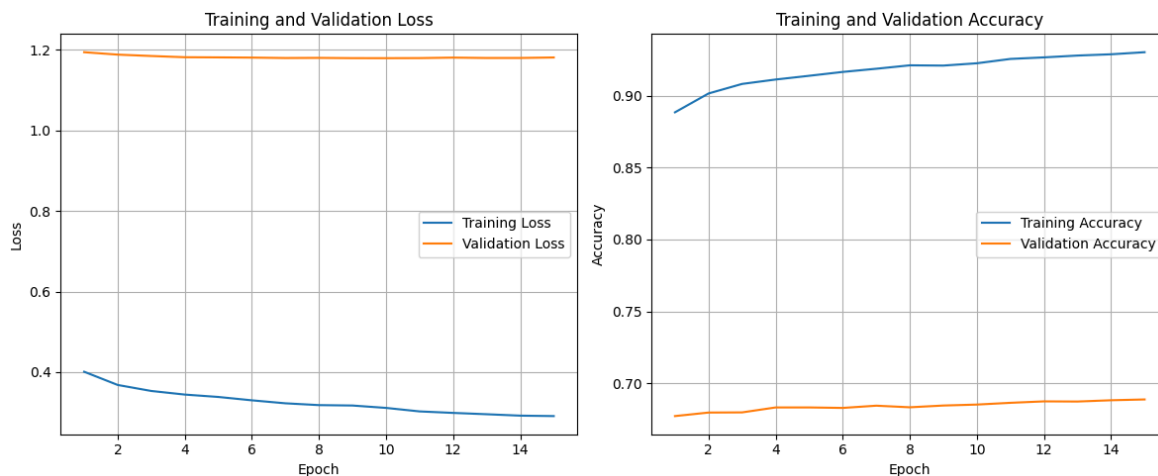
1314/1314 ————— **0s** 52ms/step - accuracy: 0.9537 - f1_score: 0.9530 - loss: 0.2322 - precision: 0.9801
Epoch 14: val_loss did not improve from 1.17862

1314/1314 ————— **75s** 57ms/step - accuracy: 0.9537 - f1_score: 0.9530 - loss: 0.2322 - precision: 0.9801 - val_accuracy: 0.6882 - val_f1_score: 0.6870 - val_loss: 1.1791 - val_precision: 0.7813 - learning_rate: 5.0000e-06
Epoch 15/15

1313/1314 ————— **0s** 52ms/step - accuracy: 0.9544 - f1_score: 0.9536 - loss: 0.2306 - precision: 0.9807
Epoch 15: val_loss did not improve from 1.17862

Epoch 15: ReduceLROnPlateau reducing learning rate to 2.499999936844688e-06.

1314/1314 ————— **75s** 57ms/step - accuracy: 0.9543 - f1_score: 0.9536 - loss: 0.2307 - precision: 0.9806 - val_accuracy: 0.6888 - val_f1_score: 0.6875 - val_loss: 1.1804 - val_precision: 0.7806 - learning_rate: 5.0000e-06



141/141 ————— **25s** 81ms/step - accuracy: 0.6732 - f1_score: 0.6660

- loss: 1.2033 - precision: 0.7724

Test Accuracy: 68.09%

Test Loss: 1.1775946617126465

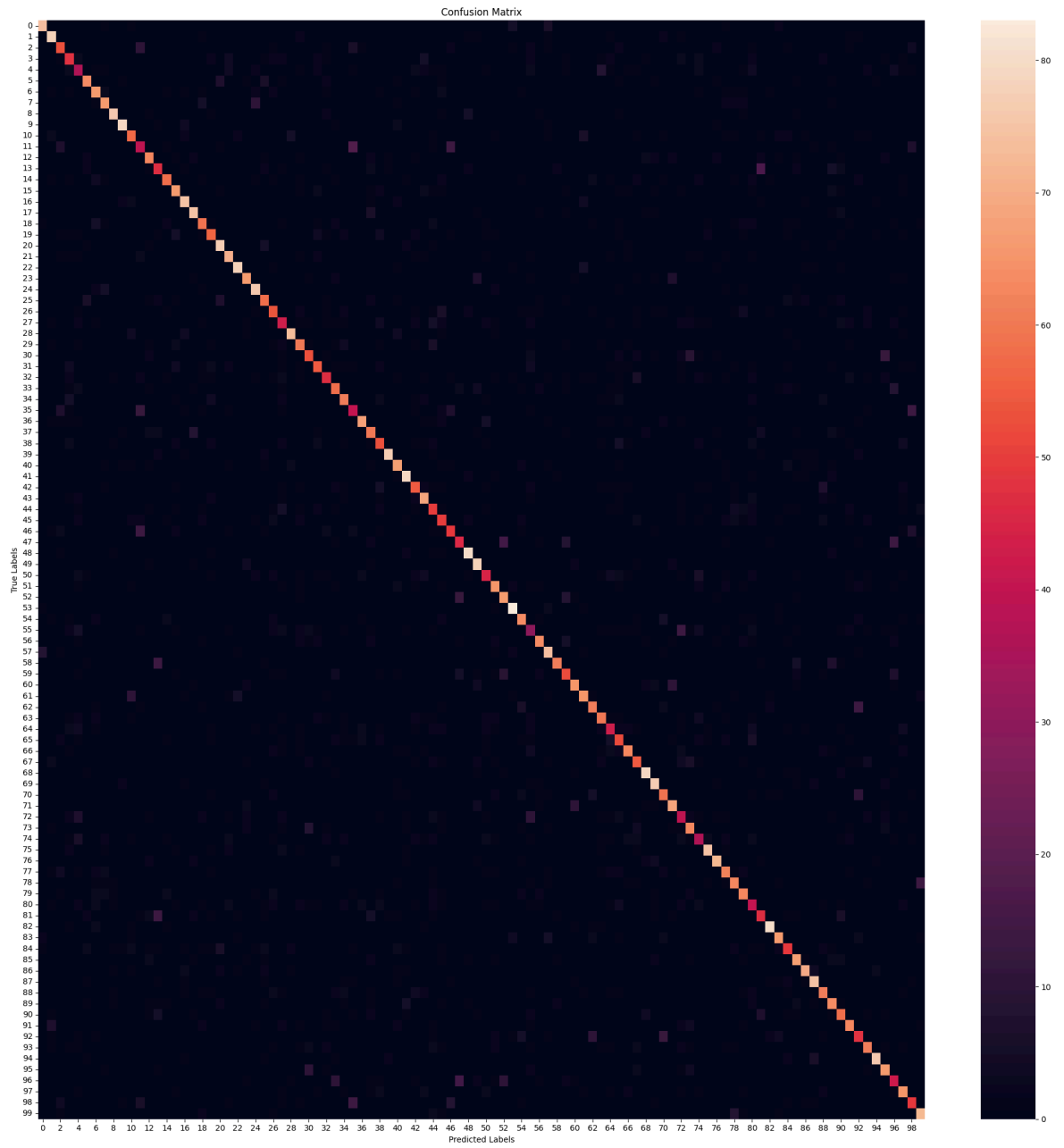
Test Precision: 78.06%

Test F1 Scores (Per Class): [83.615814 80.41236 59.550552 53.93258 39.779003 72.72727 69.84126

72.92817 86.36363 88.76403 64.40677 42.857136 70.39106 51.933693
 70.303024 72.52747 79.787224 81.08108 66.666664 64.739876 79.79274
 75.675674 87.15083 75.428566 79.99999 64.44444 59.66851 48.863632
 83.42856 65.93406 60.919537 63.5838 56.6265 62.032085 68.53932
 44.943817 79.069756 64.51612 58.24175 89.534874 71.276596 83.15788
 61.797745 72.63157 51.813465 54.64481 56.32183 53.48836 85.56149
 78.78787 54.54545 71.03825 68.74999 86.910995 69.94535 34.28571
 74.4186 77.08333 77.21518 59.770107 75.14451 74.28571 68.926544
 67.796616 51.190464 55.319138 72.83237 60.109283 84.946236 81.481476
 64.83515 74.59459 43.093918 67.02127 42.857136 84.26965 81.35593
 69.66292 68.852455 69.23077 42.780743 53.179188 88.39778 75.428566
 59.036137 70.89946 80.92485 81.96721 68.53932 66.666664 63.73626
 71.590904 53.93258 70.11494 87.86127 71.35134 47.45762 69.89246
 55.681812 77.41935]

Average Test F1 Scores:67.99

282/282 ————— **26s** 63ms/step



	precision	recall	f1-score	support
beaver	0.85	0.82	0.84	90
dolphin	0.75	0.87	0.80	90
otter	0.60	0.59	0.60	90
seal	0.55	0.53	0.54	90
whale	0.40	0.40	0.40	90
aquarium fish	0.74	0.71	0.73	90
flatfish	0.67	0.73	0.70	90
ray	0.73	0.73	0.73	90
shark	0.88	0.84	0.86	90
trout	0.90	0.88	0.89	90
orchids	0.66	0.63	0.64	90
poppies	0.42	0.43	0.43	90
roses	0.71	0.70	0.70	90
sunflowers	0.52	0.52	0.52	90
tulips	0.77	0.64	0.70	90
bottles	0.72	0.73	0.73	90
bowls	0.77	0.83	0.80	90
cans	0.79	0.83	0.81	90
cups	0.68	0.66	0.67	90
plates	0.67	0.62	0.65	90
apples	0.75	0.86	0.80	90
mushrooms	0.74	0.79	0.76	90
oranges	0.88	0.87	0.87	90
pears	0.78	0.73	0.75	90
sweet peppers	0.76	0.84	0.80	90
clock	0.64	0.64	0.64	90
computer keyboard	0.59	0.60	0.60	90
lamp	0.49	0.48	0.49	90
telephone	0.86	0.81	0.83	90
television	0.65	0.67	0.66	90
bed	0.63	0.59	0.61	90
chair	0.66	0.61	0.64	90
couch	0.63	0.52	0.57	90
table	0.60	0.64	0.62	90
wardrobe	0.69	0.68	0.69	90
bee	0.45	0.44	0.45	90
beetle	0.83	0.76	0.79	90
butterfly	0.62	0.67	0.65	90
caterpillar	0.58	0.59	0.58	90
cockroach	0.94	0.86	0.90	90
bear	0.68	0.74	0.71	90
leopard	0.79	0.88	0.83	90
lion	0.62	0.61	0.62	90
tiger	0.69	0.77	0.73	90
wolf	0.49	0.56	0.52	90
bridge	0.54	0.56	0.55	90
castle	0.58	0.54	0.56	90
house	0.56	0.51	0.53	90
road	0.82	0.89	0.86	90
skyscraper	0.72	0.87	0.79	90
cloud	0.60	0.50	0.55	90
forest	0.70	0.72	0.71	90
mountain	0.65	0.73	0.69	90
plain	0.82	0.92	0.87	90
sea	0.69	0.71	0.70	90
camel	0.35	0.33	0.34	90
cattle	0.78	0.71	0.74	90
chimpanzee	0.73	0.82	0.77	90

elephant	0.90	0.68	0.77	90
kangaroo	0.62	0.58	0.60	90
fox	0.78	0.72	0.75	90
porcupine	0.76	0.72	0.74	90
possum	0.70	0.68	0.69	90
raccoon	0.69	0.67	0.68	90
skunk	0.55	0.48	0.51	90
crab	0.53	0.58	0.55	90
lobster	0.77	0.70	0.73	90
snail	0.59	0.61	0.60	90
spider	0.82	0.88	0.85	90
worm	0.77	0.86	0.81	90
baby	0.64	0.66	0.65	90
boy	0.73	0.77	0.75	90
girl	0.43	0.43	0.43	90
man	0.64	0.70	0.67	90
woman	0.46	0.40	0.43	90
crocodile	0.85	0.83	0.84	90
dinosaur	0.83	0.80	0.81	90
lizard	0.70	0.69	0.70	90
snake	0.68	0.70	0.69	90
turtle	0.68	0.70	0.69	90
hamster	0.41	0.44	0.43	90
mouse	0.56	0.52	0.54	90
rabbit	0.88	0.89	0.88	90
shrew	0.78	0.73	0.75	90
squirrel	0.64	0.54	0.59	90
maple	0.68	0.74	0.71	90
oak	0.84	0.78	0.81	90
palm	0.81	0.83	0.82	90
pine	0.69	0.68	0.69	90
willow	0.63	0.71	0.67	90
bicycle	0.64	0.64	0.64	90
bus	0.73	0.70	0.72	90
motorcycle	0.55	0.53	0.54	90
pickup truck	0.73	0.68	0.70	90
train	0.92	0.84	0.88	90
lawn-mower	0.69	0.73	0.71	90
rocket	0.48	0.47	0.47	90
streetcar	0.68	0.72	0.70	90
tank	0.57	0.54	0.56	90
tractor	0.75	0.80	0.77	90
accuracy			0.68	9000
macro avg	0.68	0.68	0.68	9000
weighted avg	0.68	0.68	0.68	9000

X_temp.shape: (8000, 32, 32, 3)

X_temp_val.shape: (4000, 32, 32, 3)

y_temp_val.shape: (4000, 1)

X_temp_test.shape: (4000, 32, 32, 3)

y_temp_test.shape: (4000, 1)

X_train.shape: (42000, 32, 32, 3)

y_train.shape: (42000, 1)

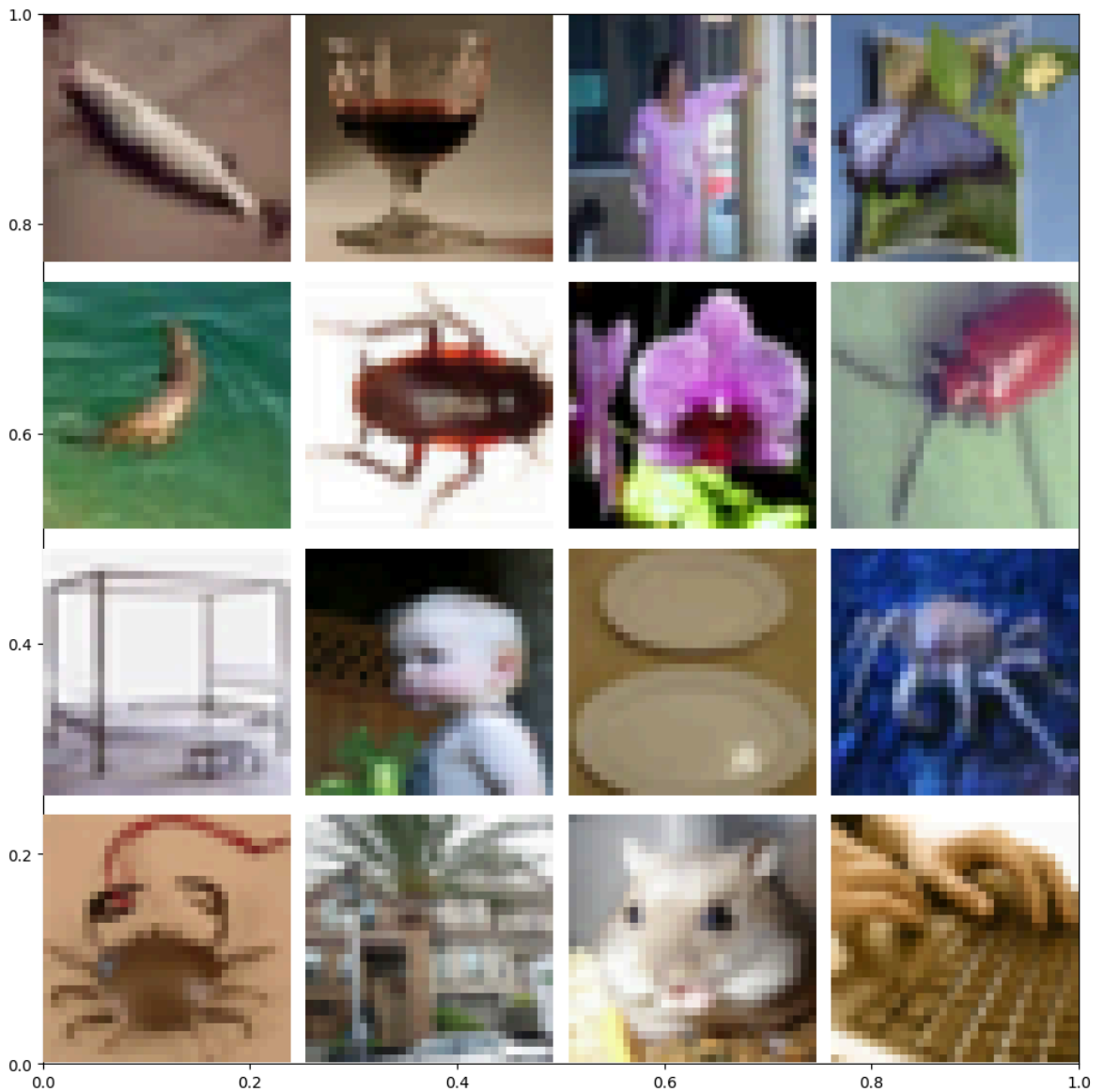
X_val.shape: (9000, 32, 32, 3)

y_val.shape: (9000, 1)

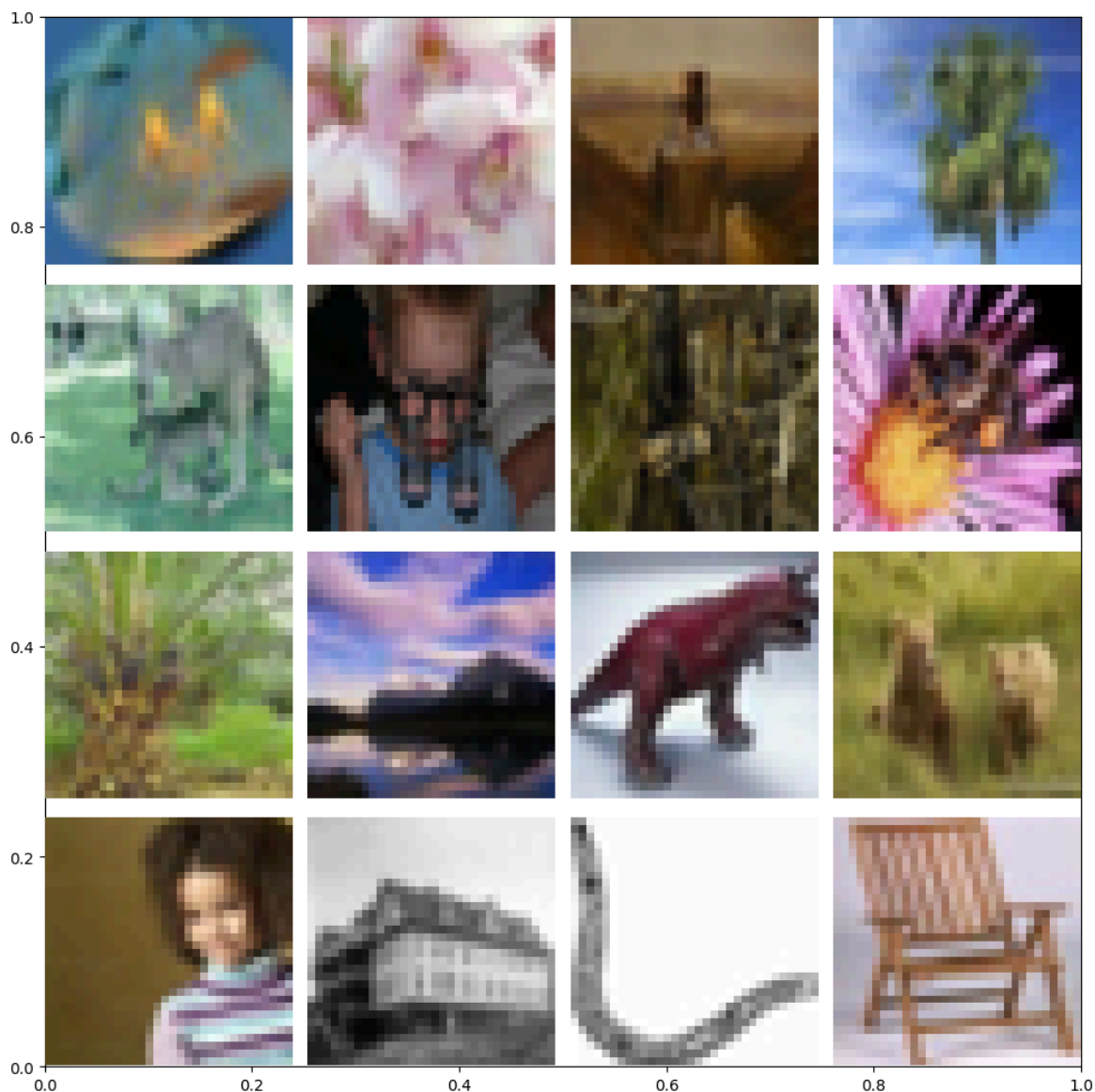
X_test.shape: (9000, 32, 32, 3)

y_test.shape: (9000, 1)

```
labels[22918].shape: (1,)  
imgs[22918].shape: (32, 32, 3)
```

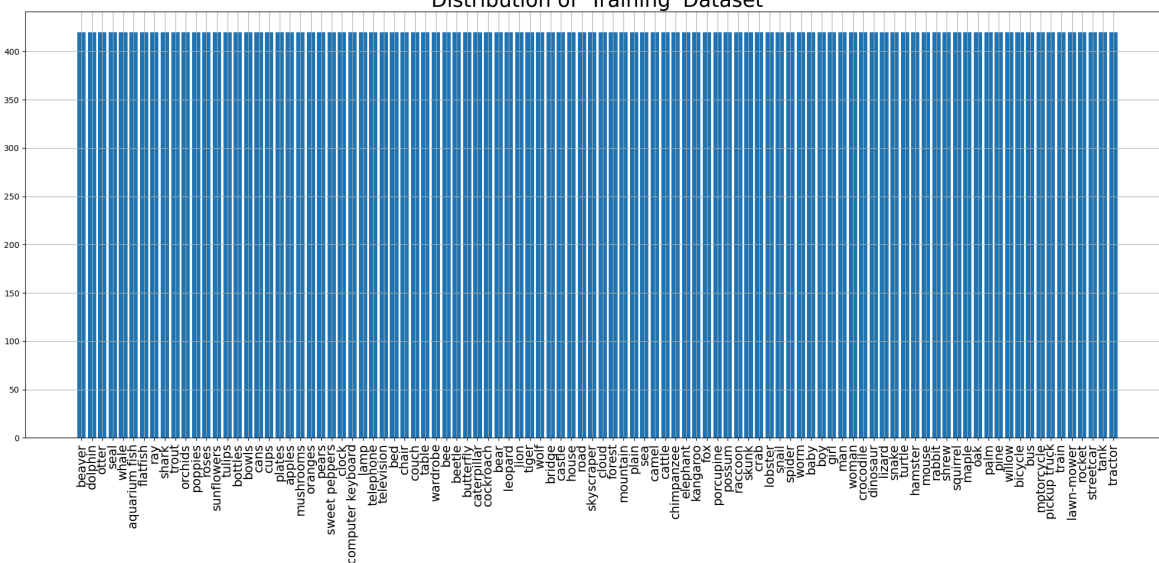


```
labels[40043].shape: (1,)  
imgs[40043].shape: (32, 32, 3)
```

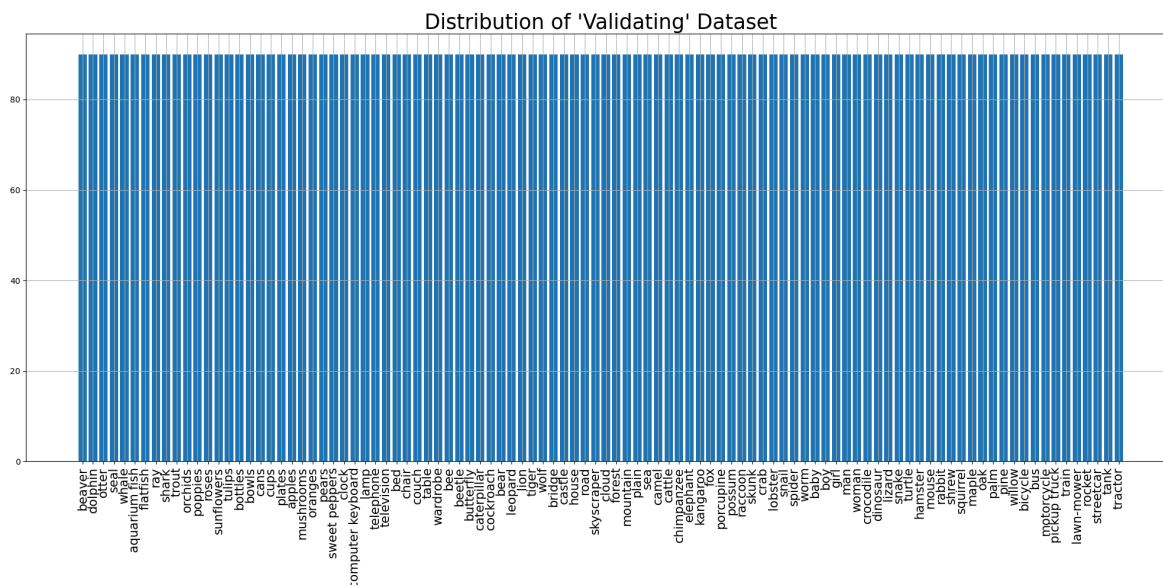


```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```

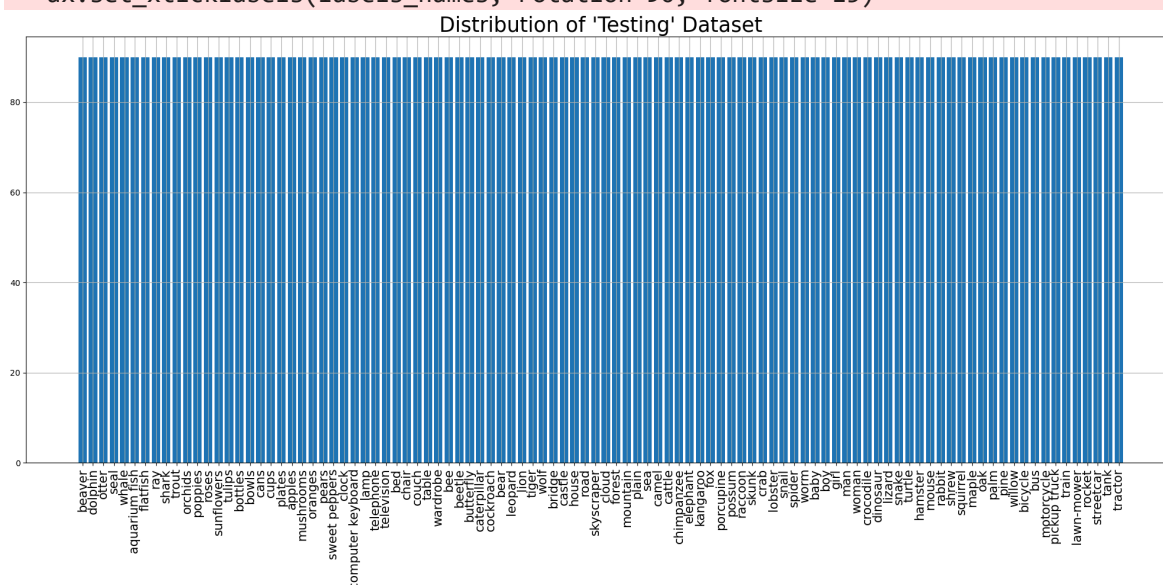
Distribution of 'Training' Dataset



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



Training dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
```

Validation dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
```

Testing dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
```

```

-----
KeyboardInterrupt                                Traceback (most recent call last)
<ipython-input-6-2c2ab8b5bc6b> in <cell line: 0>()
    105     ##### <<<<<<<<Pre-trained model>>>>>>>>>>>>
    106     # Load ResNet50 pre-trained on ImageNet (w/out the top classification
layer which is designed for ImageNet (diff dataset))
--> 107     resnet_50_base = tf.keras.applications.ResNet152V2(weights='imagenet',
include_top=False, input_shape=(224, 224, 3))
    108
    109     # Freeze the layers of VGG16 so they don't get updated during training - can unfreeze for fine tuning later

/usr/local/lib/python3.11/dist-packages/keras/src/applications/resnet_v2.py in ResNet152V2(include_top, weights, input_tensor, input_shape, pooling, classes, classifier_activation, name)
    114         )
    115
--> 116     return resnet.ResNet(
    117         stack_fn,
    118         True,

/usr/local/lib/python3.11/dist-packages/keras/src/applications/resnet.py in ResNet(stack_fn, preact, use_bias, include_top, weights, input_tensor, input_shape, pooling, classes, classifier_activation, name, weights_name)
    210         file_hash=file_hash,
    211     )
--> 212     model.load_weights(weights_path)
    213     elif weights is not None:
    214         model.load_weights(weights)

/usr/local/lib/python3.11/dist-packages/keras/src/utils/traceback_utils.py in error_handler(*args, **kwargs)
    115     filtered_tb = None
    116     try:
--> 117         return fn(*args, **kwargs)
    118     except Exception as e:
    119         filtered_tb = _process_traceback_frames(e.__traceback__)

/usr/local/lib/python3.11/dist-packages/keras/src/models/model.py in load_weights(self, filepath, skip_mismatch, **kwargs)
    353         the shape of the weights.
    354         """
--> 355         saving_api.load_weights(
    356             self, filepath, skip_mismatch=skip_mismatch, **kwargs
    357         )

/usr/local/lib/python3.11/dist-packages/keras/src/saving/saving_api.py in load_weights(model, filepath, skip_mismatch, **kwargs)
    271         )
    272         else:
--> 273             legacy_h5_format.load_weights_from_hdf5_group(f, model)
    274     else:
    275         raise ValueError(

/usr/local/lib/python3.11/dist-packages/keras/src/legacy/saving/legacy_h5_format.py in load_weights_from_hdf5_group(f, model)
    350     for name in layer_names:
    351         g = f[name]
--> 352         weight_names = load_attributes_from_hdf5_group(g, "weight_names")
    353         if weight_names:

```

```

354         filtered_layer_names.append(name)

/usr/local/lib/python3.11/dist-packages/keras/src/legacy/saving/legacy_h5_format.
py in load_attributes_from_hdf5_group(group, name)
    606         data = [
    607             n.decode("utf8") if hasattr(n, "decode") else n
--> 608             for n in group.attrs[name]
    609         ]
    610     else:

h5py/_objects.pyx in h5py._objects.with_phil.wrapper()

h5py/_objects.pyx in h5py._objects.with_phil.wrapper()

/usr/local/lib/python3.11/dist-packages/h5py/_hl/attrs.py in __getitem__(self, na
me)
    61         return Empty(attr.dtype)
    62
--> 63         dtype = attr.dtype
    64
    65         # Do this first, as we'll be fiddling with the dtype for top-level
1

h5py/h5a.pyx in h5py.h5a.AttrID.dtype.__get__()

h5py/h5a.pyx in h5py.h5a.AttrID.dtype.__get__()

h5py/h5t.pyx in h5py.h5t.TypeStringID.py_dtype()

h5py/h5t.pyx in h5py.h5t.string_dtype()

KeyboardInterrupt:

```

Dropout(0.6), weight_decay(1e-1), l2(1e-1) lr

```

In [7]: for repeat_2_times in range(2):
        ##### <<<<<<<<<Load and process data>>>>>>>>>>>>
        # Load CIFAR-100 dataset
        (X_train, y_train), (X_test, y_test) = cifar100.load_data(label_mode='fine')

        # Split (8000) of training data into temporary set
        X_temp, X_train, y_temp, y_train = train_test_split(X_train, y_train, test_s
        print(f"X_temp.shape: {X_temp.shape}\n")

        # Split temp data into equal validation (4000) and testing (4000) data
        X_temp_val, X_temp_test, y_temp_val, y_temp_test = train_test_split(X_temp,
        print(f"X_temp_val.shape: {X_temp_val.shape}")
        print(f"y_temp_val.shape: {y_temp_val.shape}")
        print(f"X_temp_test.shape: {X_temp_test.shape}")
        print(f"y_temp_test.shape: {y_temp_test.shape}\n")

        # Split test data into validation (5000) and testing (5000)
        X_val, X_test, y_val, y_test = train_test_split(X_test, y_test, test_size=0.

        # Add temp_val to validation (9000) and temp_test to testing (9000) to get a
        X_val = np.concatenate((X_val, X_temp_val), axis=0)

```

```

y_val = np.concatenate((y_val, y_temp_val), axis=0)
X_test = np.concatenate((X_test, X_temp_test), axis=0)
y_test = np.concatenate((y_test, y_temp_test), axis=0)

print(f"X_train.shape: {X_train.shape}")
print(f"y_train.shape: {y_train.shape}")
print(f"X_val.shape: {X_val.shape}")
print(f"y_val.shape: {y_val.shape}")
print(f"X_test.shape: {X_test.shape}")
print(f"y_test.shape: {y_test.shape}\n")

display_imgs(X_train, y_train)

# Normalise images (scale to range [0, 1]) - Improves convergence speed & accuracy
X_train, X_val, X_test = X_train / 255.0, X_val / 255.0, X_test / 255.0
display_imgs(X_train, y_train)

labels_names = ['beaver', 'dolphin', 'otter', 'seal', 'whale', 'aquarium fish', 'fish',
                 'orchids', 'poppies', 'roses', 'sunflowers', 'tulips', 'bottles', 'bottle',
                 'apples', 'mushrooms', 'oranges', 'pears', 'sweet peppers', 'clock',
                 'telephone', 'television', 'bed', 'chair', 'couch', 'table', 'wardrobe',
                 'caterpillar', 'cockroach', 'bear', 'leopard', 'lion', 'tiger', 'wolf',
                 'road', 'skyscraper', 'cloud', 'forest', 'mountain', 'plain', 'sea',
                 'elephant', 'kangaroo', 'fox', 'porcupine', 'possum', 'raccoon', 'squirrel',
                 'spider', 'worm', 'baby', 'boy', 'girl', 'man', 'woman', 'crocodile',
                 'turtle', 'hamster', 'mouse', 'rabbit', 'shrew', 'squirrel', 'maple',
                 'bicycle', 'bus', 'motorcycle', 'pickup truck', 'train', 'lawn-mower',
                 'tractor']

class_distrib(y_train, labels_names, "Training")
class_distrib(y_val, labels_names, "Validating")
class_distrib(y_test, labels_names, "Testing")

# Create TensorFlow datasets

batch_size = 64

train_dataset_ = (tf.data.Dataset.from_tensor_slices((X_train, y_train))
                  .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                       tf.squeeze(tf.one_hot(y, depth=100, dtype=tf.int32)))
                  .batch(batch_size)
                  .prefetch(tf.data.experimental.AUTOTUNE))

# DEBUG augment
#train_dataset_aug = (tf.data.Dataset.from_tensor_slices((X_train, y_train))
#                    .map(augment_dataset, num_parallel_calls=tf.data.experimental.DEFAULT)
#                    .batch(batch_size)
#                    .prefetch(tf.data.experimental.AUTOTUNE))

# Combine the original dataset and the augmented dataset
train_dataset = train_dataset_.concatenate(train_dataset_aug)

val_dataset = (tf.data.Dataset.from_tensor_slices((X_val, y_val))
               .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                   tf.squeeze(tf.one_hot(y, depth=100, dtype=tf.int32)))
               .batch(batch_size)
               .prefetch(tf.data.experimental.AUTOTUNE))

test_dataset = (tf.data.Dataset.from_tensor_slices((X_test, y_test))
                .map(lambda x, y: (tf.image.resize(x, (224, 224)),

```

```

        tf.squeeze(tf.one_hot(y, depth=100, dtype=
        .batch(batch_size)
        .prefetch(tf.data.experimental.AUTOTUNE))

print(f"Training dataset:\n {train_dataset}")
for img, lbl in train_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}") # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}") # Should be (batch_size, 10)
    del img, lbl
print(f"\nValidation dataset:\n {val_dataset}")
for img, lbl in val_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}") # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}") # Should be (batch_size, 10)
    del img, lbl
print(f"\nTesting dataset:\n {test_dataset}")
for img, lbl in test_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}") # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}") # Should be (batch_size, 10)
    del img, lbl

#### <<<<<<<<<Pre-trained model>>>>>>>>
# Load ResNet50 pre-trained on ImageNet (w/out the top classification layer
resnet_50_base = tf.keras.applications.ResNet50V2(weights='imagenet', includ

# Freeze the Layers of VGG16 so they don't get updated during training - can
resnet_50_base.trainable = False

for layer in resnet_50_base.layers:
    if isinstance(layer, tf.keras.layers.Conv2D):
        layer.kernel_regularizer = tf.keras.regularizers.l2(1e-1) # DEBUG L2

# Add custom classification layers for CIFAR-100 (100 classes) - adapt model
model = models.Sequential([
    resnet_50_base,
    layers.GlobalAveragePooling2D(), # Better for ResNet than Flatten
    layers.Dense(512, activation='relu'),
    layers.Dropout(0.6), # DEBUG dropout
    layers.Dense(100, activation='softmax') # CIFAR-100 has 100 classes
])

for sample in test_dataset.take(1):
    print(type(sample)) # Should be <class 'tuple'>
    print(len(sample)) # Should be 2
    print(type(sample[0]), type(sample[1])) # Both should be <class 'tensorflow.Tensor'>
    print(sample[0].shape) # Should be (batch_size, 224, 224, 3)
    print(sample[1].shape) # Should be (batch_size, 100)
print(f"Model input shape: {model.input_shape}")
print(f"Model output shape: {model.output_shape}")
sample = next(iter(test_dataset.as_numpy_iterator()))
print(len(sample)) # Should be 2
print(type(sample[0]), type(sample[1])) # Both should be <class 'numpy.ndarray'>
print(sample[0].shape, sample[1].shape) # Should match model input and outp
print("\n")
#for x, y in test_dataset.take(1):
#    print(type(x), type(y)) # Both should be <class 'tensorflow.Tensor'>
#for x_batch, y_batch in test_dataset.take(1):

```

```

# test_loss, test_acc = model.evaluate(x_batch, y_batch)
# print(f"Test Loss: {test_loss}, Test Accuracy: {test_acc}")

# Compile the model
#tensorboard_callback = keras.callbacks.TensorBoard(log_dir="./Logs")
model.compile(optimizer=optimizers.Adam(learning_rate=1e-3, weight_decay=1e-
            loss='categorical_crossentropy',
            metrics=['accuracy', 'precision', 'f1_score']))

model.summary()

#### <<<<<<<<Train Model & Track Training/Validation Error>>>>>>>>
early_stopping = EarlyStopping(monitor='val_loss', # or val_accuracy
                                patience=5, # Num. epochs with no improvement
                                restore_best_weights=True)
reduce_lr = ReduceLROnPlateau(monitor='val_loss', # or val_accuracy
                                factor=0.5, # Reduce lr by a factor
                                patience=3, # Num epochs w/ no improvement
                                min_lr=1e-6, # Min lr
                                verbose=1)
#tensorboard = TensorBoard(log_dir='./Logs', # Logs directory
#                             histogram_freq=1, # Logs histograms for weights/ac
#                             write_graph=True, # Logs graph of model
#                             write_images=True) # Log images like weight histog
checkpoint = ModelCheckpoint('best_model_v3.keras', # Path where the best mo
                                monitor='val_loss', # What to monitor (validati
                                save_best_only=True, # Only save model when val
                                mode='min', # Looking to minimise the validation
                                verbose=1) # Print out message when saving mode
checkpoint_fine_tune = ModelCheckpoint('best_model_fine_tune_v3.keras', # Pa
                                monitor='val_loss', # What to monitor (validati
                                save_best_only=True, # Only save model when val
                                mode='min', # Looking to minimise the validation
                                verbose=1) # Print out message when saving mode
#csv_logger = CSVLogger('training_log.csv', separator=',', append=True) # Sa

# Train the model
history = model.fit(train_dataset, validation_data=val_dataset, epochs=25,
                    batch_size=batch_size, callbacks=[checkpoint, reduce_lr,

#### <<<<<<<<Plot Training & Validation Error>>>>>>>>

# Extract loss and accuracy
epochs = range(1, len(history.history['loss'])+1)
train_loss = history.history['loss']
val_loss = history.history['val_loss']
train_acc = history.history['accuracy']
val_acc = history.history['val_accuracy']

plot_evidence(epochs, train_loss, val_loss, train_acc, val_acc)

#### <<<<<<<<Evaluate Model on Test Data>>>>>>>>

# Evaluate on test data
model = load_model('best_model_v3.keras')
results = model.evaluate(test_dataset)
test_loss = results[0]
test_acc = results[1]
test_precision = results[2]
test_f1_scores = results[3]

```

```

print(f"Test Accuracy: {test_acc*100:.2f}%")
print(f"Test Loss: {test_loss}")
print(f"Test Precision: {test_precision*100:.2f}%")
print(f"Test F1 Scores (Per Class): {test_f1_scores.numpy()*100}")
print(f"Average Test F1 Scores:{np.average(test_f1_scores.numpy()*100):.2f}\n

#### <<<<<<<<Generate Confusion Matrix>>>>>>>>

# Get predictions
X_test_revised = tf.image.resize(X_test, (224, 224))
y_pred = model.predict(X_test_revised)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true_classes = y_test.flatten()

# Compute confusion matrix
conf_matrix = confusion_matrix(y_true_classes, y_pred_classes)

# Plot confusion matrix
plt.figure(figsize=(20, 20))
sns.heatmap(conf_matrix) #cmap='Blues', fmt='d'
plt.xlabel('Predicted Labels')
plt.ylabel('True Labels')
plt.title('Confusion Matrix')
plt.tight_layout()
plt.show()

print(classification_report(y_true_classes, y_pred_classes, target_names=lab
#tensorboard --logdir==path_to_your_logs

# Create a DataFrame from the history of the training and store the epoch va
hist = pd.DataFrame(history.history)
hist['epoch'] = history.epoch

# Finally, display the hist DataFrame.
hist

#### <<<<<<<<Fine-Tune>>>>>>>>
# Adapt Model
for layer in resnet_50_base.layers:
    layer.trainable = True # Allow layers to be updated

# Compile again w/ lower learning rate (prevents destroying Learned features
model.compile(optimizer=optimizers.Adam(learning_rate=1e-5),
              loss='categorical_crossentropy',
              metrics=['accuracy', 'precision', 'f1_score'])

model.summary()

#### <<<<<<<<Modify Dataset>>>>>>>>

# DEBUG augment
train_dataset_aug_ = (tf.data.Dataset.from_tensor_slices((X_train, y_train))
                      .map(augment_dataset, num_parallel_calls=tf.data.experimental
                          .batch(batch_size)
                          .prefetch(tf.data.experimental.AUTOTUNE))

# Combine the original dataset and the augmented dataset
train_dataset_aug = train_dataset.concatenate(train_dataset_aug_) # Not val

```

```

#### <<<<<<<<<Train Model & Track Training/Validation Error>>>>>>>>>

# Train the model
history_fine_tune = model.fit(train_dataset_aug, validation_data=val_dataset,
                              batch_size=batch_size, callbacks=[checkpoint_f

#### <<<<<<<<<Plot Training & Validation Error>>>>>>>>>>

# Extract loss and accuracy
epochs = range(1, len(history_fine_tune.history['loss'])+1)
train_loss = history_fine_tune.history['loss']
val_loss = history_fine_tune.history['val_loss']
train_acc = history_fine_tune.history['accuracy']
val_acc = history_fine_tune.history['val_accuracy']

plot_evidence(epochs, train_loss, val_loss, train_acc, val_acc)

#### <<<<<<<<<Evaluate Model on Test Data>>>>>>>>>>

# Evaluate on test data
model_fine_tune = load_model('best_model_fine_tune_v3.keras')
results = model_fine_tune.evaluate(test_dataset)
test_loss = results[0]
test_acc = results[1]
test_precision = results[2]
test_f1_scores = results[3]
print(f"Test Accuracy: {test_acc*100:.2f}%")
print(f"Test Loss: {test_loss}")
print(f"Test Precision: {test_precision*100:.2f}%")
print(f"Test F1 Scores (Per Class): {test_f1_scores.numpy()*100}")
print(f"Average Test F1 Scores:{np.average(test_f1_scores.numpy()*100):.2f}\

#### <<<<<<<<<Generate Confusion Matrix>>>>>>>>>>

# Get predictions
X_test_revised = tf.image.resize(X_test, (224, 224))
y_pred = model_fine_tune.predict(X_test_revised)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true_classes = y_test.flatten()

# Compute confusion matrix
conf_matrix = confusion_matrix(y_true_classes, y_pred_classes)

# Plot confusion matrix
plt.figure(figsize=(20, 20))
sns.heatmap(conf_matrix) #cmap='Blues', fmt='d'
plt.xlabel('Predicted Labels')
plt.ylabel('True Labels')
plt.title('Confusion Matrix')
plt.tight_layout()
plt.show()

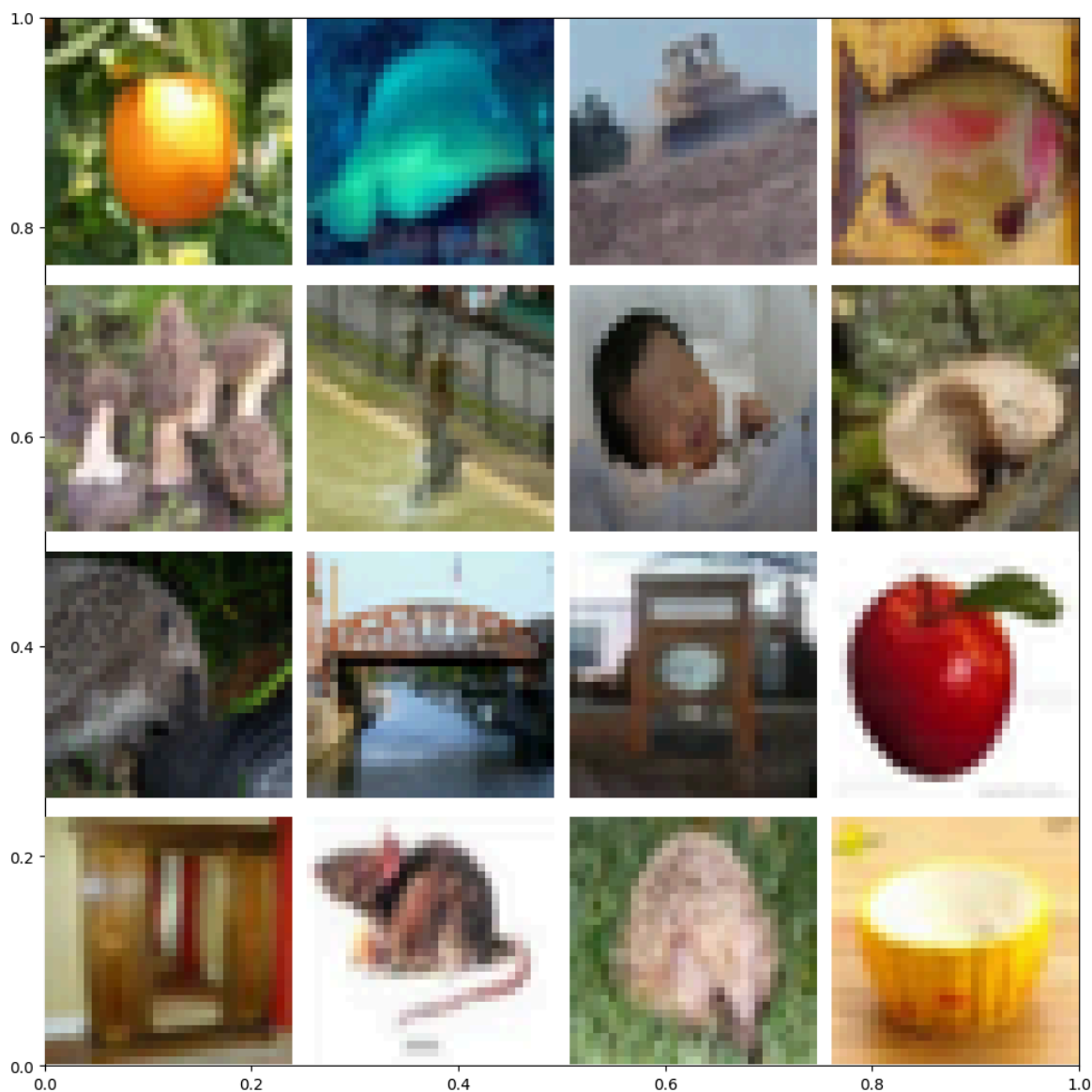
print(classification_report(y_true_classes, y_pred_classes, target_names=lab

# Create a DataFrame from the history of the training and store the epoch va
hist = pd.DataFrame(history_fine_tune.history)
hist['epoch'] = history_fine_tune.epoch

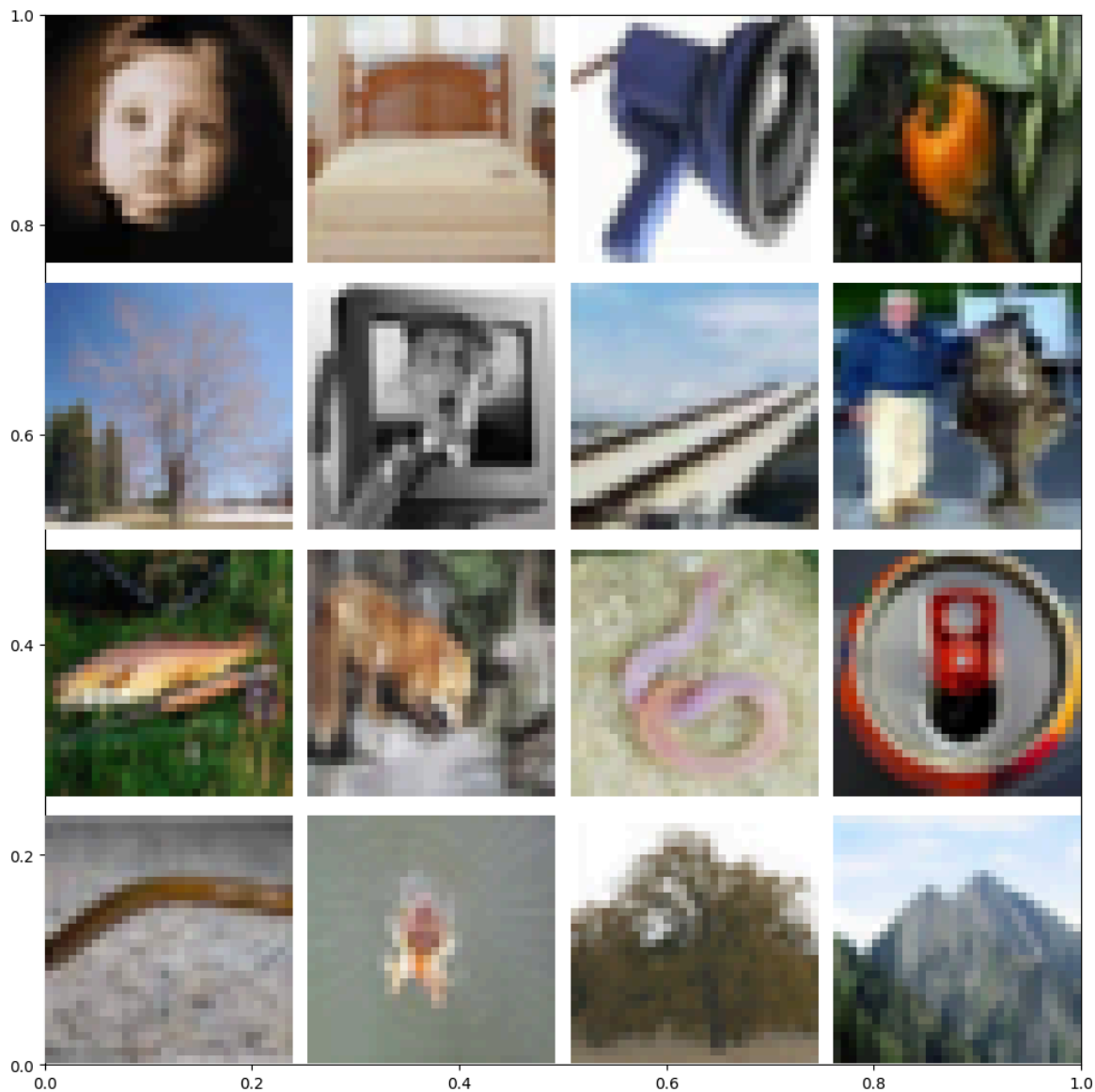
# Finally, display the hist DataFrame.
hist

```

```
labels[32704].shape: (1,)
imgs[32704].shape: (32, 32, 3)
```

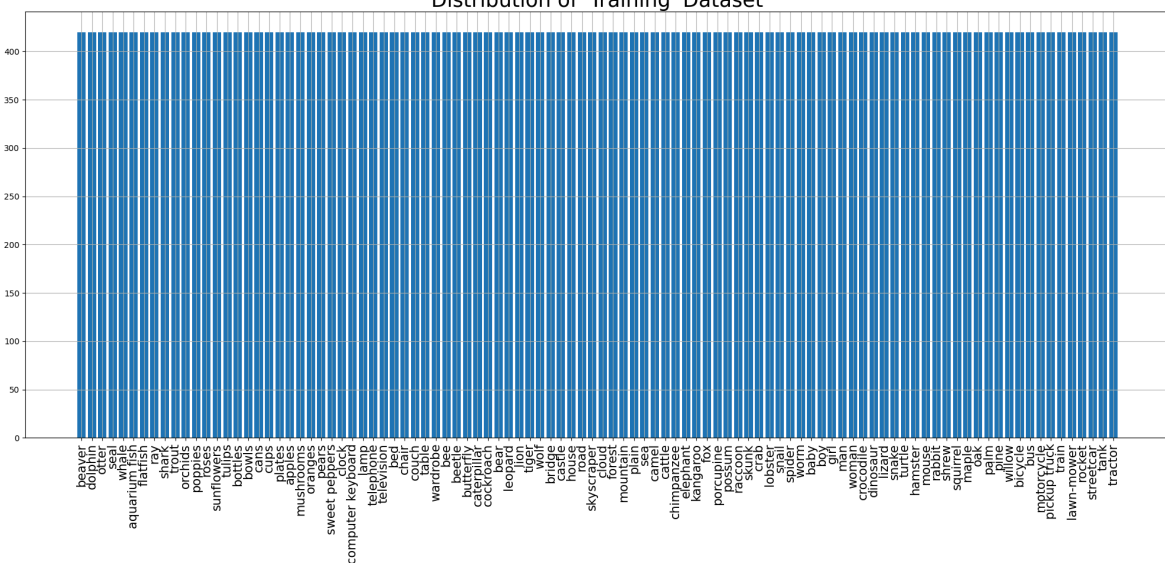


```
labels[21791].shape: (1,)
imgs[21791].shape: (32, 32, 3)
```

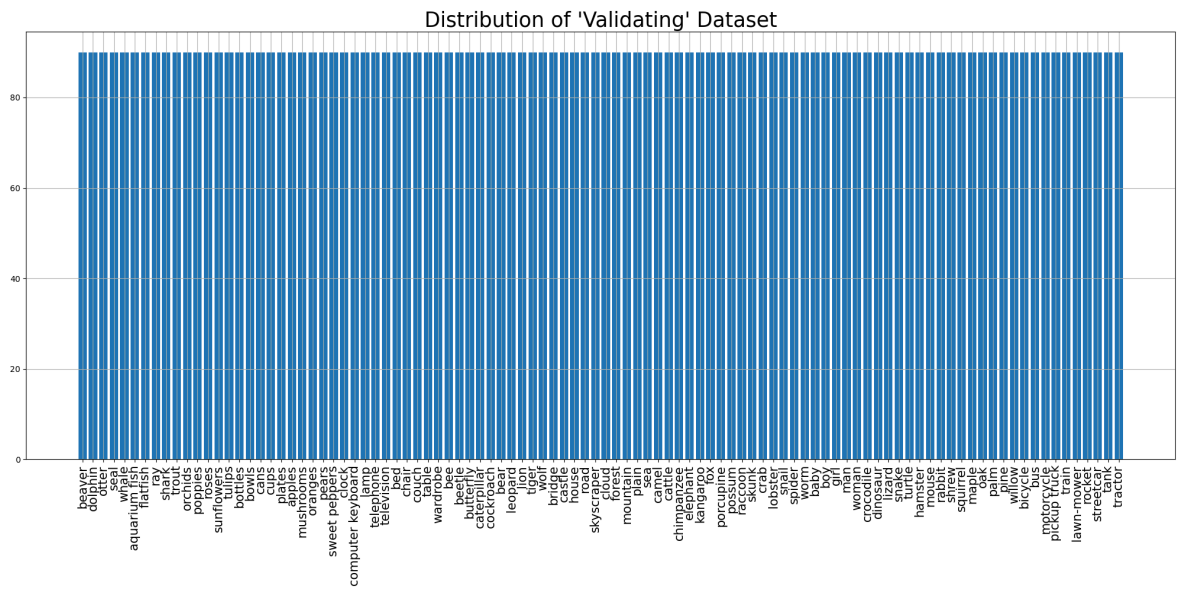


```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```

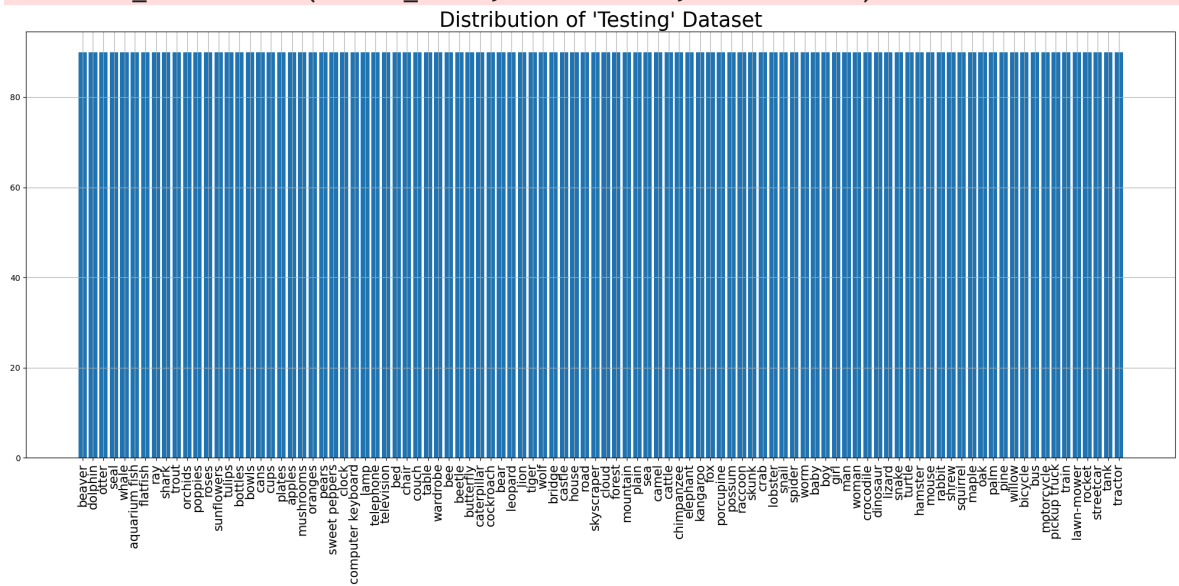
Distribution of 'Training' Dataset



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



Training dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
```

Validation dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
```

Testing dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
```

Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/resnet/resnet50v2_weights_tf_dim_ordering_tf_kernels_notop.h5

94668760/94668760 ————— **0s** 0us/step

<class 'tuple'>

2

<class 'tensorflow.python.framework.ops.EagerTensor'> <class 'tensorflow.python.framework.ops.EagerTensor'>

(64, 224, 224, 3)

(64, 100)

Model input shape: (None, 224, 224, 3)

Model output shape: (None, 100)

2

<class 'numpy.ndarray'> <class 'numpy.ndarray'>

(64, 224, 224, 3) (64, 100)

Model: "sequential_2"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None , 7 , 7 , 2048)
global_average_pooling2d_2 (GlobalAveragePooling2D)	(None , 2048)
dense_4 (Dense)	(None , 512)
dropout (Dropout)	(None , 512)
dense_5 (Dense)	(None , 100)

◀ ————— ▶

Total params: **24,665,188** (94.09 MB)

Trainable params: **1,100,388** (4.20 MB)

Non-trainable params: **23,564,800** (89.89 MB)

Epoch 1/25
657/657 ————— **0s** 29ms/step - accuracy: 0.3153 - f1_score: 0.3096 - loss: 2.9239 - precision: 0.6580
Epoch 1: val_loss improved from inf to 1.52481, saving model to best_model_v3.keras
657/657 ————— **41s** 45ms/step - accuracy: 0.3154 - f1_score: 0.3097 - loss: 2.9230 - precision: 0.6581 - val_accuracy: 0.5848 - val_f1_score: 0.5807 - val_loss: 1.5248 - val_precision: 0.8240 - learning_rate: 0.0010

Epoch 2/25
656/657 ————— **0s** 23ms/step - accuracy: 0.5279 - f1_score: 0.5238 - loss: 1.7301 - precision: 0.7500
Epoch 2: val_loss improved from 1.52481 to 1.40151, saving model to best_model_v3.keras
657/657 ————— **19s** 29ms/step - accuracy: 0.5279 - f1_score: 0.5238 - loss: 1.7300 - precision: 0.7501 - val_accuracy: 0.6110 - val_f1_score: 0.6068 - val_loss: 1.4015 - val_precision: 0.8134 - learning_rate: 0.0010

Epoch 3/25
655/657 ————— **0s** 23ms/step - accuracy: 0.5643 - f1_score: 0.5604 - loss: 1.5580 - precision: 0.7670
Epoch 3: val_loss improved from 1.40151 to 1.36723, saving model to best_model_v3.keras
657/657 ————— **19s** 29ms/step - accuracy: 0.5644 - f1_score: 0.5605 - loss: 1.5578 - precision: 0.7670 - val_accuracy: 0.6157 - val_f1_score: 0.6129 - val_loss: 1.3672 - val_precision: 0.8110 - learning_rate: 0.0010

Epoch 4/25
655/657 ————— **0s** 23ms/step - accuracy: 0.5861 - f1_score: 0.5828 - loss: 1.4545 - precision: 0.7768
Epoch 4: val_loss improved from 1.36723 to 1.32739, saving model to best_model_v3.keras
657/657 ————— **19s** 29ms/step - accuracy: 0.5861 - f1_score: 0.5829 - loss: 1.4544 - precision: 0.7769 - val_accuracy: 0.6243 - val_f1_score: 0.6219 - val_loss: 1.3274 - val_precision: 0.8112 - learning_rate: 0.0010

Epoch 5/25
655/657 ————— **0s** 23ms/step - accuracy: 0.6032 - f1_score: 0.6000 - loss: 1.3854 - precision: 0.7825
Epoch 5: val_loss improved from 1.32739 to 1.30866, saving model to best_model_v3.keras
657/657 ————— **19s** 29ms/step - accuracy: 0.6032 - f1_score: 0.6000 - loss: 1.3853 - precision: 0.7825 - val_accuracy: 0.6324 - val_f1_score: 0.6306 - val_loss: 1.3087 - val_precision: 0.8107 - learning_rate: 0.0010

Epoch 6/25
655/657 ————— **0s** 23ms/step - accuracy: 0.6210 - f1_score: 0.6171 - loss: 1.3104 - precision: 0.7878
Epoch 6: val_loss improved from 1.30866 to 1.29641, saving model to best_model_v3.keras
657/657 ————— **19s** 29ms/step - accuracy: 0.6210 - f1_score: 0.6171 - loss: 1.3104 - precision: 0.7879 - val_accuracy: 0.6342 - val_f1_score: 0.6326 - val_loss: 1.2964 - val_precision: 0.8030 - learning_rate: 0.0010

Epoch 7/25
656/657 ————— **0s** 23ms/step - accuracy: 0.6314 - f1_score: 0.6288 - loss: 1.2625 - precision: 0.7985
Epoch 7: val_loss did not improve from 1.29641
657/657 ————— **18s** 28ms/step - accuracy: 0.6314 - f1_score: 0.6289 - loss: 1.2625 - precision: 0.7985 - val_accuracy: 0.6390 - val_f1_score: 0.6389 - val_loss: 1.3023 - val_precision: 0.8091 - learning_rate: 0.0010

Epoch 8/25
655/657 ————— **0s** 23ms/step - accuracy: 0.6404 - f1_score: 0.6378 - loss: 1.2222 - precision: 0.7965
Epoch 8: val_loss improved from 1.29641 to 1.28583, saving model to best_model_v3.keras

657/657 ————— **19s** 29ms/step - accuracy: 0.6404 - f1_score: 0.6378
- loss: 1.2221 - precision: 0.7965 - val_accuracy: 0.6404 - val_f1_score: 0.6384
- val_loss: 1.2858 - val_precision: 0.8030 - learning_rate: 0.0010
Epoch 9/25

655/657 ————— **0s** 23ms/step - accuracy: 0.6532 - f1_score: 0.6510 -
loss: 1.1753 - precision: 0.8056
Epoch 9: val_loss did not improve from 1.28583

657/657 ————— **18s** 28ms/step - accuracy: 0.6533 - f1_score: 0.6510
- loss: 1.1752 - precision: 0.8056 - val_accuracy: 0.6438 - val_f1_score: 0.6427
- val_loss: 1.2877 - val_precision: 0.7950 - learning_rate: 0.0010
Epoch 10/25

655/657 ————— **0s** 23ms/step - accuracy: 0.6610 - f1_score: 0.6584 -
loss: 1.1437 - precision: 0.8018
Epoch 10: val_loss improved from 1.28583 to 1.28522, saving model to best_model_v3.keras

657/657 ————— **19s** 29ms/step - accuracy: 0.6611 - f1_score: 0.6584
- loss: 1.1437 - precision: 0.8018 - val_accuracy: 0.6422 - val_f1_score: 0.6422
- val_loss: 1.2852 - val_precision: 0.7953 - learning_rate: 0.0010
Epoch 11/25

657/657 ————— **0s** 23ms/step - accuracy: 0.6700 - f1_score: 0.6678 -
loss: 1.1101 - precision: 0.8131
Epoch 11: val_loss improved from 1.28522 to 1.26962, saving model to best_model_v3.keras

657/657 ————— **21s** 31ms/step - accuracy: 0.6701 - f1_score: 0.6678
- loss: 1.1101 - precision: 0.8132 - val_accuracy: 0.6470 - val_f1_score: 0.6449
- val_loss: 1.2696 - val_precision: 0.7937 - learning_rate: 0.0010
Epoch 12/25

656/657 ————— **0s** 23ms/step - accuracy: 0.6703 - f1_score: 0.6677 -
loss: 1.0860 - precision: 0.8110
Epoch 12: val_loss did not improve from 1.26962

657/657 ————— **18s** 28ms/step - accuracy: 0.6703 - f1_score: 0.6678
- loss: 1.0860 - precision: 0.8111 - val_accuracy: 0.6461 - val_f1_score: 0.6440
- val_loss: 1.2795 - val_precision: 0.7924 - learning_rate: 0.0010
Epoch 13/25

657/657 ————— **0s** 23ms/step - accuracy: 0.6805 - f1_score: 0.6780 -
loss: 1.0529 - precision: 0.8170
Epoch 13: val_loss did not improve from 1.26962

657/657 ————— **18s** 28ms/step - accuracy: 0.6805 - f1_score: 0.6780
- loss: 1.0529 - precision: 0.8170 - val_accuracy: 0.6443 - val_f1_score: 0.6433
- val_loss: 1.2790 - val_precision: 0.7921 - learning_rate: 0.0010
Epoch 14/25

657/657 ————— **0s** 23ms/step - accuracy: 0.6909 - f1_score: 0.6881 -
loss: 1.0284 - precision: 0.8212
Epoch 14: val_loss did not improve from 1.26962

Epoch 14: ReduceLROnPlateau reducing learning rate to 0.000500000237487257.

657/657 ————— **18s** 28ms/step - accuracy: 0.6909 - f1_score: 0.6881
- loss: 1.0284 - precision: 0.8212 - val_accuracy: 0.6447 - val_f1_score: 0.6434
- val_loss: 1.2833 - val_precision: 0.7894 - learning_rate: 0.0010
Epoch 15/25

657/657 ————— **0s** 23ms/step - accuracy: 0.7209 - f1_score: 0.7187 -
loss: 0.9092 - precision: 0.8463
Epoch 15: val_loss improved from 1.26962 to 1.23841, saving model to best_model_v3.keras

657/657 ————— **19s** 29ms/step - accuracy: 0.7210 - f1_score: 0.7187
- loss: 0.9091 - precision: 0.8463 - val_accuracy: 0.6541 - val_f1_score: 0.6521
- val_loss: 1.2384 - val_precision: 0.7891 - learning_rate: 5.0000e-04
Epoch 16/25

657/657 ————— **0s** 23ms/step - accuracy: 0.7503 - f1_score: 0.7477 -
loss: 0.8078 - precision: 0.8607

Epoch 16: val_loss improved from 1.23841 to 1.23527, saving model to best_model_v3.keras

657/657  **19s** 29ms/step - accuracy: 0.7503 - f1_score: 0.7477
- loss: 0.8077 - precision: 0.8607 - val_accuracy: 0.6604 - val_f1_score: 0.6594
- val_loss: 1.2353 - val_precision: 0.7906 - learning_rate: 5.0000e-04

Epoch 17/25

657/657  **0s** 23ms/step - accuracy: 0.7559 - f1_score: 0.7538 -
loss: 0.7835 - precision: 0.8657

Epoch 17: val_loss improved from 1.23527 to 1.23201, saving model to best_model_v3.keras

657/657  **19s** 29ms/step - accuracy: 0.7559 - f1_score: 0.7539
- loss: 0.7835 - precision: 0.8657 - val_accuracy: 0.6602 - val_f1_score: 0.6592
- val_loss: 1.2320 - val_precision: 0.7910 - learning_rate: 5.0000e-04

Epoch 18/25

656/657  **0s** 23ms/step - accuracy: 0.7675 - f1_score: 0.7654 -
loss: 0.7415 - precision: 0.8714

Epoch 18: val_loss did not improve from 1.23201

657/657  **18s** 28ms/step - accuracy: 0.7675 - f1_score: 0.7654
- loss: 0.7415 - precision: 0.8714 - val_accuracy: 0.6570 - val_f1_score: 0.6559
- val_loss: 1.2357 - val_precision: 0.7849 - learning_rate: 5.0000e-04

Epoch 19/25

655/657  **0s** 23ms/step - accuracy: 0.7718 - f1_score: 0.7699 -
loss: 0.7256 - precision: 0.8744

Epoch 19: val_loss did not improve from 1.23201

657/657  **18s** 28ms/step - accuracy: 0.7718 - f1_score: 0.7699
- loss: 0.7255 - precision: 0.8744 - val_accuracy: 0.6636 - val_f1_score: 0.6613
- val_loss: 1.2330 - val_precision: 0.7905 - learning_rate: 5.0000e-04

Epoch 20/25

657/657  **0s** 23ms/step - accuracy: 0.7815 - f1_score: 0.7798 -
loss: 0.7015 - precision: 0.8817

Epoch 20: val_loss improved from 1.23201 to 1.22990, saving model to best_model_v3.keras

657/657  **19s** 30ms/step - accuracy: 0.7815 - f1_score: 0.7798
- loss: 0.7015 - precision: 0.8817 - val_accuracy: 0.6629 - val_f1_score: 0.6628
- val_loss: 1.2299 - val_precision: 0.7882 - learning_rate: 5.0000e-04

Epoch 21/25

656/657  **0s** 23ms/step - accuracy: 0.7880 - f1_score: 0.7857 -
loss: 0.6817 - precision: 0.8813

Epoch 21: val_loss did not improve from 1.22990

657/657  **18s** 28ms/step - accuracy: 0.7880 - f1_score: 0.7857
- loss: 0.6817 - precision: 0.8813 - val_accuracy: 0.6617 - val_f1_score: 0.6598
- val_loss: 1.2347 - val_precision: 0.7867 - learning_rate: 5.0000e-04

Epoch 22/25

655/657  **0s** 23ms/step - accuracy: 0.7895 - f1_score: 0.7872 -
loss: 0.6737 - precision: 0.8803

Epoch 22: val_loss did not improve from 1.22990

657/657  **18s** 28ms/step - accuracy: 0.7895 - f1_score: 0.7872
- loss: 0.6737 - precision: 0.8803 - val_accuracy: 0.6621 - val_f1_score: 0.6598
- val_loss: 1.2418 - val_precision: 0.7839 - learning_rate: 5.0000e-04

Epoch 23/25

655/657  **0s** 23ms/step - accuracy: 0.7978 - f1_score: 0.7958 -
loss: 0.6530 - precision: 0.8873

Epoch 23: val_loss did not improve from 1.22990

Epoch 23: ReduceLROnPlateau reducing learning rate to 0.0002500000118743628.

657/657  **18s** 28ms/step - accuracy: 0.7978 - f1_score: 0.7958
- loss: 0.6529 - precision: 0.8873 - val_accuracy: 0.6644 - val_f1_score: 0.6627
- val_loss: 1.2460 - val_precision: 0.7836 - learning_rate: 5.0000e-04

Epoch 24/25

656/657  **0s** 23ms/step - accuracy: 0.8113 - f1_score: 0.8096 -

loss: 0.5985 - precision: 0.8966

Epoch 24: val_loss improved from 1.2290 to 1.22905, saving model to best_model_v3.keras

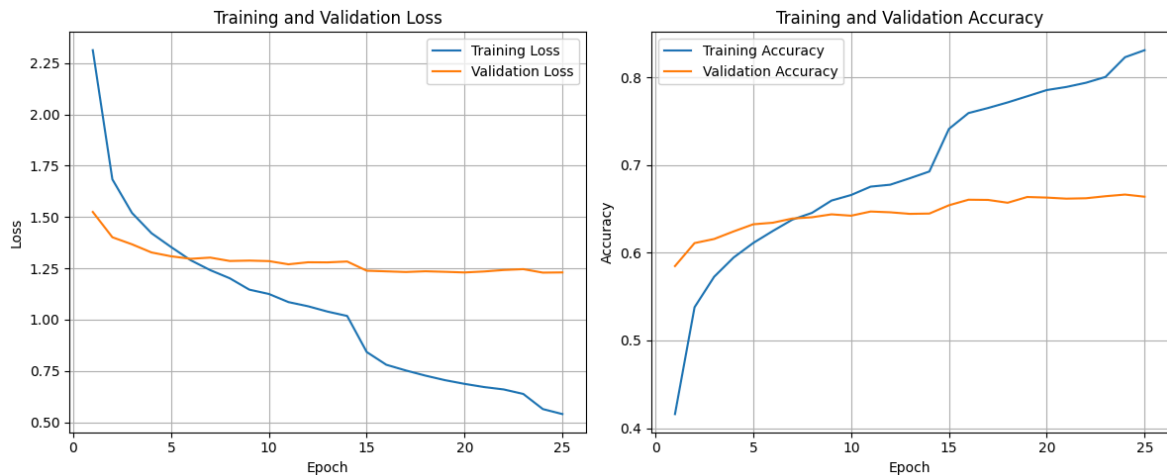
657/657 ————— 19s 30ms/step - accuracy: 0.8114 - f1_score: 0.8097
 - loss: 0.5984 - precision: 0.8966 - val_accuracy: 0.6663 - val_f1_score: 0.6653
 - val_loss: 1.2291 - val_precision: 0.7802 - learning_rate: 2.5000e-04

Epoch 25/25

657/657 ————— 0s 23ms/step - accuracy: 0.8260 - f1_score: 0.8243 -
 loss: 0.5608 - precision: 0.9057

Epoch 25: val_loss did not improve from 1.22905

657/657 ————— 18s 28ms/step - accuracy: 0.8260 - f1_score: 0.8243
 - loss: 0.5607 - precision: 0.9057 - val_accuracy: 0.6639 - val_f1_score: 0.6623
 - val_loss: 1.2301 - val_precision: 0.7764 - learning_rate: 2.5000e-04



141/141 ————— 11s 39ms/step - accuracy: 0.6603 - f1_score: 0.6528
 - loss: 1.2323 - precision: 0.7771

Test Accuracy: 67.19%

Test Loss: 1.2073662281036377

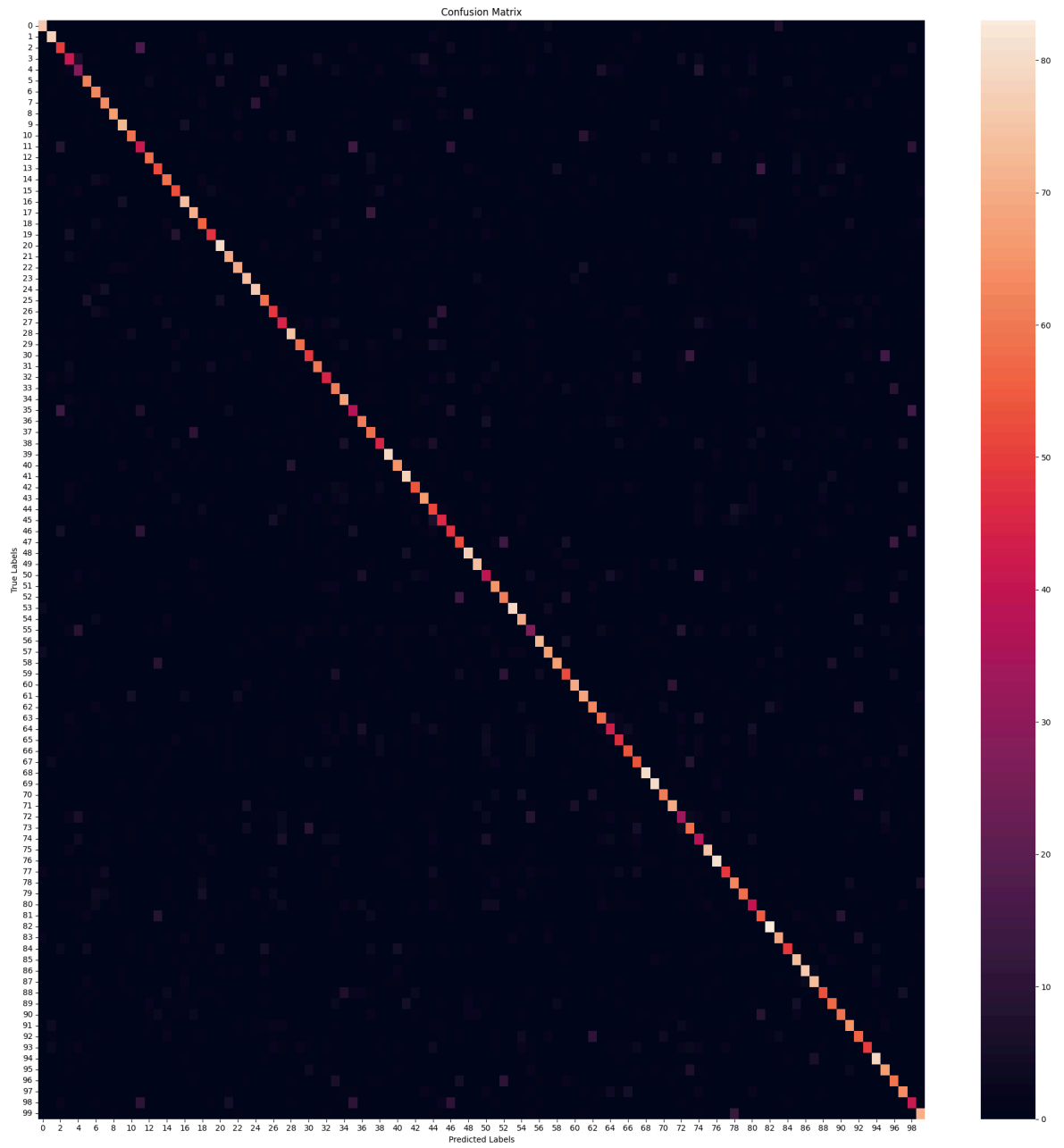
Test Precision: 78.48%

Test F1 Scores (Per Class): [85.39326 82.97872 54.945045 47.67441 35.15151 7
 0.85714 65.62499

71.50838	79.761894	80.446915	71.08433	44.808735	69.87952	61.53846
65.92178	60.571426	78.723404	77.59562	61.538452	58.536575	84.21052
74.8663	79.09604	81.31868	82.608696	68.208084	56.32183	50.56179
80.21389	70.731705	59.036137	70.17543	53.571423	56.744183	71.13402
44.31137	69.31818	59.793808	54.878044	89.77272	67.70833	81.24999
61.01694	73.33333	47.663544	50.549446	51.891888	61.90476	82.35293
79.99999	41.269836	68.783066	64.893616	88.76403	72.916664	32.72727
81.35593	73.224045	79.761894	59.770107	79.54544	75.82417	69.23077
65.53671	49.999992	55.95237	65.060234	56.842102	89.385475	86.48647
72.18934	74.8663	39.52095	61.375652	38.38383	83.33333	85.263145
60.1227	68.47825	69.047615	41.450768	61.538452	92.22221	73.79678
63.636356	73.36682	84.44444	76.43979	61.6279	66.27907	67.42856
68.783066	57.731956	62.893074	84.946236	70.89946	61.7801	67.36841
45.30386	79.329605]					

Average Test F1 Scores:67.10

282/282 ————— 12s 28ms/step



	precision	recall	f1-score	support
beaver	0.86	0.84	0.85	90
dolphin	0.80	0.87	0.83	90
otter	0.54	0.56	0.55	90
seal	0.50	0.46	0.48	90
whale	0.39	0.32	0.35	90
aquarium fish	0.73	0.69	0.71	90
flatfish	0.62	0.70	0.66	90
ray	0.72	0.71	0.72	90
shark	0.86	0.74	0.80	90
trout	0.81	0.80	0.80	90
orchids	0.78	0.66	0.71	90
poppies	0.44	0.46	0.45	90
roses	0.76	0.64	0.70	90
sunflowers	0.66	0.58	0.62	90
tulips	0.66	0.66	0.66	90
bottles	0.62	0.59	0.61	90
bowls	0.76	0.82	0.79	90
cans	0.76	0.79	0.78	90
cups	0.61	0.62	0.62	90
plates	0.65	0.53	0.59	90
apples	0.80	0.89	0.84	90
mushrooms	0.72	0.78	0.75	90
oranges	0.80	0.78	0.79	90
pears	0.80	0.82	0.81	90
sweet peppers	0.81	0.84	0.83	90
clock	0.71	0.66	0.68	90
computer keyboard	0.58	0.54	0.56	90
lamp	0.51	0.50	0.51	90
telephone	0.77	0.83	0.80	90
television	0.78	0.64	0.71	90
bed	0.64	0.54	0.59	90
chair	0.74	0.67	0.70	90
couch	0.58	0.50	0.54	90
table	0.49	0.68	0.57	90
wardrobe	0.66	0.77	0.71	90
bee	0.48	0.41	0.44	90
beetle	0.71	0.68	0.69	90
butterfly	0.56	0.64	0.60	90
caterpillar	0.61	0.50	0.55	90
cockroach	0.92	0.88	0.90	90
bear	0.64	0.72	0.68	90
leopard	0.76	0.87	0.81	90
lion	0.62	0.60	0.61	90
tiger	0.73	0.73	0.73	90
wolf	0.41	0.57	0.48	90
bridge	0.50	0.51	0.51	90
castle	0.51	0.53	0.52	90
house	0.67	0.58	0.62	90
road	0.79	0.86	0.82	90
skyscraper	0.78	0.82	0.80	90
cloud	0.39	0.43	0.41	90
forest	0.66	0.72	0.69	90
mountain	0.62	0.68	0.65	90
plain	0.90	0.88	0.89	90
sea	0.69	0.78	0.73	90
camel	0.36	0.30	0.33	90
cattle	0.83	0.80	0.81	90
chimpanzee	0.72	0.74	0.73	90

elephant	0.86	0.74	0.80	90
kangaroo	0.62	0.58	0.60	90
fox	0.81	0.78	0.80	90
porcupine	0.75	0.77	0.76	90
possum	0.68	0.70	0.69	90
raccoon	0.67	0.64	0.66	90
skunk	0.52	0.48	0.50	90
crab	0.60	0.52	0.56	90
lobster	0.71	0.60	0.65	90
snail	0.54	0.60	0.57	90
spider	0.90	0.89	0.89	90
worm	0.84	0.89	0.86	90
baby	0.77	0.68	0.72	90
boy	0.72	0.78	0.75	90
girl	0.43	0.37	0.40	90
man	0.59	0.64	0.61	90
woman	0.35	0.42	0.38	90
crocodile	0.83	0.83	0.83	90
dinosaur	0.81	0.90	0.85	90
lizard	0.67	0.54	0.60	90
snake	0.67	0.70	0.68	90
turtle	0.74	0.64	0.69	90
hamster	0.39	0.44	0.41	90
mouse	0.60	0.61	0.61	90
rabbit	0.92	0.92	0.92	90
shrew	0.71	0.77	0.74	90
squirrel	0.77	0.54	0.64	90
maple	0.67	0.81	0.73	90
oak	0.84	0.84	0.84	90
palm	0.72	0.81	0.76	90
pine	0.65	0.59	0.62	90
willow	0.70	0.63	0.66	90
bicycle	0.69	0.66	0.67	90
bus	0.66	0.72	0.69	90
motorcycle	0.54	0.62	0.58	90
pickup truck	0.72	0.56	0.63	90
train	0.82	0.88	0.85	90
lawn-mower	0.68	0.74	0.71	90
rocket	0.58	0.66	0.62	90
streetcar	0.64	0.71	0.67	90
tank	0.45	0.46	0.45	90
tractor	0.80	0.79	0.79	90
accuracy			0.67	9000
macro avg	0.67	0.67	0.67	9000
weighted avg	0.67	0.67	0.67	9000

Model: "sequential_2"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None , 7 , 7 , 2048)
global_average_pooling2d_2 (GlobalAveragePooling2D)	(None , 2048)
dense_4 (Dense)	(None , 512)
dropout (Dropout)	(None , 512)
dense_5 (Dense)	(None , 100)



Total params: [24,665,188](#) (94.09 MB)

Trainable params: [1,100,388](#) (4.20 MB)

Non-trainable params: [23,564,800](#) (89.89 MB)

Epoch 1/25
1313/1314 ————— **0s** 26ms/step - accuracy: 0.8164 - f1_score: 0.8155
- loss: 0.5882 - precision: 0.8973
Epoch 1: val_loss improved from inf to 1.20597, saving model to best_model_fine_tune_v3.keras
1314/1314 ————— **56s** 34ms/step - accuracy: 0.8164 - f1_score: 0.8155 - loss: 0.5883 - precision: 0.8972 - val_accuracy: 0.6683 - val_f1_score: 0.6666 - val_loss: 1.2060 - val_precision: 0.7850 - learning_rate: 1.0000e-05

Epoch 2/25
1313/1314 ————— **0s** 24ms/step - accuracy: 0.8294 - f1_score: 0.8283 - loss: 0.5580 - precision: 0.9095
Epoch 2: val_loss improved from 1.20597 to 1.20072, saving model to best_model_fine_tune_v3.keras
1314/1314 ————— **35s** 27ms/step - accuracy: 0.8293 - f1_score: 0.8283 - loss: 0.5581 - precision: 0.9095 - val_accuracy: 0.6702 - val_f1_score: 0.6684 - val_loss: 1.2007 - val_precision: 0.7873 - learning_rate: 1.0000e-05

Epoch 3/25
1312/1314 ————— **0s** 24ms/step - accuracy: 0.8333 - f1_score: 0.8322 - loss: 0.5526 - precision: 0.9127
Epoch 3: val_loss improved from 1.20072 to 1.19886, saving model to best_model_fine_tune_v3.keras
1314/1314 ————— **36s** 27ms/step - accuracy: 0.8333 - f1_score: 0.8321 - loss: 0.5528 - precision: 0.9126 - val_accuracy: 0.6698 - val_f1_score: 0.6681 - val_loss: 1.1989 - val_precision: 0.7896 - learning_rate: 1.0000e-05

Epoch 4/25
1312/1314 ————— **0s** 24ms/step - accuracy: 0.8333 - f1_score: 0.8322 - loss: 0.5488 - precision: 0.9134
Epoch 4: val_loss improved from 1.19886 to 1.19719, saving model to best_model_fine_tune_v3.keras
1314/1314 ————— **35s** 27ms/step - accuracy: 0.8333 - f1_score: 0.8322 - loss: 0.5490 - precision: 0.9134 - val_accuracy: 0.6699 - val_f1_score: 0.6683 - val_loss: 1.1972 - val_precision: 0.7889 - learning_rate: 1.0000e-05

Epoch 5/25
1312/1314 ————— **0s** 24ms/step - accuracy: 0.8372 - f1_score: 0.8362 - loss: 0.5410 - precision: 0.9161
Epoch 5: val_loss improved from 1.19719 to 1.19613, saving model to best_model_fine_tune_v3.keras
1314/1314 ————— **35s** 27ms/step - accuracy: 0.8372 - f1_score: 0.8362 - loss: 0.5412 - precision: 0.9161 - val_accuracy: 0.6717 - val_f1_score: 0.6702 - val_loss: 1.1961 - val_precision: 0.7899 - learning_rate: 1.0000e-05

Epoch 6/25
1312/1314 ————— **0s** 24ms/step - accuracy: 0.8382 - f1_score: 0.8371 - loss: 0.5364 - precision: 0.9156
Epoch 6: val_loss improved from 1.19613 to 1.19600, saving model to best_model_fine_tune_v3.keras
1314/1314 ————— **36s** 27ms/step - accuracy: 0.8381 - f1_score: 0.8371 - loss: 0.5366 - precision: 0.9156 - val_accuracy: 0.6723 - val_f1_score: 0.6707 - val_loss: 1.1960 - val_precision: 0.7901 - learning_rate: 1.0000e-05

Epoch 7/25
1312/1314 ————— **0s** 24ms/step - accuracy: 0.8381 - f1_score: 0.8369 - loss: 0.5339 - precision: 0.9158
Epoch 7: val_loss improved from 1.19600 to 1.19573, saving model to best_model_fine_tune_v3.keras
1314/1314 ————— **35s** 27ms/step - accuracy: 0.8381 - f1_score: 0.8369 - loss: 0.5341 - precision: 0.9157 - val_accuracy: 0.6703 - val_f1_score: 0.6688 - val_loss: 1.1957 - val_precision: 0.7898 - learning_rate: 1.0000e-05

Epoch 8/25
1313/1314 ————— **0s** 24ms/step - accuracy: 0.8408 - f1_score: 0.8399 - loss: 0.5251 - precision: 0.9167
Epoch 8: val_loss improved from 1.19573 to 1.19492, saving model to best_model_fine_tune_v3.keras

ne_tune_v3.keras

1314/1314 ————— **35s** 27ms/step - accuracy: 0.8408 - f1_score: 0.839
9 - loss: 0.5252 - precision: 0.9167 - val_accuracy: 0.6722 - val_f1_score: 0.670
6 - val_loss: 1.1949 - val_precision: 0.7892 - learning_rate: 1.0000e-05

Epoch 9/25

1314/1314 ————— **0s** 24ms/step - accuracy: 0.8410 - f1_score: 0.8400
- loss: 0.5258 - precision: 0.9176

Epoch 9: val_loss did not improve from 1.19492

1314/1314 ————— **34s** 26ms/step - accuracy: 0.8410 - f1_score: 0.840
0 - loss: 0.5259 - precision: 0.9176 - val_accuracy: 0.6728 - val_f1_score: 0.671
1 - val_loss: 1.1953 - val_precision: 0.7893 - learning_rate: 1.0000e-05

Epoch 10/25

1312/1314 ————— **0s** 24ms/step - accuracy: 0.8432 - f1_score: 0.8422
- loss: 0.5262 - precision: 0.9169

Epoch 10: val_loss did not improve from 1.19492

1314/1314 ————— **34s** 26ms/step - accuracy: 0.8431 - f1_score: 0.842
1 - loss: 0.5264 - precision: 0.9169 - val_accuracy: 0.6720 - val_f1_score: 0.670
4 - val_loss: 1.1951 - val_precision: 0.7897 - learning_rate: 1.0000e-05

Epoch 11/25

1313/1314 ————— **0s** 24ms/step - accuracy: 0.8449 - f1_score: 0.8438
- loss: 0.5188 - precision: 0.9193

Epoch 11: val_loss did not improve from 1.19492

Epoch 11: ReduceLROnPlateau reducing learning rate to 4.999999873689376e-06.

1314/1314 ————— **34s** 26ms/step - accuracy: 0.8448 - f1_score: 0.843
8 - loss: 0.5189 - precision: 0.9193 - val_accuracy: 0.6724 - val_f1_score: 0.670
9 - val_loss: 1.1953 - val_precision: 0.7880 - learning_rate: 1.0000e-05

Epoch 12/25

1314/1314 ————— **0s** 24ms/step - accuracy: 0.8450 - f1_score: 0.8439
- loss: 0.5178 - precision: 0.9217

Epoch 12: val_loss did not improve from 1.19492

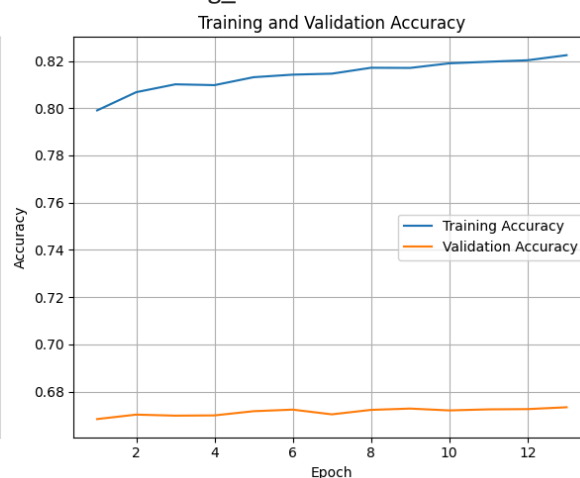
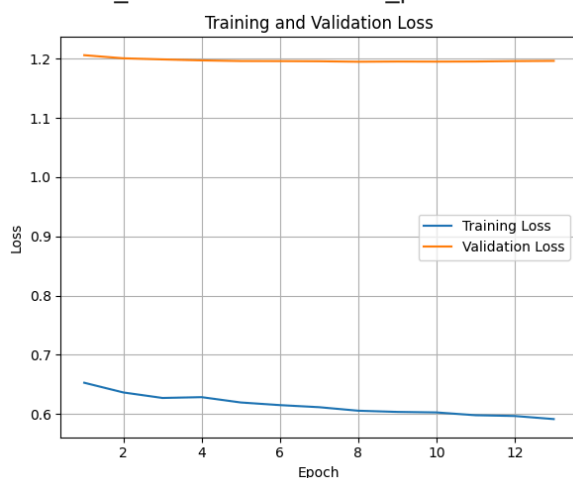
1314/1314 ————— **34s** 26ms/step - accuracy: 0.8449 - f1_score: 0.843
9 - loss: 0.5178 - precision: 0.9217 - val_accuracy: 0.6726 - val_f1_score: 0.671
0 - val_loss: 1.1960 - val_precision: 0.7871 - learning_rate: 5.0000e-06

Epoch 13/25

1314/1314 ————— **0s** 24ms/step - accuracy: 0.8473 - f1_score: 0.8461
- loss: 0.5154 - precision: 0.9215

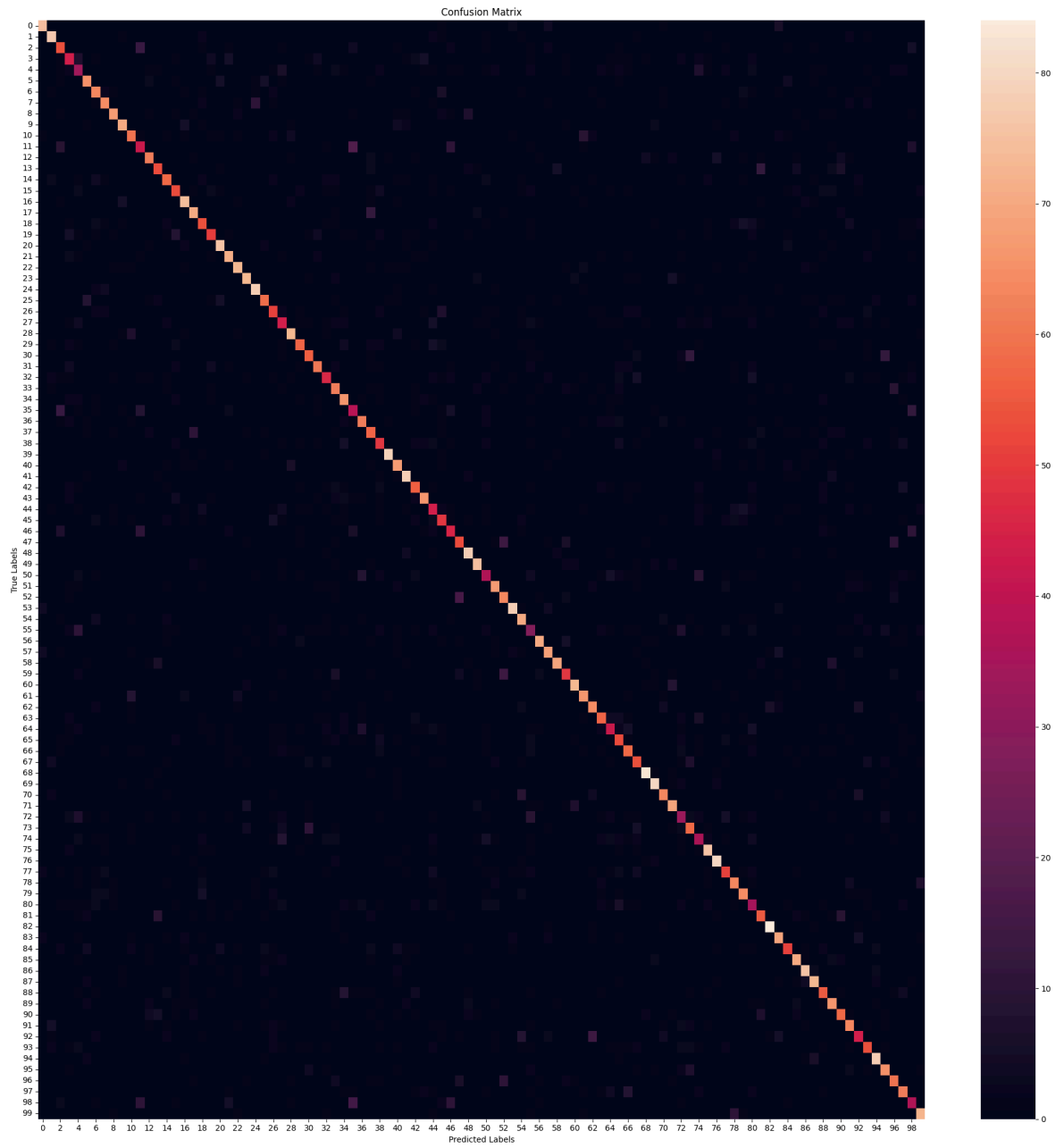
Epoch 13: val_loss did not improve from 1.19492

1314/1314 ————— **34s** 26ms/step - accuracy: 0.8472 - f1_score: 0.846
1 - loss: 0.5155 - precision: 0.9215 - val_accuracy: 0.6733 - val_f1_score: 0.671
8 - val_loss: 1.1964 - val_precision: 0.7866 - learning_rate: 5.0000e-06



141/141  **11s** 39ms/step - accuracy: 0.6676 - f1_score: 0.6588
- loss: 1.2010 - precision: 0.7866
Test Accuracy: 67.57%
Test Loss: 1.1776788234710693
Test Precision: 79.41%
Test F1 Scores (Per Class): [84.26965 81.05263 56.544495 49.71751 36.756752 7
1.03825 65.62499
73.56321 79.28994 80. 67.415726 46.66666 71.26436 63.09523
66.28571 60.227264 77.89473 75.675674 60.67415 60.60605 82.608696
75.93583 80.21977 81.11111 84.32432 69.047615 56.043945 47.567562
80.66298 68.67469 64.739876 71.764694 56.097557 61.38614 67.6923
43.093918 70.05649 60.31745 54.748596 90.28571 69.430046 82.53968
61.87845 76.74419 46.31578 55.05617 51.72413 61.9883 82.53968
82.41757 44.848476 74.86033 65.97938 87.15083 70.35175 32.558136
81.39534 75.55555 81.6568 59.39393 81.56424 74.157295 69.94535
65.517235 50.602406 56.084652 63.73626 59.999996 88.770035 85.10638
72.41379 75.40983 39.053246 62.032085 42.10526 81.72042 85.10638
60.714275 69.99999 69.18918 37.837833 62.499992 91.304344 72.63157
63.354034 74.8663 82.608696 78.918915 62.85714 69.10995 63.387978
67.37968 53.333324 62.427734 84.7826 70.65217 63.492058 68.508286
42.28571 78.68852]
Average Test F1 Scores:67.43

282/282  **11s** 27ms/step



	precision	recall	f1-score	support
beaver	0.85	0.83	0.84	90
dolphin	0.77	0.86	0.81	90
otter	0.53	0.60	0.57	90
seal	0.51	0.49	0.50	90
whale	0.36	0.38	0.37	90
aquarium fish	0.70	0.72	0.71	90
flatfish	0.62	0.70	0.66	90
ray	0.76	0.71	0.74	90
shark	0.85	0.74	0.79	90
trout	0.82	0.78	0.80	90
orchids	0.68	0.67	0.67	90
poppies	0.47	0.47	0.47	90
roses	0.74	0.69	0.71	90
sunflowers	0.68	0.59	0.63	90
tulips	0.68	0.64	0.66	90
bottles	0.62	0.59	0.60	90
bowls	0.74	0.82	0.78	90
cans	0.74	0.78	0.76	90
cups	0.61	0.60	0.61	90
plates	0.67	0.56	0.61	90
apples	0.81	0.84	0.83	90
mushrooms	0.73	0.79	0.76	90
oranges	0.79	0.81	0.80	90
pears	0.81	0.81	0.81	90
sweet peppers	0.82	0.87	0.84	90
clock	0.75	0.66	0.70	90
computer keyboard	0.55	0.57	0.56	90
lamp	0.47	0.49	0.48	90
telephone	0.80	0.81	0.81	90
television	0.75	0.63	0.69	90
bed	0.67	0.62	0.65	90
chair	0.76	0.68	0.72	90
couch	0.62	0.51	0.56	90
table	0.55	0.69	0.61	90
wardrobe	0.63	0.73	0.68	90
bee	0.43	0.43	0.43	90
beetle	0.71	0.69	0.70	90
butterfly	0.58	0.63	0.60	90
caterpillar	0.55	0.54	0.55	90
cockroach	0.93	0.88	0.90	90
bear	0.65	0.74	0.69	90
leopard	0.79	0.87	0.83	90
lion	0.62	0.62	0.62	90
tiger	0.80	0.73	0.77	90
wolf	0.44	0.49	0.46	90
bridge	0.56	0.54	0.55	90
castle	0.54	0.50	0.52	90
house	0.65	0.59	0.62	90
road	0.79	0.87	0.83	90
skyscraper	0.81	0.83	0.82	90
cloud	0.49	0.41	0.45	90
forest	0.75	0.74	0.75	90
mountain	0.61	0.70	0.65	90
plain	0.88	0.87	0.87	90
sea	0.64	0.78	0.70	90
camel	0.34	0.31	0.33	90
cattle	0.85	0.78	0.81	90
chimpanzee	0.76	0.76	0.76	90

elephant	0.87	0.77	0.82	90
kangaroo	0.65	0.54	0.59	90
fox	0.82	0.81	0.82	90
porcupine	0.75	0.73	0.74	90
possum	0.69	0.71	0.70	90
raccoon	0.68	0.63	0.66	90
skunk	0.55	0.47	0.51	90
crab	0.54	0.59	0.56	90
lobster	0.63	0.64	0.64	90
snail	0.60	0.60	0.60	90
spider	0.86	0.92	0.89	90
worm	0.82	0.89	0.85	90
baby	0.75	0.70	0.72	90
boy	0.74	0.77	0.75	90
girl	0.42	0.37	0.39	90
man	0.60	0.64	0.62	90
woman	0.44	0.40	0.42	90
crocodile	0.79	0.84	0.82	90
dinosaur	0.82	0.89	0.85	90
lizard	0.65	0.57	0.61	90
snake	0.70	0.70	0.70	90
turtle	0.67	0.71	0.69	90
hamster	0.37	0.39	0.38	90
mouse	0.64	0.61	0.62	90
rabbit	0.89	0.93	0.91	90
shrew	0.69	0.77	0.73	90
squirrel	0.72	0.57	0.63	90
maple	0.72	0.78	0.75	90
oak	0.81	0.84	0.83	90
palm	0.78	0.81	0.79	90
pine	0.65	0.61	0.63	90
willow	0.65	0.73	0.69	90
bicycle	0.62	0.64	0.63	90
bus	0.65	0.70	0.67	90
motorcycle	0.59	0.49	0.53	90
pickup truck	0.65	0.60	0.62	90
train	0.83	0.87	0.85	90
lawn-mower	0.69	0.72	0.71	90
rocket	0.61	0.67	0.63	90
streetcar	0.68	0.69	0.69	90
tank	0.44	0.41	0.42	90
tractor	0.77	0.80	0.79	90
accuracy			0.68	9000
macro avg	0.68	0.68	0.67	9000
weighted avg	0.68	0.68	0.67	9000

X_temp.shape: (8000, 32, 32, 3)

X_temp_val.shape: (4000, 32, 32, 3)

y_temp_val.shape: (4000, 1)

X_temp_test.shape: (4000, 32, 32, 3)

y_temp_test.shape: (4000, 1)

X_train.shape: (42000, 32, 32, 3)

y_train.shape: (42000, 1)

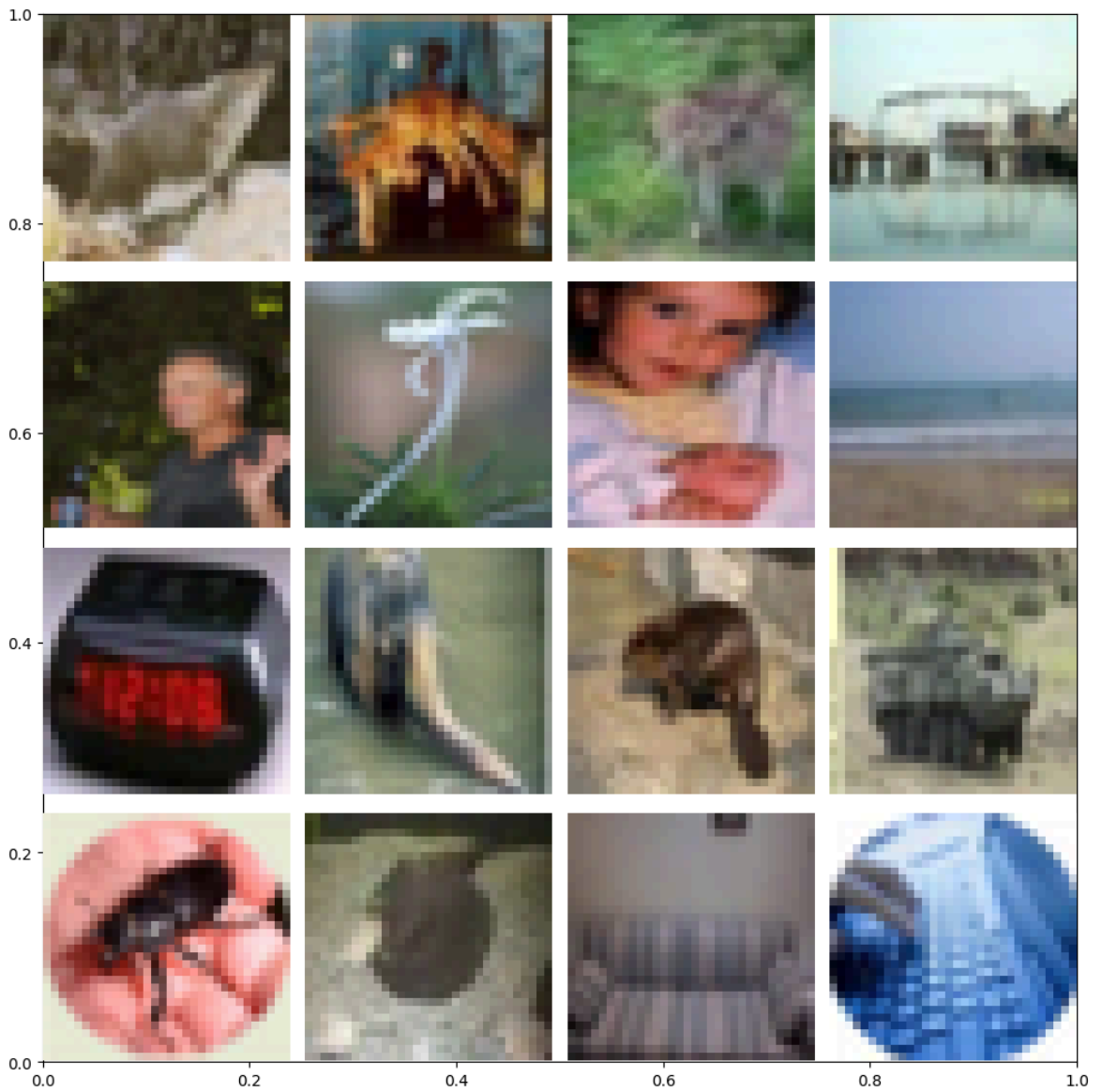
X_val.shape: (9000, 32, 32, 3)

y_val.shape: (9000, 1)

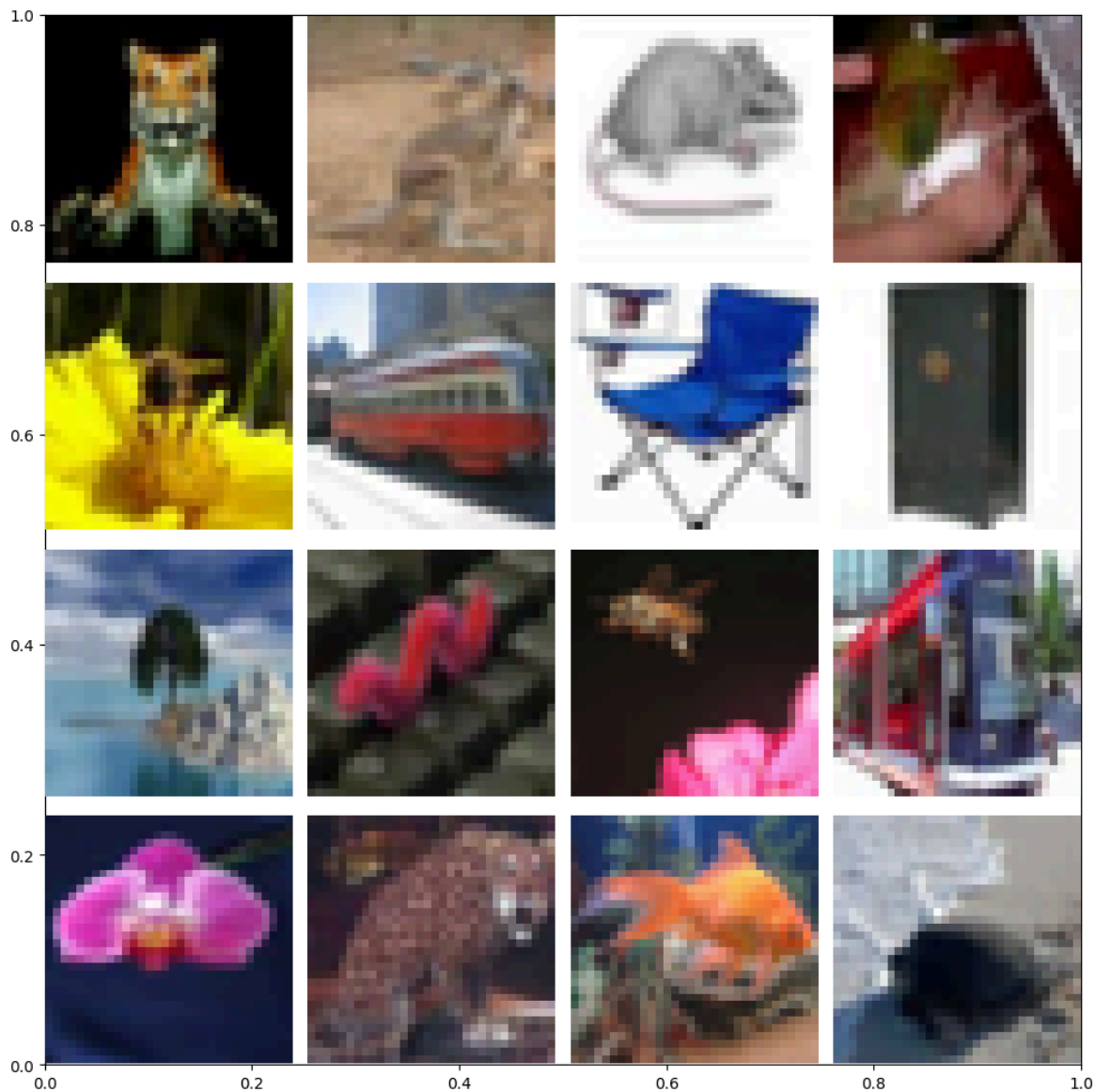
X_test.shape: (9000, 32, 32, 3)

y_test.shape: (9000, 1)

```
labels[37416].shape: (1,)  
imgs[37416].shape: (32, 32, 3)
```

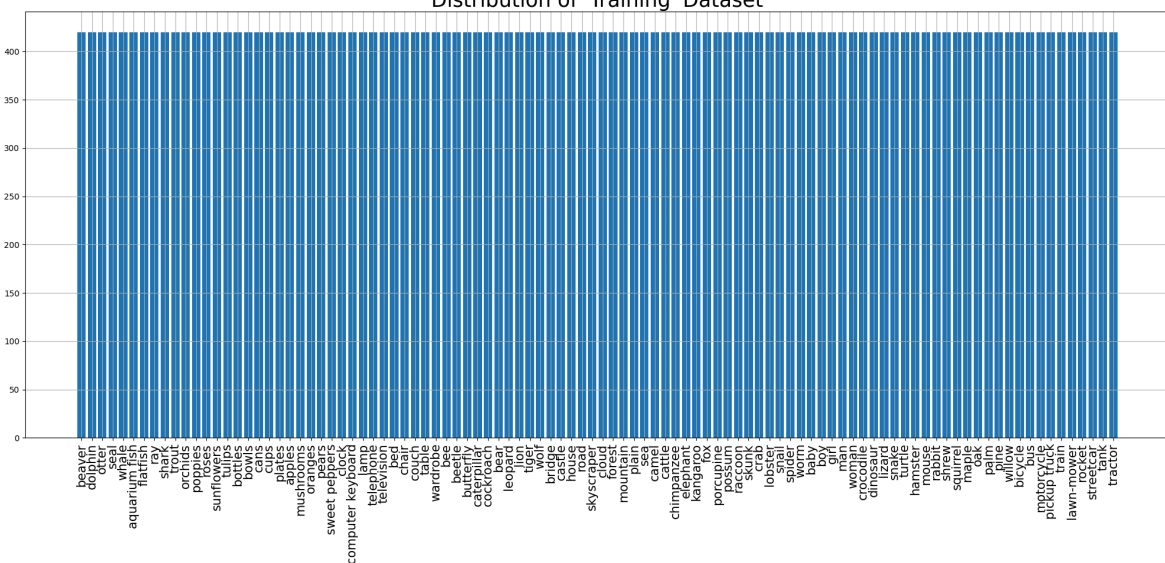


```
labels[34923].shape: (1,)  
imgs[34923].shape: (32, 32, 3)
```

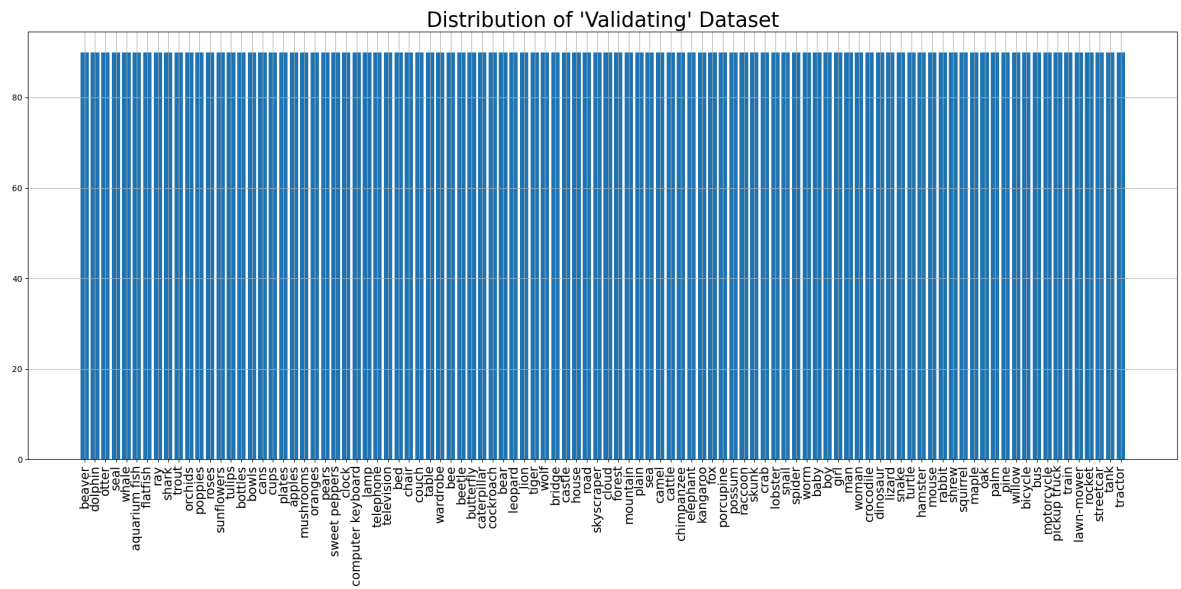


```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```

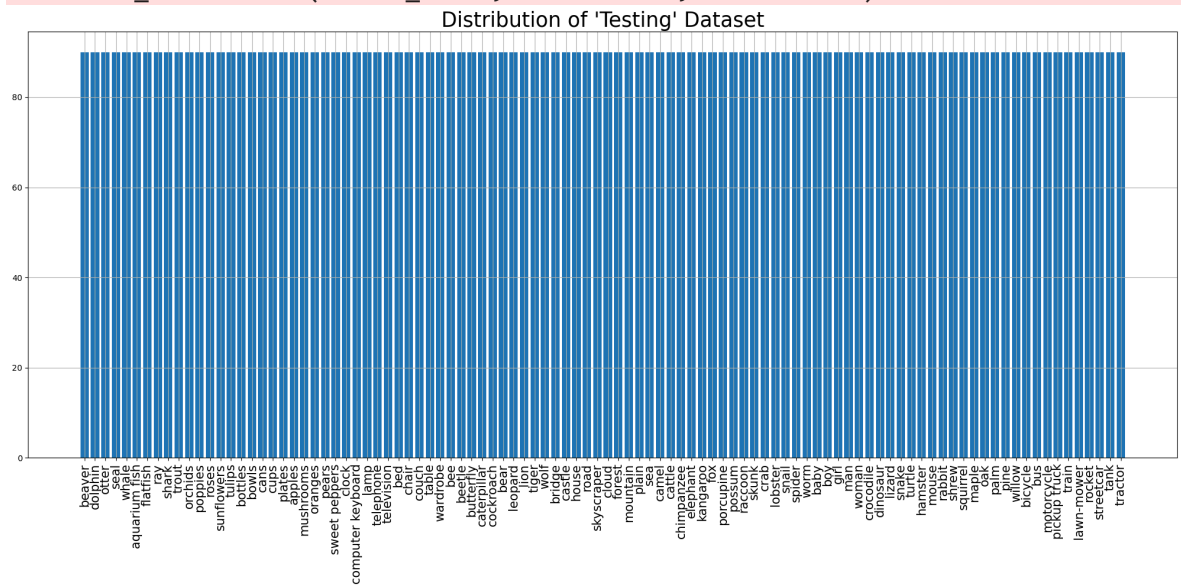
Distribution of 'Training' Dataset



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



Training dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
```

Validation dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
```

Testing dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
<class 'tuple'>
2
<class 'tensorflow.python.framework.ops.EagerTensor'> <class 'tensorflow.python.framework.ops.EagerTensor'>
(64, 224, 224, 3)
(64, 100)
Model input shape: (None, 224, 224, 3)
Model output shape: (None, 100)
2
<class 'numpy.ndarray'> <class 'numpy.ndarray'>
(64, 224, 224, 3) (64, 100)
```

Model: "sequential_3"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None , 7 , 7 , 2048)
global_average_pooling2d_3 (GlobalAveragePooling2D)	(None , 2048)
dense_6 (Dense)	(None , 512)
dropout_1 (Dropout)	(None , 512)
dense_7 (Dense)	(None , 100)

Total params: [24,665,188](#) (94.09 MB)

Trainable params: [1,100,388](#) (4.20 MB)

Non-trainable params: [23,564,800](#) (89.89 MB)

Epoch 1/25

657/657 ————— **0s** 29ms/step - accuracy: 0.3190 - f1_score: 0.3141 - loss: 2.8938 - precision: 0.6685

Epoch 1: val_loss improved from inf to 1.51700, saving model to best_model_v3.keras

657/657 ————— **40s** 45ms/step - accuracy: 0.3192 - f1_score: 0.3142 - loss: 2.8929 - precision: 0.6686 - val_accuracy: 0.5859 - val_f1_score: 0.5770 - val_loss: 1.5170 - val_precision: 0.8235 - learning_rate: 0.0010

Epoch 2/25

657/657 ————— **0s** 24ms/step - accuracy: 0.5241 - f1_score: 0.5202 - loss: 1.7370 - precision: 0.7475

Epoch 2: val_loss improved from 1.51700 to 1.41212, saving model to best_model_v3.keras

657/657 ————— **20s** 30ms/step - accuracy: 0.5241 - f1_score: 0.5202 - loss: 1.7369 - precision: 0.7475 - val_accuracy: 0.6101 - val_f1_score: 0.6061 - val_loss: 1.4121 - val_precision: 0.8082 - learning_rate: 0.0010

Epoch 3/25

657/657 ————— **0s** 24ms/step - accuracy: 0.5691 - f1_score: 0.5655 - loss: 1.5513 - precision: 0.7680

Epoch 3: val_loss improved from 1.41212 to 1.36015, saving model to best_model_v3.keras

657/657 ————— **20s** 30ms/step - accuracy: 0.5691 - f1_score: 0.5655 - loss: 1.5513 - precision: 0.7680 - val_accuracy: 0.6194 - val_f1_score: 0.6152 - val_loss: 1.3601 - val_precision: 0.8049 - learning_rate: 0.0010

Epoch 4/25

656/657 ————— **0s** 24ms/step - accuracy: 0.5857 - f1_score: 0.5820 - loss: 1.4525 - precision: 0.7757

Epoch 4: val_loss improved from 1.36015 to 1.32150, saving model to best_model_v3.keras

657/657 ————— **20s** 30ms/step - accuracy: 0.5857 - f1_score: 0.5820 - loss: 1.4524 - precision: 0.7758 - val_accuracy: 0.6267 - val_f1_score: 0.6243 - val_loss: 1.3215 - val_precision: 0.8036 - learning_rate: 0.0010

Epoch 5/25

656/657 ————— **0s** 24ms/step - accuracy: 0.6076 - f1_score: 0.6043 - loss: 1.3663 - precision: 0.7799

Epoch 5: val_loss improved from 1.32150 to 1.31312, saving model to best_model_v3.keras

657/657 ————— **20s** 30ms/step - accuracy: 0.6076 - f1_score: 0.6043 - loss: 1.3663 - precision: 0.7799 - val_accuracy: 0.6298 - val_f1_score: 0.6286 - val_loss: 1.3131 - val_precision: 0.8022 - learning_rate: 0.0010

Epoch 6/25

656/657 ————— **0s** 24ms/step - accuracy: 0.6238 - f1_score: 0.6207 - loss: 1.3023 - precision: 0.7933

Epoch 6: val_loss improved from 1.31312 to 1.31008, saving model to best_model_v3.keras

657/657 ————— **20s** 30ms/step - accuracy: 0.6238 - f1_score: 0.6208 - loss: 1.3022 - precision: 0.7933 - val_accuracy: 0.6339 - val_f1_score: 0.6320 - val_loss: 1.3101 - val_precision: 0.7960 - learning_rate: 0.0010

Epoch 7/25

657/657 ————— **0s** 24ms/step - accuracy: 0.6388 - f1_score: 0.6358 - loss: 1.2424 - precision: 0.7970

Epoch 7: val_loss improved from 1.31008 to 1.30405, saving model to best_model_v3.keras

657/657 ————— **20s** 30ms/step - accuracy: 0.6388 - f1_score: 0.6358 - loss: 1.2424 - precision: 0.7970 - val_accuracy: 0.6361 - val_f1_score: 0.6355 - val_loss: 1.3040 - val_precision: 0.7964 - learning_rate: 0.0010

Epoch 8/25

655/657 ————— **0s** 24ms/step - accuracy: 0.6446 - f1_score: 0.6422 - loss: 1.2072 - precision: 0.8028

Epoch 8: val_loss improved from 1.30405 to 1.29542, saving model to best_model_v

3.keras

657/657 ————— **20s** 30ms/step - accuracy: 0.6447 - f1_score: 0.6422
- loss: 1.2071 - precision: 0.8028 - val_accuracy: 0.6439 - val_f1_score: 0.6423
- val_loss: 1.2954 - val_precision: 0.7993 - learning_rate: 0.0010

Epoch 9/25

655/657 ————— **0s** 24ms/step - accuracy: 0.6549 - f1_score: 0.6518 -
loss: 1.1670 - precision: 0.8073

Epoch 9: val_loss improved from 1.29542 to 1.29385, saving model to best_model_v

3.keras

657/657 ————— **20s** 30ms/step - accuracy: 0.6549 - f1_score: 0.6519
- loss: 1.1669 - precision: 0.8074 - val_accuracy: 0.6388 - val_f1_score: 0.6374
- val_loss: 1.2938 - val_precision: 0.7899 - learning_rate: 0.0010

Epoch 10/25

656/657 ————— **0s** 24ms/step - accuracy: 0.6629 - f1_score: 0.6605 -
loss: 1.1330 - precision: 0.8075

Epoch 10: val_loss improved from 1.29385 to 1.29015, saving model to best_model_v

3.keras

657/657 ————— **20s** 30ms/step - accuracy: 0.6629 - f1_score: 0.6605
- loss: 1.1330 - precision: 0.8075 - val_accuracy: 0.6402 - val_f1_score: 0.6384
- val_loss: 1.2901 - val_precision: 0.7940 - learning_rate: 0.0010

Epoch 11/25

655/657 ————— **0s** 24ms/step - accuracy: 0.6718 - f1_score: 0.6691 -
loss: 1.0911 - precision: 0.8104

Epoch 11: val_loss improved from 1.29015 to 1.28081, saving model to best_model_v

3.keras

657/657 ————— **20s** 30ms/step - accuracy: 0.6718 - f1_score: 0.6691
- loss: 1.0910 - precision: 0.8104 - val_accuracy: 0.6413 - val_f1_score: 0.6403
- val_loss: 1.2808 - val_precision: 0.7892 - learning_rate: 0.0010

Epoch 12/25

657/657 ————— **0s** 24ms/step - accuracy: 0.6763 - f1_score: 0.6739 -
loss: 1.0679 - precision: 0.8119

Epoch 12: val_loss did not improve from 1.28081

657/657 ————— **19s** 28ms/step - accuracy: 0.6763 - f1_score: 0.6740
- loss: 1.0679 - precision: 0.8119 - val_accuracy: 0.6396 - val_f1_score: 0.6378
- val_loss: 1.2941 - val_precision: 0.7853 - learning_rate: 0.0010

Epoch 13/25

656/657 ————— **0s** 24ms/step - accuracy: 0.6860 - f1_score: 0.6835 -
loss: 1.0300 - precision: 0.8194

Epoch 13: val_loss improved from 1.28081 to 1.27896, saving model to best_model_v

3.keras

657/657 ————— **20s** 30ms/step - accuracy: 0.6860 - f1_score: 0.6835
- loss: 1.0299 - precision: 0.8194 - val_accuracy: 0.6466 - val_f1_score: 0.6436
- val_loss: 1.2790 - val_precision: 0.7944 - learning_rate: 0.0010

Epoch 14/25

655/657 ————— **0s** 24ms/step - accuracy: 0.6893 - f1_score: 0.6870 -
loss: 1.0151 - precision: 0.8197

Epoch 14: val_loss did not improve from 1.27896

657/657 ————— **19s** 28ms/step - accuracy: 0.6893 - f1_score: 0.6871
- loss: 1.0150 - precision: 0.8197 - val_accuracy: 0.6408 - val_f1_score: 0.6393
- val_loss: 1.3114 - val_precision: 0.7774 - learning_rate: 0.0010

Epoch 15/25

655/657 ————— **0s** 24ms/step - accuracy: 0.6945 - f1_score: 0.6920 -
loss: 0.9941 - precision: 0.8247

Epoch 15: val_loss did not improve from 1.27896

657/657 ————— **19s** 28ms/step - accuracy: 0.6946 - f1_score: 0.6921
- loss: 0.9940 - precision: 0.8247 - val_accuracy: 0.6446 - val_f1_score: 0.6424
- val_loss: 1.2985 - val_precision: 0.7808 - learning_rate: 0.0010

Epoch 16/25

656/657 ————— **0s** 24ms/step - accuracy: 0.6971 - f1_score: 0.6945 -
loss: 0.9810 - precision: 0.8238

Epoch 16: val_loss did not improve from 1.27896

Epoch 16: ReduceLROnPlateau reducing learning rate to 0.0005000000237487257.

657/657 ————— **19s** 28ms/step - accuracy: 0.6971 - f1_score: 0.6945
- loss: 0.9810 - precision: 0.8239 - val_accuracy: 0.6444 - val_f1_score: 0.6420
- val_loss: 1.2928 - val_precision: 0.7823 - learning_rate: 0.0010

Epoch 17/25

656/657 ————— **0s** 24ms/step - accuracy: 0.7334 - f1_score: 0.7313 -
loss: 0.8573 - precision: 0.8462

Epoch 17: val_loss improved from 1.27896 to 1.25818, saving model to best_model_v3.keras

657/657 ————— **20s** 31ms/step - accuracy: 0.7335 - f1_score: 0.7313
- loss: 0.8571 - precision: 0.8462 - val_accuracy: 0.6559 - val_f1_score: 0.6529
- val_loss: 1.2582 - val_precision: 0.7847 - learning_rate: 5.0000e-04

Epoch 18/25

655/657 ————— **0s** 24ms/step - accuracy: 0.7622 - f1_score: 0.7600 -
loss: 0.7619 - precision: 0.8675

Epoch 18: val_loss improved from 1.25818 to 1.24228, saving model to best_model_v3.keras

657/657 ————— **20s** 30ms/step - accuracy: 0.7623 - f1_score: 0.7601
- loss: 0.7617 - precision: 0.8675 - val_accuracy: 0.6651 - val_f1_score: 0.6628
- val_loss: 1.2423 - val_precision: 0.7926 - learning_rate: 5.0000e-04

Epoch 19/25

655/657 ————— **0s** 24ms/step - accuracy: 0.7728 - f1_score: 0.7707 -
loss: 0.7287 - precision: 0.8737

Epoch 19: val_loss did not improve from 1.24228

657/657 ————— **19s** 28ms/step - accuracy: 0.7728 - f1_score: 0.7708
- loss: 0.7286 - precision: 0.8738 - val_accuracy: 0.6597 - val_f1_score: 0.6572
- val_loss: 1.2467 - val_precision: 0.7875 - learning_rate: 5.0000e-04

Epoch 20/25

655/657 ————— **0s** 24ms/step - accuracy: 0.7828 - f1_score: 0.7805 -
loss: 0.6987 - precision: 0.8813

Epoch 20: val_loss did not improve from 1.24228

657/657 ————— **19s** 28ms/step - accuracy: 0.7828 - f1_score: 0.7805
- loss: 0.6986 - precision: 0.8813 - val_accuracy: 0.6598 - val_f1_score: 0.6595
- val_loss: 1.2600 - val_precision: 0.7840 - learning_rate: 5.0000e-04

Epoch 21/25

655/657 ————— **0s** 24ms/step - accuracy: 0.7841 - f1_score: 0.7826 -
loss: 0.6788 - precision: 0.8836

Epoch 21: val_loss did not improve from 1.24228

Epoch 21: ReduceLROnPlateau reducing learning rate to 0.0002500000118743628.

657/657 ————— **19s** 28ms/step - accuracy: 0.7841 - f1_score: 0.7826
- loss: 0.6787 - precision: 0.8837 - val_accuracy: 0.6587 - val_f1_score: 0.6570
- val_loss: 1.2498 - val_precision: 0.7830 - learning_rate: 5.0000e-04

Epoch 22/25

657/657 ————— **0s** 24ms/step - accuracy: 0.8058 - f1_score: 0.8041 -
loss: 0.6263 - precision: 0.8945

Epoch 22: val_loss improved from 1.24228 to 1.23258, saving model to best_model_v3.keras

657/657 ————— **20s** 30ms/step - accuracy: 0.8058 - f1_score: 0.8041
- loss: 0.6262 - precision: 0.8946 - val_accuracy: 0.6647 - val_f1_score: 0.6636
- val_loss: 1.2326 - val_precision: 0.7868 - learning_rate: 2.5000e-04

Epoch 23/25

657/657 ————— **0s** 24ms/step - accuracy: 0.8190 - f1_score: 0.8175 -
loss: 0.5814 - precision: 0.9007

Epoch 23: val_loss improved from 1.23258 to 1.23199, saving model to best_model_v3.keras

657/657 ————— **20s** 30ms/step - accuracy: 0.8190 - f1_score: 0.8175
- loss: 0.5813 - precision: 0.9007 - val_accuracy: 0.6652 - val_f1_score: 0.6649

- val_loss: 1.2320 - val_precision: 0.7897 - learning_rate: 2.5000e-04

Epoch 24/25

655/657 ————— **0s** 24ms/step - accuracy: 0.8267 - f1_score: 0.8250 - loss: 0.5560 - precision: 0.9101

Epoch 24: val_loss improved from 1.23199 to 1.23047, saving model to best_model_v3.keras

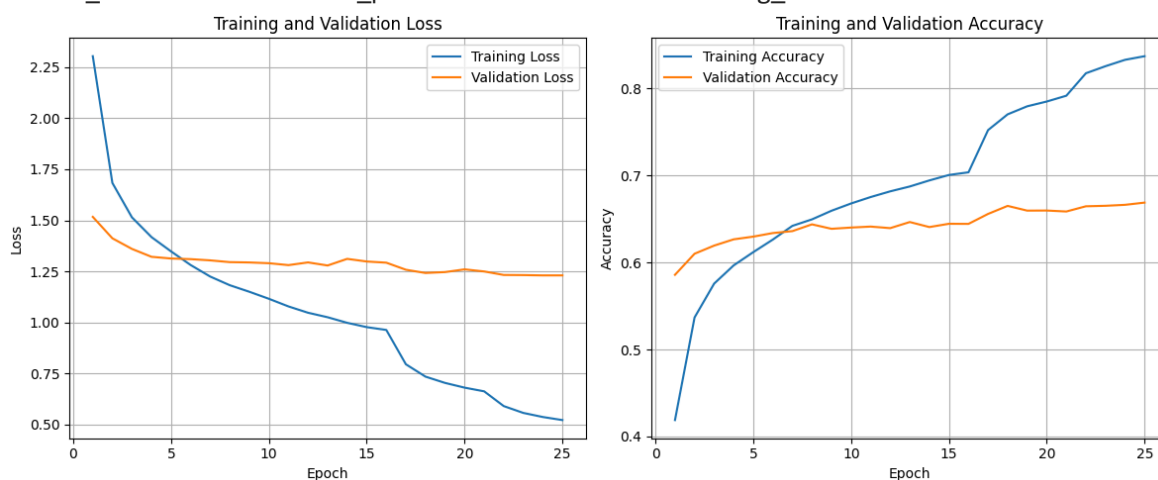
657/657 ————— **20s** 30ms/step - accuracy: 0.8267 - f1_score: 0.8250 - loss: 0.5559 - precision: 0.9101 - val_accuracy: 0.6663 - val_f1_score: 0.6647 - val_loss: 1.2305 - val_precision: 0.7862 - learning_rate: 2.5000e-04

Epoch 25/25

655/657 ————— **0s** 24ms/step - accuracy: 0.8296 - f1_score: 0.8282 - loss: 0.5438 - precision: 0.9093

Epoch 25: val_loss did not improve from 1.23047

657/657 ————— **19s** 28ms/step - accuracy: 0.8297 - f1_score: 0.8283 - loss: 0.5437 - precision: 0.9093 - val_accuracy: 0.6689 - val_f1_score: 0.6672 - val_loss: 1.2305 - val_precision: 0.7897 - learning_rate: 2.5000e-04



141/141 ————— **11s** 37ms/step - accuracy: 0.6609 - f1_score: 0.6529 - loss: 1.2480 - precision: 0.7791

Test Accuracy: 67.01%

Test Loss: 1.2199194431304932

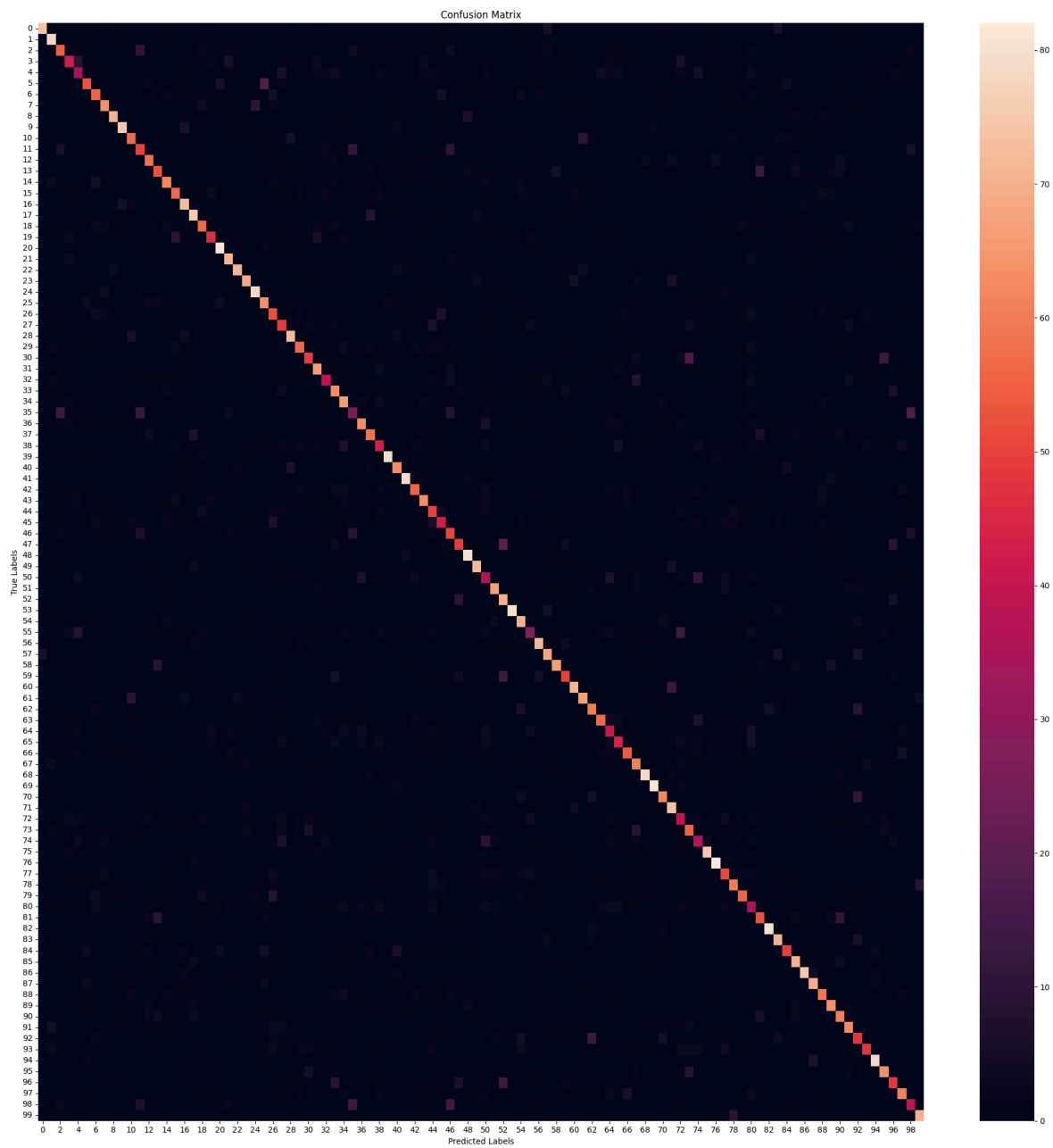
Test Precision: 78.83%

Test F1 Scores (Per Class): [83.615814 77.227715 62.14689 49.999992 39.999996 65.43209 59.259247

71.50838	82.08092	83.33333	65.14285	55.555553	65.53671	61.9883
68.888885	63.33333	74.87179	78.53403	62.29508	57.499992	84.37499
75.13227	80.22598	78.85714	83.422455	65.30612	51.999992	49.49494
80.66298	66.66666	60.1227	71.73912	49.689434	58.878494	67.34693
31.847128	71.18643	65.16853	50.588226	89.265526	67.74193	86.33879
66.666664	74.117645	49.999992	49.122803	53.124992	60.24096	82.90154
80.	38.74345	71.35134	66.66666	90.90909	70.29703	35.999996
80.89887	72.34043	79.99999	59.523804	81.14285	73.74301	66.666664
68.67469	46.06741	52.380943	63.905323	63.541664	87.64044	88.04347
71.26436	75.12689	42.162155	59.259247	43.181812	84.26965	88.17203
58.757053	69.36415	67.878784	36.082466	57.45856	89.887634	73.19587
59.39393	72.53885	84.74575	77.77777	67.42856	69.565216	65.59139
69.23077	53.33333	58.385086	87.15083	71.1111	55.172413	68.53932
45.61403	75.675674]					

Average Test F1 Scores:66.86

282/282 ————— **11s** 28ms/step



	precision	recall	f1-score	support
beaver	0.85	0.82	0.84	90
dolphin	0.70	0.87	0.77	90
otter	0.63	0.61	0.62	90
seal	0.54	0.47	0.50	90
whale	0.42	0.38	0.40	90
aquarium fish	0.74	0.59	0.65	90
flatfish	0.57	0.62	0.59	90
ray	0.72	0.71	0.72	90
shark	0.86	0.79	0.82	90
trout	0.83	0.83	0.83	90
orchids	0.67	0.63	0.65	90
poppies	0.56	0.56	0.56	90
roses	0.67	0.64	0.66	90
sunflowers	0.65	0.59	0.62	90
tulips	0.69	0.69	0.69	90
bottles	0.63	0.63	0.63	90
bowls	0.70	0.81	0.75	90
cans	0.74	0.83	0.79	90
cups	0.61	0.63	0.62	90
plates	0.66	0.51	0.57	90
apples	0.79	0.90	0.84	90
mushrooms	0.72	0.79	0.76	90
oranges	0.82	0.79	0.80	90
pears	0.81	0.77	0.79	90
sweet peppers	0.80	0.87	0.83	90
clock	0.60	0.71	0.65	90
computer keyboard	0.47	0.58	0.52	90
lamp	0.45	0.54	0.49	90
telephone	0.80	0.81	0.81	90
television	0.70	0.63	0.67	90
bed	0.67	0.54	0.60	90
chair	0.70	0.73	0.72	90
couch	0.56	0.44	0.50	90
table	0.51	0.70	0.59	90
wardrobe	0.62	0.73	0.67	90
bee	0.37	0.28	0.32	90
beetle	0.72	0.70	0.71	90
butterfly	0.66	0.64	0.65	90
caterpillar	0.54	0.48	0.51	90
cockroach	0.91	0.88	0.89	90
bear	0.66	0.70	0.68	90
leopard	0.85	0.88	0.86	90
lion	0.72	0.62	0.67	90
tiger	0.79	0.70	0.74	90
wolf	0.45	0.56	0.50	90
bridge	0.52	0.47	0.49	90
castle	0.50	0.57	0.53	90
house	0.65	0.54	0.59	90
road	0.78	0.89	0.83	90
skyscraper	0.80	0.80	0.80	90
cloud	0.37	0.41	0.39	90
forest	0.69	0.73	0.71	90
mountain	0.58	0.78	0.67	90
plain	0.93	0.89	0.91	90
sea	0.63	0.79	0.70	90
camel	0.45	0.30	0.36	90
cattle	0.82	0.80	0.81	90
chimpanzee	0.69	0.76	0.72	90

elephant	0.88	0.73	0.80	90
kangaroo	0.64	0.56	0.60	90
fox	0.84	0.79	0.81	90
porcupine	0.74	0.73	0.74	90
possum	0.67	0.67	0.67	90
raccoon	0.75	0.63	0.69	90
skunk	0.47	0.46	0.46	90
crab	0.56	0.49	0.52	90
lobster	0.68	0.60	0.64	90
snail	0.60	0.68	0.64	90
spider	0.89	0.87	0.88	90
worm	0.86	0.90	0.88	90
baby	0.74	0.69	0.71	90
boy	0.69	0.82	0.75	90
girl	0.41	0.43	0.42	90
man	0.57	0.62	0.59	90
woman	0.44	0.42	0.43	90
crocodile	0.85	0.83	0.84	90
dinosaur	0.85	0.91	0.88	90
lizard	0.59	0.57	0.58	90
snake	0.72	0.67	0.69	90
turtle	0.75	0.62	0.68	90
hamster	0.34	0.39	0.36	90
mouse	0.57	0.58	0.57	90
rabbit	0.91	0.89	0.90	90
shrew	0.68	0.79	0.73	90
squirrel	0.65	0.54	0.59	90
maple	0.68	0.78	0.73	90
oak	0.86	0.83	0.85	90
palm	0.78	0.78	0.78	90
pine	0.69	0.66	0.67	90
willow	0.68	0.71	0.70	90
bicycle	0.64	0.68	0.66	90
bus	0.68	0.70	0.69	90
motorcycle	0.53	0.53	0.53	90
pickup truck	0.67	0.53	0.59	90
train	0.88	0.87	0.87	90
lawn-mower	0.71	0.71	0.71	90
rocket	0.56	0.53	0.55	90
streetcar	0.69	0.68	0.69	90
tank	0.48	0.43	0.46	90
tractor	0.74	0.78	0.76	90
accuracy			0.67	9000
macro avg	0.67	0.67	0.67	9000
weighted avg	0.67	0.67	0.67	9000

Model: "sequential_3"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None , 7 , 7 , 2048)
global_average_pooling2d_3 (GlobalAveragePooling2D)	(None , 2048)
dense_6 (Dense)	(None , 512)
dropout_1 (Dropout)	(None , 512)
dense_7 (Dense)	(None , 100)



Total params: [24,665,188](#) (94.09 MB)

Trainable params: [1,100,388](#) (4.20 MB)

Non-trainable params: [23,564,800](#) (89.89 MB)

Epoch 1/25
1314/1314 ————— **0s** 26ms/step - accuracy: 0.8244 - f1_score: 0.8232
- loss: 0.5680 - precision: 0.9036
Epoch 1: val_loss improved from inf to 1.21142, saving model to best_model_fine_tune_v3.keras
1314/1314 ————— **56s** 35ms/step - accuracy: 0.8244 - f1_score: 0.8232 - loss: 0.5680 - precision: 0.9036 - val_accuracy: 0.6700 - val_f1_score: 0.6688 - val_loss: 1.2114 - val_precision: 0.7926 - learning_rate: 1.0000e-05

Epoch 2/25
1313/1314 ————— **0s** 24ms/step - accuracy: 0.8345 - f1_score: 0.8334 - loss: 0.5459 - precision: 0.9133
Epoch 2: val_loss improved from 1.21142 to 1.20769, saving model to best_model_fine_tune_v3.keras
1314/1314 ————— **35s** 27ms/step - accuracy: 0.8344 - f1_score: 0.8334 - loss: 0.5460 - precision: 0.9133 - val_accuracy: 0.6699 - val_f1_score: 0.6687 - val_loss: 1.2077 - val_precision: 0.7935 - learning_rate: 1.0000e-05

Epoch 3/25
1314/1314 ————— **0s** 24ms/step - accuracy: 0.8366 - f1_score: 0.8357 - loss: 0.5371 - precision: 0.9157
Epoch 3: val_loss improved from 1.20769 to 1.20576, saving model to best_model_fine_tune_v3.keras
1314/1314 ————— **35s** 27ms/step - accuracy: 0.8366 - f1_score: 0.8357 - loss: 0.5371 - precision: 0.9157 - val_accuracy: 0.6703 - val_f1_score: 0.6692 - val_loss: 1.2058 - val_precision: 0.7931 - learning_rate: 1.0000e-05

Epoch 4/25
1312/1314 ————— **0s** 24ms/step - accuracy: 0.8374 - f1_score: 0.8365 - loss: 0.5371 - precision: 0.9158
Epoch 4: val_loss improved from 1.20576 to 1.20446, saving model to best_model_fine_tune_v3.keras
1314/1314 ————— **35s** 27ms/step - accuracy: 0.8373 - f1_score: 0.8365 - loss: 0.5373 - precision: 0.9158 - val_accuracy: 0.6713 - val_f1_score: 0.6703 - val_loss: 1.2045 - val_precision: 0.7948 - learning_rate: 1.0000e-05

Epoch 5/25
1314/1314 ————— **0s** 24ms/step - accuracy: 0.8396 - f1_score: 0.8386 - loss: 0.5303 - precision: 0.9183
Epoch 5: val_loss improved from 1.20446 to 1.20407, saving model to best_model_fine_tune_v3.keras
1314/1314 ————— **35s** 27ms/step - accuracy: 0.8396 - f1_score: 0.8386 - loss: 0.5303 - precision: 0.9183 - val_accuracy: 0.6720 - val_f1_score: 0.6709 - val_loss: 1.2041 - val_precision: 0.7951 - learning_rate: 1.0000e-05

Epoch 6/25
1312/1314 ————— **0s** 24ms/step - accuracy: 0.8430 - f1_score: 0.8420 - loss: 0.5260 - precision: 0.9212
Epoch 6: val_loss improved from 1.20407 to 1.20400, saving model to best_model_fine_tune_v3.keras
1314/1314 ————— **35s** 27ms/step - accuracy: 0.8430 - f1_score: 0.8419 - loss: 0.5262 - precision: 0.9212 - val_accuracy: 0.6702 - val_f1_score: 0.6691 - val_loss: 1.2040 - val_precision: 0.7939 - learning_rate: 1.0000e-05

Epoch 7/25
1313/1314 ————— **0s** 24ms/step - accuracy: 0.8434 - f1_score: 0.8425 - loss: 0.5232 - precision: 0.9193
Epoch 7: val_loss did not improve from 1.20400
1314/1314 ————— **34s** 26ms/step - accuracy: 0.8433 - f1_score: 0.8424 - loss: 0.5233 - precision: 0.9193 - val_accuracy: 0.6710 - val_f1_score: 0.6698 - val_loss: 1.2042 - val_precision: 0.7947 - learning_rate: 1.0000e-05

Epoch 8/25
1312/1314 ————— **0s** 24ms/step - accuracy: 0.8425 - f1_score: 0.8415 - loss: 0.5218 - precision: 0.9181
Epoch 8: val_loss improved from 1.20400 to 1.20344, saving model to best_model_fine_tune_v3.keras

1314/1314 ————— **35s** 27ms/step - accuracy: 0.8425 - f1_score: 0.8414 - loss: 0.5220 - precision: 0.9181 - val_accuracy: 0.6714 - val_f1_score: 0.6703 - val_loss: 1.2034 - val_precision: 0.7943 - learning_rate: 1.0000e-05
Epoch 9/25

1313/1314 ————— **0s** 24ms/step - accuracy: 0.8460 - f1_score: 0.8449 - loss: 0.5145 - precision: 0.9220
Epoch 9: val_loss did not improve from 1.20344

1314/1314 ————— **34s** 26ms/step - accuracy: 0.8460 - f1_score: 0.8449 - loss: 0.5146 - precision: 0.9219 - val_accuracy: 0.6727 - val_f1_score: 0.6715 - val_loss: 1.2036 - val_precision: 0.7933 - learning_rate: 1.0000e-05
Epoch 10/25

1313/1314 ————— **0s** 24ms/step - accuracy: 0.8458 - f1_score: 0.8448 - loss: 0.5120 - precision: 0.9200
Epoch 10: val_loss improved from 1.20344 to 1.20276, saving model to best_model_fine_tune_v3.keras

1314/1314 ————— **35s** 27ms/step - accuracy: 0.8458 - f1_score: 0.8448 - loss: 0.5121 - precision: 0.9200 - val_accuracy: 0.6721 - val_f1_score: 0.6710 - val_loss: 1.2028 - val_precision: 0.7938 - learning_rate: 1.0000e-05
Epoch 11/25

1312/1314 ————— **0s** 24ms/step - accuracy: 0.8481 - f1_score: 0.8472 - loss: 0.5098 - precision: 0.9221
Epoch 11: val_loss improved from 1.20276 to 1.20273, saving model to best_model_fine_tune_v3.keras

1314/1314 ————— **35s** 27ms/step - accuracy: 0.8481 - f1_score: 0.8471 - loss: 0.5100 - precision: 0.9221 - val_accuracy: 0.6728 - val_f1_score: 0.6717 - val_loss: 1.2027 - val_precision: 0.7952 - learning_rate: 1.0000e-05
Epoch 12/25

1313/1314 ————— **0s** 24ms/step - accuracy: 0.8474 - f1_score: 0.8464 - loss: 0.5122 - precision: 0.9184
Epoch 12: val_loss did not improve from 1.20273

1314/1314 ————— **34s** 26ms/step - accuracy: 0.8473 - f1_score: 0.8463 - loss: 0.5123 - precision: 0.9184 - val_accuracy: 0.6722 - val_f1_score: 0.6712 - val_loss: 1.2033 - val_precision: 0.7958 - learning_rate: 1.0000e-05
Epoch 13/25

1313/1314 ————— **0s** 24ms/step - accuracy: 0.8496 - f1_score: 0.8486 - loss: 0.5017 - precision: 0.9229
Epoch 13: val_loss did not improve from 1.20273

Epoch 13: ReduceLROnPlateau reducing learning rate to 4.999999873689376e-06.

1314/1314 ————— **34s** 26ms/step - accuracy: 0.8495 - f1_score: 0.8486 - loss: 0.5018 - precision: 0.9229 - val_accuracy: 0.6728 - val_f1_score: 0.6717 - val_loss: 1.2028 - val_precision: 0.7947 - learning_rate: 1.0000e-05
Epoch 14/25

1313/1314 ————— **0s** 24ms/step - accuracy: 0.8511 - f1_score: 0.8500 - loss: 0.4996 - precision: 0.9221
Epoch 14: val_loss did not improve from 1.20273

1314/1314 ————— **34s** 26ms/step - accuracy: 0.8511 - f1_score: 0.8500 - loss: 0.4997 - precision: 0.9221 - val_accuracy: 0.6724 - val_f1_score: 0.6714 - val_loss: 1.2028 - val_precision: 0.7939 - learning_rate: 5.0000e-06
Epoch 15/25

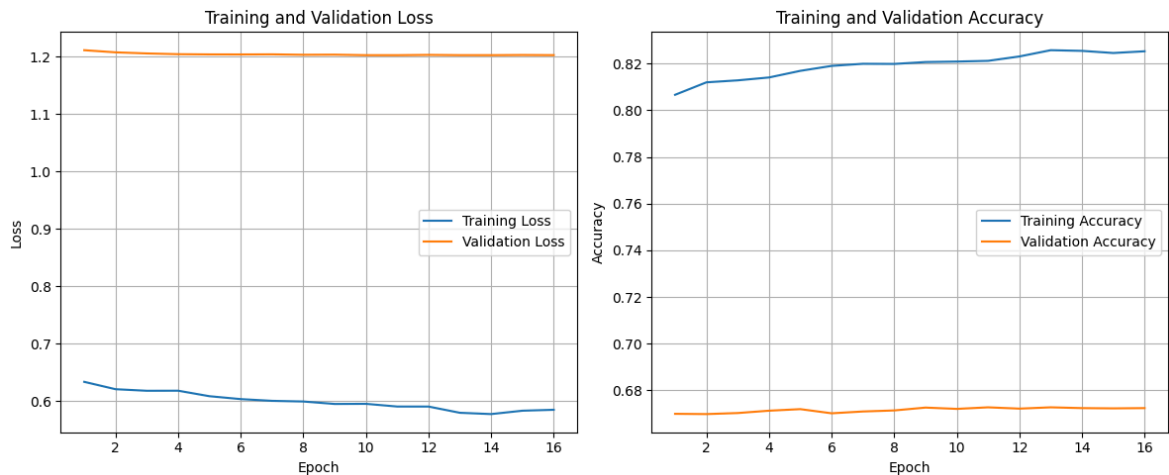
1312/1314 ————— **0s** 24ms/step - accuracy: 0.8483 - f1_score: 0.8473 - loss: 0.5040 - precision: 0.9218
Epoch 15: val_loss did not improve from 1.20273

1314/1314 ————— **34s** 26ms/step - accuracy: 0.8483 - f1_score: 0.8473 - loss: 0.5042 - precision: 0.9218 - val_accuracy: 0.6723 - val_f1_score: 0.6712 - val_loss: 1.2031 - val_precision: 0.7933 - learning_rate: 5.0000e-06
Epoch 16/25

1314/1314 ————— **0s** 24ms/step - accuracy: 0.8500 - f1_score: 0.8490 - loss: 0.5034 - precision: 0.9236
Epoch 16: val_loss did not improve from 1.20273

Epoch 16: ReduceLROnPlateau reducing learning rate to 2.499999936844688e-06.

1314/1314 ————— **34s** 26ms/step - accuracy: 0.8500 - f1_score: 0.849
0 - loss: 0.5035 - precision: 0.9235 - val_accuracy: 0.6724 - val_f1_score: 0.671
3 - val_loss: 1.2028 - val_precision: 0.7935 - learning_rate: 5.0000e-06



141/141 ————— **12s** 37ms/step - accuracy: 0.6626 - f1_score: 0.6543
- loss: 1.2195 - precision: 0.7891
Test Accuracy: 67.58%
Test Loss: 1.1938602924346924
Test Precision: 79.74%
Test F1 Scores (Per Class): [84.44444 77.61194 62.63736 47.953213 42.328033 7
0.329666 61.702126

69.71428	83.04093	82.22221	66.29213	52.439022	65.14285	60.919537
68.208084	62.06896	74.8663	79.14438	60.63829	60.1227	84.04254
76.8421	81.56424	77.9661	82.7957	67.77777	56.043945	52.261307
80.87432	70.658676	59.770107	73.56321	51.851852	61.16504	68.508286
36.809814	68.888885	62.032085	52.127655	90.28571	66.99999	85.55555
69.71428	71.91011	49.72375	50.847454	51.891888	62.275444	82.47422
80.	38.55421	74.59459	66.31015	91.01122	71.35678	37.426895
81.14285	74.31693	79.04191	61.9883	80.446915	74.4186	67.37968
69.36415	44.97041	54.748596	62.499992	62.85714	88.17203	86.48647
72.83237	77.65957	42.937847	60.73298	41.142853	83.33333	89.72973
58.100555	68.926544	64.13043	35.97883	59.550552	90.10989	74.8663
59.493668	74.19354	83.798874	78.65168	67.37968	67.357506	65.217384
72.52747	54.21686	63.09523	87.43169	68.888885	59.139782	69.66292
47.513805	78.68852					

Average Test F1 Scores:67.43

282/282 ————— **11s** 28ms/step



	precision	recall	f1-score	support
beaver	0.84	0.84	0.84	90
dolphin	0.70	0.87	0.78	90
otter	0.61	0.63	0.62	90
seal	0.51	0.46	0.48	90
whale	0.40	0.44	0.42	90
aquarium fish	0.70	0.71	0.70	90
flatfish	0.59	0.64	0.62	90
ray	0.72	0.68	0.70	90
shark	0.88	0.79	0.83	90
trout	0.82	0.82	0.82	90
orchids	0.67	0.66	0.66	90
poppies	0.58	0.48	0.52	90
roses	0.67	0.63	0.65	90
sunflowers	0.63	0.59	0.61	90
tulips	0.71	0.66	0.68	90
bottles	0.64	0.60	0.62	90
bowls	0.72	0.78	0.75	90
cans	0.76	0.82	0.79	90
cups	0.58	0.63	0.61	90
plates	0.67	0.54	0.60	90
apples	0.81	0.88	0.84	90
mushrooms	0.73	0.81	0.77	90
oranges	0.82	0.81	0.82	90
pears	0.79	0.77	0.78	90
sweet peppers	0.80	0.86	0.83	90
clock	0.68	0.68	0.68	90
computer keyboard	0.55	0.57	0.56	90
lamp	0.48	0.58	0.52	90
telephone	0.80	0.82	0.81	90
television	0.77	0.66	0.71	90
bed	0.62	0.58	0.60	90
chair	0.76	0.71	0.74	90
couch	0.58	0.47	0.52	90
table	0.54	0.70	0.61	90
wardrobe	0.68	0.69	0.69	90
bee	0.41	0.33	0.37	90
beetle	0.69	0.69	0.69	90
butterfly	0.60	0.64	0.62	90
caterpillar	0.50	0.54	0.52	90
cockroach	0.93	0.88	0.90	90
bear	0.61	0.74	0.67	90
leopard	0.86	0.86	0.86	90
lion	0.72	0.68	0.70	90
tiger	0.73	0.71	0.72	90
wolf	0.49	0.50	0.50	90
bridge	0.52	0.50	0.51	90
castle	0.51	0.53	0.52	90
house	0.68	0.58	0.62	90
road	0.77	0.89	0.82	90
skyscraper	0.80	0.80	0.80	90
cloud	0.42	0.36	0.39	90
forest	0.73	0.77	0.75	90
mountain	0.64	0.69	0.66	90
plain	0.92	0.90	0.91	90
sea	0.65	0.79	0.71	90
camel	0.40	0.36	0.37	90
cattle	0.84	0.79	0.81	90
chimpanzee	0.73	0.76	0.74	90

elephant	0.86	0.73	0.79	90
kangaroo	0.65	0.59	0.62	90
fox	0.81	0.80	0.80	90
porcupine	0.78	0.71	0.74	90
possum	0.65	0.70	0.67	90
raccoon	0.72	0.67	0.69	90
skunk	0.48	0.42	0.45	90
crab	0.55	0.54	0.55	90
lobster	0.64	0.61	0.62	90
snail	0.64	0.60	0.62	90
spider	0.85	0.91	0.88	90
worm	0.84	0.89	0.86	90
baby	0.76	0.70	0.73	90
boy	0.74	0.81	0.78	90
girl	0.44	0.42	0.43	90
man	0.57	0.64	0.61	90
woman	0.42	0.40	0.41	90
crocodile	0.83	0.83	0.83	90
dinosaur	0.87	0.92	0.90	90
lizard	0.58	0.58	0.58	90
snake	0.70	0.68	0.69	90
turtle	0.63	0.66	0.64	90
hamster	0.34	0.38	0.36	90
mouse	0.60	0.59	0.60	90
rabbit	0.89	0.91	0.90	90
shrew	0.72	0.78	0.75	90
squirrel	0.69	0.52	0.59	90
maple	0.72	0.77	0.74	90
oak	0.84	0.83	0.84	90
palm	0.80	0.78	0.79	90
pine	0.65	0.70	0.67	90
willow	0.63	0.72	0.67	90
bicycle	0.64	0.67	0.65	90
bus	0.72	0.73	0.73	90
motorcycle	0.59	0.50	0.54	90
pickup truck	0.68	0.59	0.63	90
train	0.86	0.89	0.87	90
lawn-mower	0.69	0.69	0.69	90
rocket	0.57	0.61	0.59	90
streetcar	0.70	0.69	0.70	90
tank	0.47	0.48	0.48	90
tractor	0.77	0.80	0.79	90
accuracy			0.68	9000
macro avg	0.68	0.68	0.67	9000
weighted avg	0.68	0.68	0.67	9000

Dropout(0.5), weight_decay(5e-2), l2(5e-2), lr

```
In [8]: for repeat_2_times in range(2):
        ##### <<<<<<<<<Load and process data>>>>>>>>>>>>
        # Load CIFAR-100 dataset
        (X_train, y_train), (X_test, y_test) = cifar100.load_data(label_mode='fine')

        # Split (8000) of training data into temporary set
```

```

X_temp, X_train, y_temp, y_train = train_test_split(X_train, y_train, test_s
print(f"X_temp.shape: {X_temp.shape}\n")

# Split temp data into equal validation (4000) and testing (4000) data
X_temp_val, X_temp_test, y_temp_val, y_temp_test = train_test_split(X_temp,
print(f"X_temp_val.shape: {X_temp_val.shape}")
print(f"y_temp_val.shape: {y_temp_val.shape}")
print(f"X_temp_test.shape: {X_temp_test.shape}")
print(f"y_temp_test.shape: {y_temp_test.shape}\n")

# Split test data into validation (5000) and testing (5000)
X_val, X_test, y_val, y_test = train_test_split(X_test, y_test, test_size=0.

# Add temp_val to validation (9000) and temp_test to testing (9000) to get a
X_val = np.concatenate((X_val, X_temp_val), axis=0)
y_val = np.concatenate((y_val, y_temp_val), axis=0)
X_test = np.concatenate((X_test, X_temp_test), axis=0)
y_test = np.concatenate((y_test, y_temp_test), axis=0)

print(f"X_train.shape: {X_train.shape}")
print(f"y_train.shape: {y_train.shape}")
print(f"X_val.shape: {X_val.shape}")
print(f"y_val.shape: {y_val.shape}")
print(f"X_test.shape: {X_test.shape}")
print(f"y_test.shape: {y_test.shape}\n")

display_imgs(X_train, y_train)

# Normalise images (scale to range [0, 1]) - Improves convergence speed & ac
X_train, X_val, X_test = X_train / 255.0, X_val / 255.0, X_test / 255.0
display_imgs(X_train, y_train)

labels_names = ['beaver', 'dolphin', 'otter', 'seal', 'whale', 'aquarium fish', 'f
                'orchids', 'poppies', 'roses', 'sunflowers', 'tulips', 'bottles', '
                'apples', 'mushrooms', 'oranges', 'pears', 'sweet peppers', 'clock
                'telephone', 'television', 'bed', 'chair', 'couch', 'table', 'wardr
                'caterpillar', 'cockroach', 'bear', 'leopard', 'lion', 'tiger', 'wo
                'road', 'skyscraper', 'cloud', 'forest', 'mountain', 'plain', 'sea'
                'elephant', 'kangaroo', 'fox', 'porcupine', 'possum', 'raccoon', 's
                'spider', 'worm', 'baby', 'boy', 'girl', 'man', 'woman', 'crocodile'
                'turtle', 'hamster', 'mouse', 'rabbit', 'shrew', 'squirrel', 'maple
                'bicycle', 'bus', 'motorcycle', 'pickup truck', 'train', 'lawn-mow
                'tractor']

class_distrib(y_train, labels_names, "Training")
class_distrib(y_val, labels_names, "Validating")
class_distrib(y_test, labels_names, "Testing")

# Create TensorFlow datasets

batch_size = 64

train_dataset_ = (tf.data.Dataset.from_tensor_slices((X_train, y_train))
                  .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                     tf.squeeze(tf.one_hot(y, depth=100, dtype=
                  .batch(batch_size)
                  .prefetch(tf.data.experimental.AUTOTUNE)))

# DEBUG augment
#train_dataset_aug = (tf.data.Dataset.from_tensor_slices((X_train, y_train))

```

```

#             .map(augment_dataset, num_parallel_calls=tf.data.experimental.
#             .batch(batch_size)
#             .prefetch(tf.data.experimental.AUTOTUNE))

# Combine the original dataset and the augmented dataset
train_dataset = train_dataset#.concatenate(train_dataset_aug)

val_dataset = (tf.data.Dataset.from_tensor_slices((X_val, y_val))
               .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                   tf.squeeze(tf.one_hot(y, depth=100, dtype=
               .batch(batch_size)
               .prefetch(tf.data.experimental.AUTOTUNE)))

test_dataset = (tf.data.Dataset.from_tensor_slices((X_test, y_test))
                .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                   tf.squeeze(tf.one_hot(y, depth=100, dtype=
                .batch(batch_size)
                .prefetch(tf.data.experimental.AUTOTUNE)))

print(f"Training dataset:\n {train_dataset}")
for img, lbl in train_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}") # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}") # Should be (batch_size, 10)
    del img, lbl
print(f"\nValidation dataset:\n {val_dataset}")
for img, lbl in val_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}") # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}") # Should be (batch_size, 10)
    del img, lbl
print(f"\nTesting dataset:\n {test_dataset}")
for img, lbl in test_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}") # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}") # Should be (batch_size, 10)
    del img, lbl

#### <<<<<<<<<Pre-trained model>>>>>>>>>
# Load ResNet50 pre-trained on ImageNet (w/out the top classification layer
resnet_50_base = tf.keras.applications.ResNet50V2(weights='imagenet', includ

# Freeze the Layers of VGG16 so they don't get updated during training - can
resnet_50_base.trainable = False

for layer in resnet_50_base.layers:
    if isinstance(layer, tf.keras.layers.Conv2D):
        layer.kernel_regularizer = tf.keras.regularizers.l2(5e-2) # DEBUG L2

# Add custom classification layers for CIFAR-100 (100 classes) - adapt model
model = models.Sequential([
    resnet_50_base,
    layers.GlobalAveragePooling2D(), # Better for ResNet than Flatten
    layers.Dense(512, activation='relu'),
    layers.Dropout(0.5), # DEBUG dropout
    layers.Dense(100, activation='softmax') # CIFAR-100 has 100 classes
])

for sample in test_dataset.take(1):

```

```

print(type(sample)) # Should be <class 'tuple'>
print(len(sample)) # Should be 2
print(type(sample[0]), type(sample[1])) # Both should be <class 'tensor'
print(sample[0].shape) # Should be (batch_size, 224, 224, 3)
print(sample[1].shape) # Should be (batch_size, 100)
print(f"Model input shape: {model.input_shape}")
print(f"Model output shape: {model.output_shape}")
sample = next(iter(test_dataset.as_numpy_iterator()))
print(len(sample)) # Should be 2
print(type(sample[0]), type(sample[1])) # Both should be <class 'numpy.ndarray'
print(sample[0].shape, sample[1].shape) # Should match model input and output
print("\n")
#for x, y in test_dataset.take(1):
#    print(type(x), type(y)) # Both should be <class 'tensorflow.Tensor'>
#for x_batch, y_batch in test_dataset.take(1):
#    test_loss, test_acc = model.evaluate(x_batch, y_batch)
#    print(f"Test Loss: {test_loss}, Test Accuracy: {test_acc}")

# Compile the model
#tensorboard_callback = keras.callbacks.TensorBoard(log_dir="./Logs")
model.compile(optimizer=optimizers.Adam(learning_rate=1e-3, weight_decay=5e-5),
              loss='categorical_crossentropy',
              metrics=['accuracy', 'precision', 'f1_score'])

model.summary()

#### <<<<<<<<Train Model & Track Training/Validation Error>>>>>>>>
early_stopping = EarlyStopping(monitor='val_loss', # or val_accuracy
                               patience=5, # Num. epochs with no improvement
                               restore_best_weights=True)
reduce_lr = ReduceLROnPlateau(monitor='val_loss', # or val_accuracy
                              factor=0.5, # Reduce lr by a factor
                              patience=3, # Num epochs w/ no improvement
                              min_lr=1e-6, # Min lr
                              verbose=1)

#tensorboard = TensorBoard(log_dir='./logs', # Logs directory
#                           histogram_freq=1, # Logs histograms for weights/acc
#                           write_graph=True, # Logs graph of model
#                           write_images=True) # Log images like weight histograms
checkpoint = ModelCheckpoint('best_model_v4.keras', # Path where the best model is saved
                             monitor='val_loss', # What to monitor (validation loss)
                             save_best_only=True, # Only save model when val loss is best
                             mode='min', # Looking to minimize the validation loss
                             verbose=1) # Print out message when saving model
checkpoint_fine_tune = ModelCheckpoint('best_model_fine_tune_v4.keras', # Path where the best model is saved
                                       monitor='val_loss', # What to monitor (validation loss)
                                       save_best_only=True, # Only save model when val loss is best
                                       mode='min', # Looking to minimize the validation loss
                                       verbose=1) # Print out message when saving model
#csv_logger = CSVLogger('training_log.csv', separator=',', append=True) # Save training logs

# Train the model
history = model.fit(train_dataset, validation_data=val_dataset, epochs=25,
                   batch_size=batch_size, callbacks=[checkpoint, reduce_lr,
#csv_logger])

#### <<<<<<<<Plot Training & Validation Error>>>>>>>>

# Extract loss and accuracy
epochs = range(1, len(history.history['loss'])+1)
train_loss = history.history['loss']

```

```

val_loss = history.history['val_loss']
train_acc = history.history['accuracy']
val_acc = history.history['val_accuracy']

plot_evidence(epochs, train_loss, val_loss, train_acc, val_acc)

#### <<<<<<<<<Evaluate Model on Test Data>>>>>>>>>>>>

# Evaluate on test data
model = load_model('best_model_v4.keras')
results = model.evaluate(test_dataset)
test_loss = results[0]
test_acc = results[1]
test_precision = results[2]
test_f1_scores = results[3]
print(f"Test Accuracy: {test_acc*100:.2f}%")
print(f"Test Loss: {test_loss}")
print(f"Test Precision: {test_precision*100:.2f}%")
print(f"Test F1 Scores (Per Class): {test_f1_scores.numpy()*100}")
print(f"Average Test F1 Scores:{np.average(test_f1_scores.numpy()*100):.2f}\n")

#### <<<<<<<<<Generate Confusion Matrix>>>>>>>>>>>>

# Get predictions
X_test_revised = tf.image.resize(X_test, (224, 224))
y_pred = model.predict(X_test_revised)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true_classes = y_test.flatten()

# Compute confusion matrix
conf_matrix = confusion_matrix(y_true_classes, y_pred_classes)

# Plot confusion matrix
plt.figure(figsize=(20, 20))
sns.heatmap(conf_matrix) #cmap='Blues', fmt='d'
plt.xlabel('Predicted Labels')
plt.ylabel('True Labels')
plt.title('Confusion Matrix')
plt.tight_layout()
plt.show()

print(classification_report(y_true_classes, y_pred_classes, target_names=lab
#tensorboard --logdir==path_to_your_logs

# Create a DataFrame from the history of the training and store the epoch va
hist = pd.DataFrame(history.history)
hist['epoch'] = history.epoch

# Finally, display the hist DataFrame.
hist

#### <<<<<<<<<Fine-Tune>>>>>>>>>>>>
# Adapt Model
for layer in resnet_50_base.layers:
    layer.trainable = True # Allow layers to be updated

# Compile again w/ Lower Learning rate (prevents destroying Learned features
model.compile(optimizer=optimizers.Adam(learning_rate=1e-5),
              loss='categorical_crossentropy',
              metrics=['accuracy', 'precision', 'f1_score'])

```

```

model.summary()

#### <<<<<<<<Modify Dataset>>>>>>>>>>>>

# DEBUG augment
train_dataset_aug_ = (tf.data.Dataset.from_tensor_slices((X_train, y_train))
                      .map(augment_dataset, num_parallel_calls=tf.data.experimental
                           .batch(batch_size)
                           .prefetch(tf.data.experimental.AUTOTUNE)))

# Combine the original dataset and the augmented dataset
train_dataset_aug = train_dataset.concatenate(train_dataset_aug_) # Not val

#### <<<<<<<<Train Model & Track Training/Validation Error>>>>>>>>>>>>

# Train the model
history_fine_tune = model.fit(train_dataset_aug, validation_data=val_dataset
                              batch_size=batch_size, callbacks=[checkpoint_f

#### <<<<<<<<Plot Training & Validation Error>>>>>>>>>>>>

# Extract loss and accuracy
epochs = range(1, len(history_fine_tune.history['loss'])+1)
train_loss = history_fine_tune.history['loss']
val_loss = history_fine_tune.history['val_loss']
train_acc = history_fine_tune.history['accuracy']
val_acc = history_fine_tune.history['val_accuracy']

plot_evidence(epochs, train_loss, val_loss, train_acc, val_acc)

#### <<<<<<<<Evaluate Model on Test Data>>>>>>>>>>>>

# Evaluate on test data
model_fine_tune = load_model('best_model_fine_tune_v4.keras')
results = model_fine_tune.evaluate(test_dataset)
test_loss = results[0]
test_acc = results[1]
test_precision = results[2]
test_f1_scores = results[3]
print(f"Test Accuracy: {test_acc*100:.2f}%")
print(f"Test Loss: {test_loss}")
print(f"Test Precision: {test_precision*100:.2f}%")
print(f"Test F1 Scores (Per Class): {test_f1_scores.numpy()*100}")
print(f"Average Test F1 Scores:{np.average(test_f1_scores.numpy()*100):.2f}")

#### <<<<<<<<Generate Confusion Matrix>>>>>>>>>>>>

# Get predictions
X_test_revised = tf.image.resize(X_test, (224, 224))
y_pred = model_fine_tune.predict(X_test_revised)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true_classes = y_test.flatten()

# Compute confusion matrix
conf_matrix = confusion_matrix(y_true_classes, y_pred_classes)

# Plot confusion matrix
plt.figure(figsize=(20, 20))

```

```
sns.heatmap(conf_matrix) #cmap='Blues', fmt='d'
plt.xlabel('Predicted Labels')
plt.ylabel('True Labels')
plt.title('Confusion Matrix')
plt.tight_layout()
plt.show()

print(classification_report(y_true_classes, y_pred_classes, target_names=lab

# Create a DataFrame from the history of the training and store the epoch va
hist = pd.DataFrame(history_fine_tune.history)
hist['epoch'] = history_fine_tune.epoch

# Finally, display the hist DataFrame.
hist
```

X_temp.shape: (8000, 32, 32, 3)

X_temp_val.shape: (4000, 32, 32, 3)

y_temp_val.shape: (4000, 1)

X_temp_test.shape: (4000, 32, 32, 3)

y_temp_test.shape: (4000, 1)

X_train.shape: (42000, 32, 32, 3)

y_train.shape: (42000, 1)

X_val.shape: (9000, 32, 32, 3)

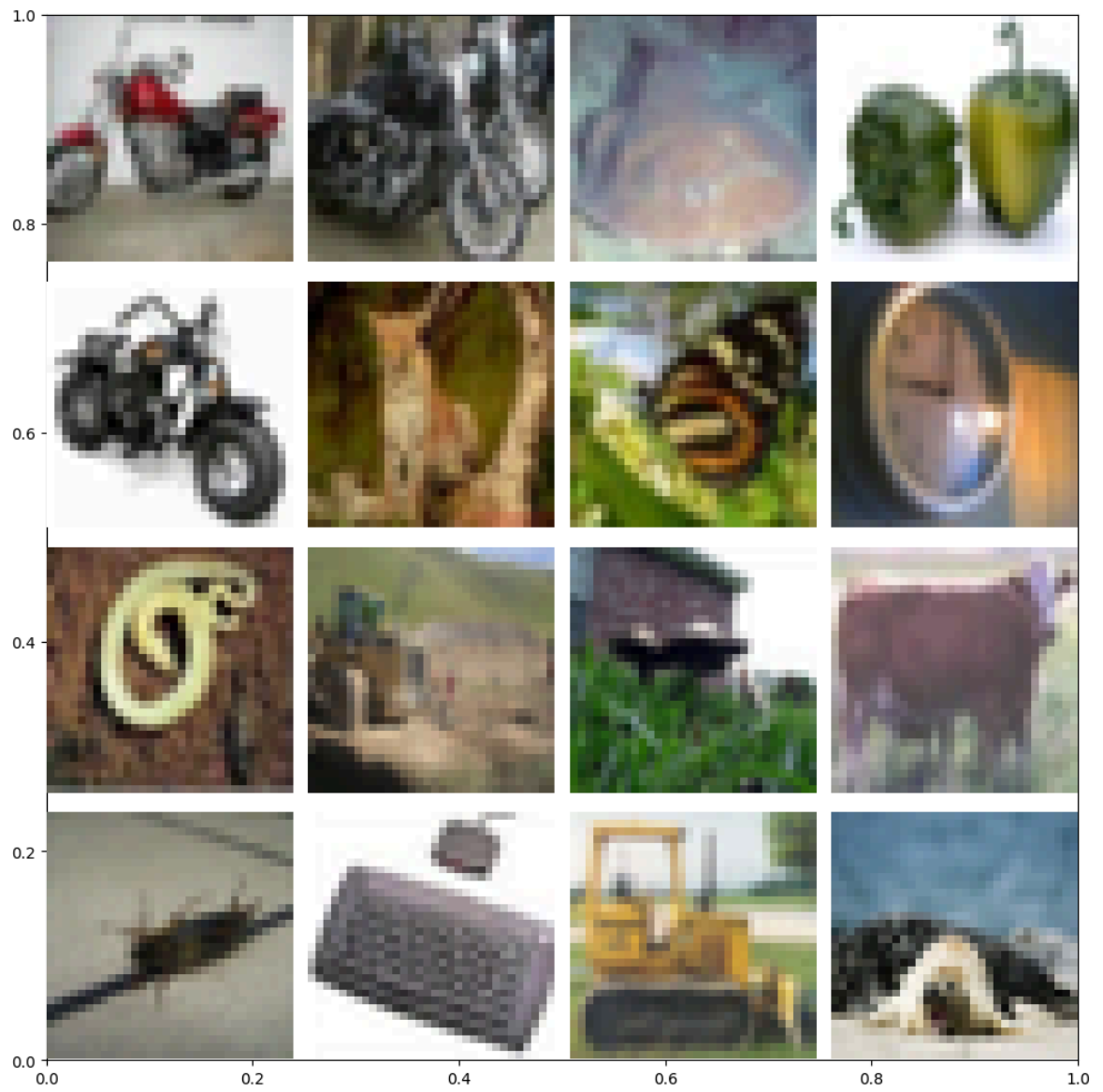
y_val.shape: (9000, 1)

X_test.shape: (9000, 32, 32, 3)

y_test.shape: (9000, 1)

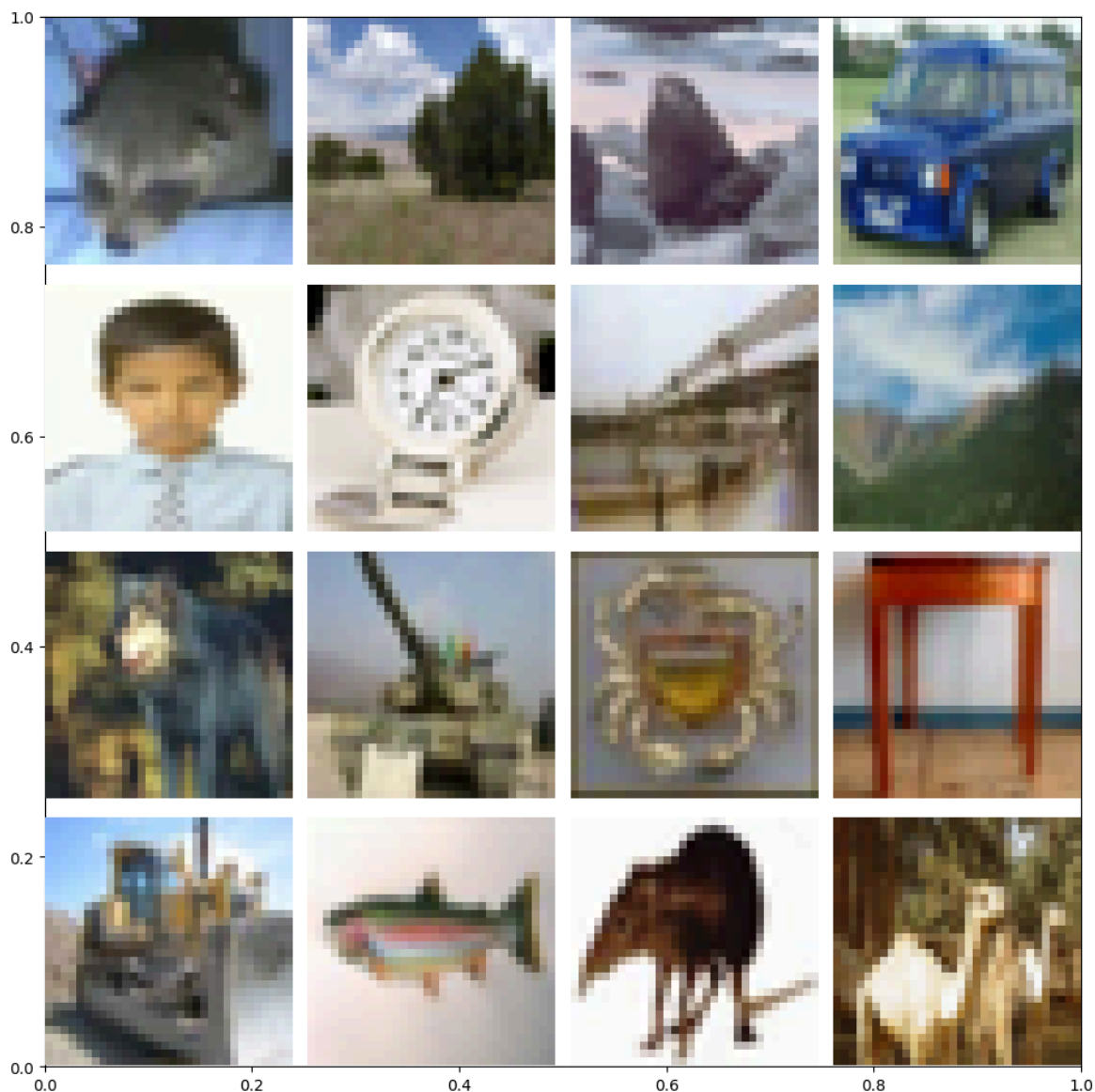
labels[36952].shape: (1,)

imgs[36952].shape: (32, 32, 3)



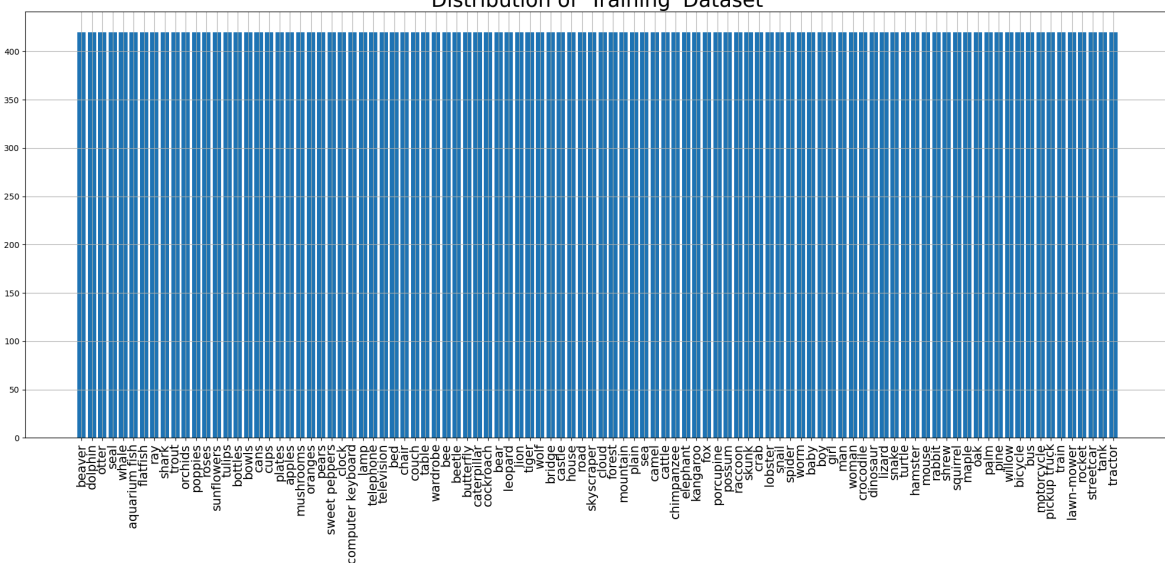
labels[14092].shape: (1,)

imgs[14092].shape: (32, 32, 3)

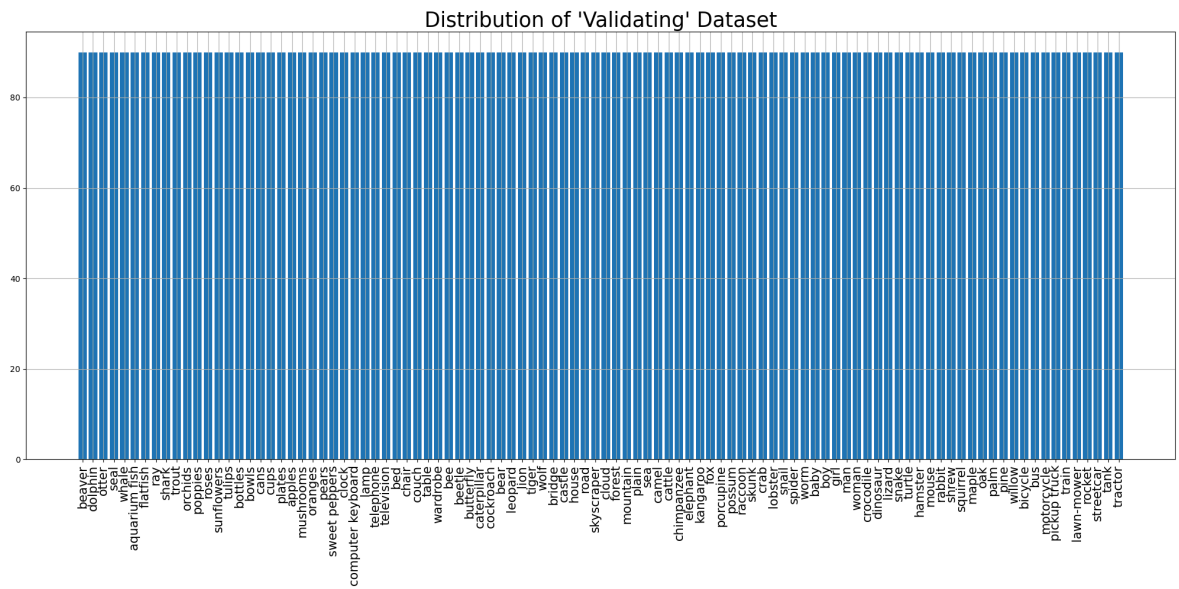


```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```

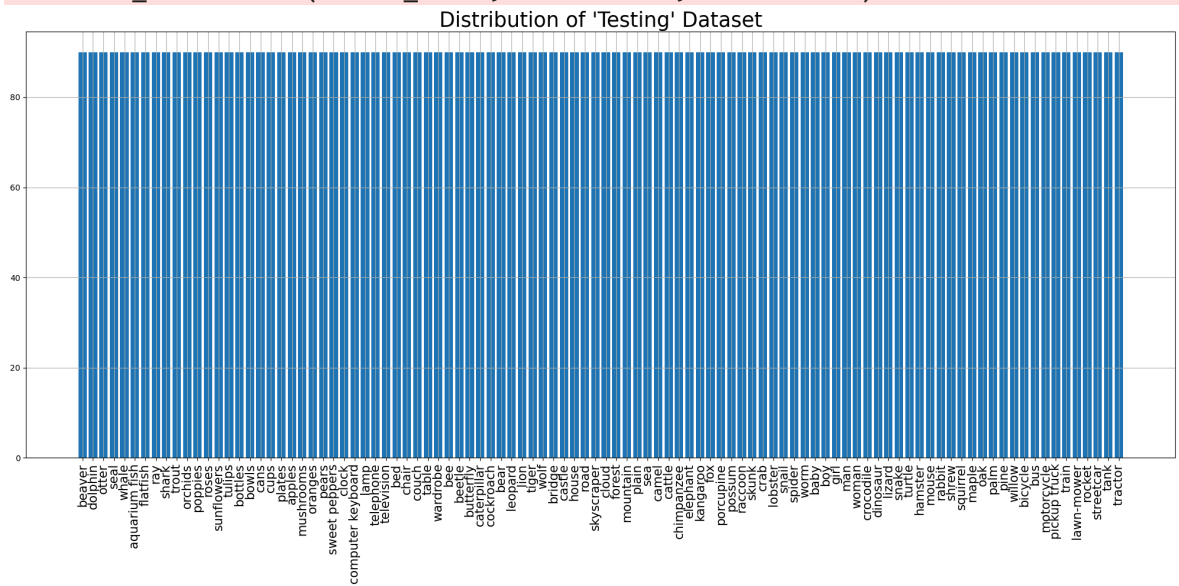
Distribution of 'Training' Dataset



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



Training dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
```

Validation dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
```

Testing dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
<class 'tuple'>
2
<class 'tensorflow.python.framework.ops.EagerTensor'> <class 'tensorflow.python.framework.ops.EagerTensor'>
(64, 224, 224, 3)
(64, 100)
Model input shape: (None, 224, 224, 3)
Model output shape: (None, 100)
2
<class 'numpy.ndarray'> <class 'numpy.ndarray'>
(64, 224, 224, 3) (64, 100)
```

Model: "sequential_4"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None , 7 , 7 , 2048)
global_average_pooling2d_4 (GlobalAveragePooling2D)	(None , 2048)
dense_8 (Dense)	(None , 512)
dropout_2 (Dropout)	(None , 512)
dense_9 (Dense)	(None , 100)

Total params: [24,665,188](#) (94.09 MB)

Trainable params: [1,100,388](#) (4.20 MB)

Non-trainable params: [23,564,800](#) (89.89 MB)

Epoch 1/25
657/657 ————— **0s** 29ms/step - accuracy: 0.3538 - f1_score: 0.3488 - loss: 2.6949 - precision: 0.6913
Epoch 1: val_loss improved from inf to 1.47143, saving model to best_model_v4.keras
657/657 ————— **40s** 45ms/step - accuracy: 0.3539 - f1_score: 0.3489 - loss: 2.6941 - precision: 0.6914 - val_accuracy: 0.5913 - val_f1_score: 0.5871 - val_loss: 1.4714 - val_precision: 0.8048 - learning_rate: 0.0010
Epoch 2/25
656/657 ————— **0s** 24ms/step - accuracy: 0.5662 - f1_score: 0.5623 - loss: 1.5745 - precision: 0.7630
Epoch 2: val_loss improved from 1.47143 to 1.37340, saving model to best_model_v4.keras
657/657 ————— **20s** 30ms/step - accuracy: 0.5662 - f1_score: 0.5624 - loss: 1.5743 - precision: 0.7630 - val_accuracy: 0.6134 - val_f1_score: 0.6096 - val_loss: 1.3734 - val_precision: 0.8034 - learning_rate: 0.0010
Epoch 3/25
656/657 ————— **0s** 23ms/step - accuracy: 0.6028 - f1_score: 0.5997 - loss: 1.3842 - precision: 0.7813
Epoch 3: val_loss improved from 1.37340 to 1.34628, saving model to best_model_v4.keras
657/657 ————— **19s** 30ms/step - accuracy: 0.6028 - f1_score: 0.5997 - loss: 1.3841 - precision: 0.7813 - val_accuracy: 0.6191 - val_f1_score: 0.6151 - val_loss: 1.3463 - val_precision: 0.7969 - learning_rate: 0.0010
Epoch 4/25
656/657 ————— **0s** 23ms/step - accuracy: 0.6341 - f1_score: 0.6314 - loss: 1.2625 - precision: 0.7946
Epoch 4: val_loss improved from 1.34628 to 1.33145, saving model to best_model_v4.keras
657/657 ————— **19s** 29ms/step - accuracy: 0.6342 - f1_score: 0.6315 - loss: 1.2624 - precision: 0.7946 - val_accuracy: 0.6250 - val_f1_score: 0.6228 - val_loss: 1.3314 - val_precision: 0.7878 - learning_rate: 0.0010
Epoch 5/25
657/657 ————— **0s** 23ms/step - accuracy: 0.6554 - f1_score: 0.6530 - loss: 1.1774 - precision: 0.8045
Epoch 5: val_loss improved from 1.33145 to 1.30498, saving model to best_model_v4.keras
657/657 ————— **20s** 31ms/step - accuracy: 0.6554 - f1_score: 0.6530 - loss: 1.1773 - precision: 0.8045 - val_accuracy: 0.6338 - val_f1_score: 0.6323 - val_loss: 1.3050 - val_precision: 0.7899 - learning_rate: 0.0010
Epoch 6/25
656/657 ————— **0s** 23ms/step - accuracy: 0.6807 - f1_score: 0.6784 - loss: 1.0702 - precision: 0.8196
Epoch 6: val_loss did not improve from 1.30498
657/657 ————— **18s** 28ms/step - accuracy: 0.6807 - f1_score: 0.6785 - loss: 1.0701 - precision: 0.8196 - val_accuracy: 0.6314 - val_f1_score: 0.6300 - val_loss: 1.3134 - val_precision: 0.7738 - learning_rate: 0.0010
Epoch 7/25
656/657 ————— **0s** 23ms/step - accuracy: 0.6959 - f1_score: 0.6938 - loss: 1.0045 - precision: 0.8239
Epoch 7: val_loss did not improve from 1.30498
657/657 ————— **18s** 28ms/step - accuracy: 0.6959 - f1_score: 0.6938 - loss: 1.0045 - precision: 0.8239 - val_accuracy: 0.6406 - val_f1_score: 0.6383 - val_loss: 1.3129 - val_precision: 0.7812 - learning_rate: 0.0010
Epoch 8/25
655/657 ————— **0s** 23ms/step - accuracy: 0.7069 - f1_score: 0.7048 - loss: 0.9576 - precision: 0.8262
Epoch 8: val_loss did not improve from 1.30498

Epoch 8: ReduceLROnPlateau reducing learning rate to 0.0005000000237487257.

657/657 ————— **18s** 28ms/step - accuracy: 0.7070 - f1_score: 0.7049
- loss: 0.9575 - precision: 0.8263 - val_accuracy: 0.6374 - val_f1_score: 0.6362
- val_loss: 1.3277 - val_precision: 0.7702 - learning_rate: 0.0010
Epoch 9/25

656/657 ————— **0s** 23ms/step - accuracy: 0.7472 - f1_score: 0.7455 -
loss: 0.8091 - precision: 0.8546
Epoch 9: val_loss improved from 1.30498 to 1.28034, saving model to best_model_v
4.keras

657/657 ————— **19s** 30ms/step - accuracy: 0.7473 - f1_score: 0.7455
- loss: 0.8089 - precision: 0.8547 - val_accuracy: 0.6577 - val_f1_score: 0.6573
- val_loss: 1.2803 - val_precision: 0.7755 - learning_rate: 5.0000e-04
Epoch 10/25

655/657 ————— **0s** 23ms/step - accuracy: 0.7798 - f1_score: 0.7779 -
loss: 0.7040 - precision: 0.8747
Epoch 10: val_loss improved from 1.28034 to 1.27529, saving model to best_model_v
4.keras

657/657 ————— **19s** 29ms/step - accuracy: 0.7798 - f1_score: 0.7779
- loss: 0.7038 - precision: 0.8747 - val_accuracy: 0.6567 - val_f1_score: 0.6558
- val_loss: 1.2753 - val_precision: 0.7765 - learning_rate: 5.0000e-04
Epoch 11/25

656/657 ————— **0s** 23ms/step - accuracy: 0.7941 - f1_score: 0.7923 -
loss: 0.6546 - precision: 0.8822
Epoch 11: val_loss did not improve from 1.27529

657/657 ————— **18s** 28ms/step - accuracy: 0.7942 - f1_score: 0.7923
- loss: 0.6545 - precision: 0.8822 - val_accuracy: 0.6579 - val_f1_score: 0.6567
- val_loss: 1.2819 - val_precision: 0.7739 - learning_rate: 5.0000e-04
Epoch 12/25

657/657 ————— **0s** 23ms/step - accuracy: 0.8093 - f1_score: 0.8075 -
loss: 0.6088 - precision: 0.8915
Epoch 12: val_loss did not improve from 1.27529

657/657 ————— **18s** 28ms/step - accuracy: 0.8093 - f1_score: 0.8075
- loss: 0.6088 - precision: 0.8915 - val_accuracy: 0.6531 - val_f1_score: 0.6515
- val_loss: 1.2915 - val_precision: 0.7657 - learning_rate: 5.0000e-04
Epoch 13/25

655/657 ————— **0s** 23ms/step - accuracy: 0.8208 - f1_score: 0.8185 -
loss: 0.5695 - precision: 0.8996
Epoch 13: val_loss did not improve from 1.27529

Epoch 13: ReduceLROnPlateau reducing learning rate to 0.0002500000118743628.

657/657 ————— **18s** 28ms/step - accuracy: 0.8208 - f1_score: 0.8185
- loss: 0.5695 - precision: 0.8996 - val_accuracy: 0.6559 - val_f1_score: 0.6554
- val_loss: 1.2874 - val_precision: 0.7670 - learning_rate: 5.0000e-04
Epoch 14/25

656/657 ————— **0s** 23ms/step - accuracy: 0.8390 - f1_score: 0.8374 -
loss: 0.5150 - precision: 0.9087
Epoch 14: val_loss improved from 1.27529 to 1.27402, saving model to best_model_v
4.keras

657/657 ————— **19s** 30ms/step - accuracy: 0.8391 - f1_score: 0.8375
- loss: 0.5149 - precision: 0.9088 - val_accuracy: 0.6608 - val_f1_score: 0.6601
- val_loss: 1.2740 - val_precision: 0.7683 - learning_rate: 2.5000e-04
Epoch 15/25

657/657 ————— **0s** 23ms/step - accuracy: 0.8545 - f1_score: 0.8526 -
loss: 0.4659 - precision: 0.9180
Epoch 15: val_loss did not improve from 1.27402

657/657 ————— **18s** 28ms/step - accuracy: 0.8545 - f1_score: 0.8526
- loss: 0.4659 - precision: 0.9180 - val_accuracy: 0.6629 - val_f1_score: 0.6625
- val_loss: 1.2856 - val_precision: 0.7631 - learning_rate: 2.5000e-04
Epoch 16/25

655/657 ————— **0s** 23ms/step - accuracy: 0.8610 - f1_score: 0.8591 -
loss: 0.4458 - precision: 0.9217

Epoch 16: val_loss did not improve from 1.27402

657/657 ————— **18s** 28ms/step - accuracy: 0.8610 - f1_score: 0.8592
 - loss: 0.4457 - precision: 0.9217 - val_accuracy: 0.6623 - val_f1_score: 0.6618
 - val_loss: 1.2840 - val_precision: 0.7642 - learning_rate: 2.5000e-04

Epoch 17/25

655/657 ————— **0s** 23ms/step - accuracy: 0.8689 - f1_score: 0.8673 -
 loss: 0.4220 - precision: 0.9260

Epoch 17: val_loss did not improve from 1.27402

Epoch 17: ReduceLROnPlateau reducing learning rate to 0.0001250000059371814.

657/657 ————— **18s** 28ms/step - accuracy: 0.8690 - f1_score: 0.8673
 - loss: 0.4220 - precision: 0.9260 - val_accuracy: 0.6644 - val_f1_score: 0.6634
 - val_loss: 1.2827 - val_precision: 0.7662 - learning_rate: 2.5000e-04

Epoch 18/25

656/657 ————— **0s** 24ms/step - accuracy: 0.8771 - f1_score: 0.8753 -
 loss: 0.3936 - precision: 0.9292

Epoch 18: val_loss did not improve from 1.27402

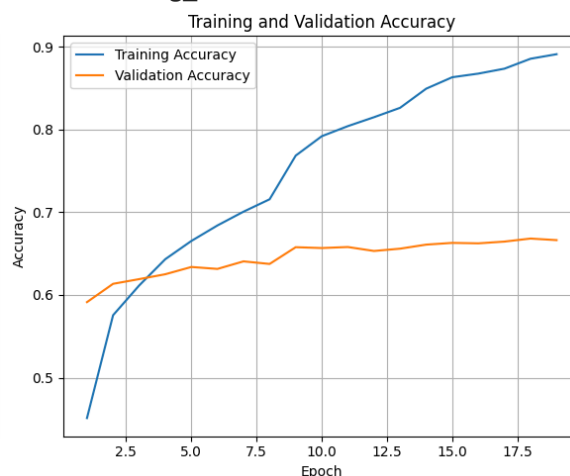
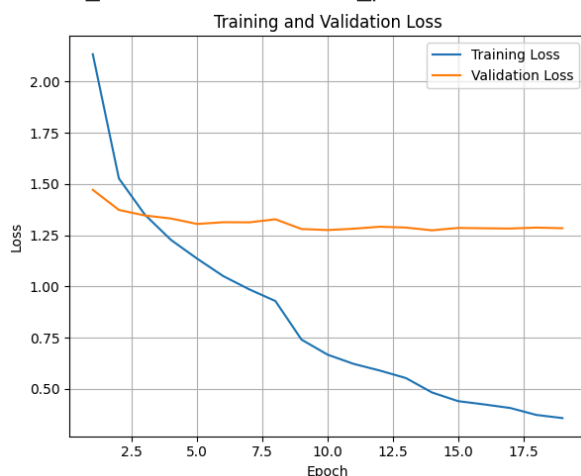
657/657 ————— **19s** 28ms/step - accuracy: 0.8771 - f1_score: 0.8754
 - loss: 0.3935 - precision: 0.9292 - val_accuracy: 0.6681 - val_f1_score: 0.6677
 - val_loss: 1.2873 - val_precision: 0.7620 - learning_rate: 1.2500e-04

Epoch 19/25

656/657 ————— **0s** 23ms/step - accuracy: 0.8859 - f1_score: 0.8839 -
 loss: 0.3713 - precision: 0.9360

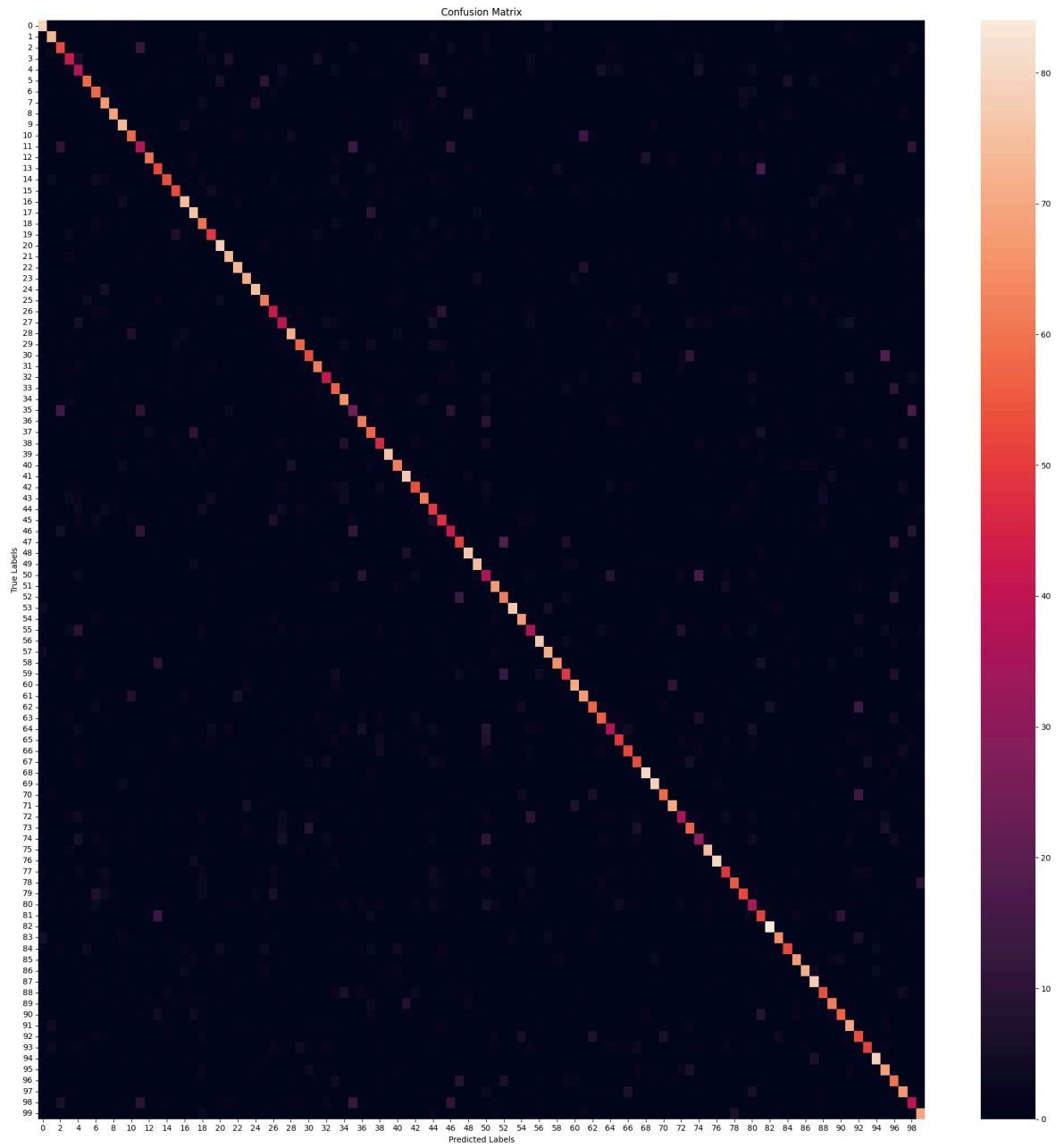
Epoch 19: val_loss did not improve from 1.27402

657/657 ————— **18s** 28ms/step - accuracy: 0.8859 - f1_score: 0.8839
 - loss: 0.3713 - precision: 0.9360 - val_accuracy: 0.6662 - val_f1_score: 0.6652
 - val_loss: 1.2843 - val_precision: 0.7638 - learning_rate: 1.2500e-04



141/141  **11s** 37ms/step - accuracy: 0.6545 - f1_score: 0.6467
- loss: 1.3005 - precision: 0.7638
Test Accuracy: 66.28%
Test Loss: 1.2741423845291138
Test Precision: 77.12%
Test F1 Scores (Per Class): [85.405396 78.07486 55.789463 49.999992 40.211636 6
9.87952 61.458332
69.47368 79.768776 77.24867 66.666664 43.575413 71.856285 57.777775
67.92453 62.72189 78.723404 77.15736 59.113293 58.682625 83.422455
78.49462 79.78141 79.775276 83.798874 68.888885 47.727264 44.692734
80.22598 65.14285 62.72189 69.27374 50.299393 57.731956 68.42105
29.999996 70.05649 59.685856 54.117645 88.888885 67.3913 80.628265
62.06896 70.05649 48.756214 51.64835 46.92737 58.139526 81.72042
82.68156 34.285706 75.28088 64.921455 88.50574 72.04301 42.28571
83.24323 78.02197 79.75459 60.1227 77.34807 69.430046 68.263466
65.88235 45.783127 59.036137 62.650597 59.886997 86.02149 83.59788
70.303024 76.92307 44.444443 62.29508 37.349392 82.41757 87.91208
57.142853 68.71165 63.030296 39.57219 52.083332 90.32257 73.56321
61.53846 76.13636 81.14285 78. 62.427734 69.27374 61.87845
72.63157 51.456303 61.818176 87.29281 68. 55.81395 68.04123
43.478252 75.82417]
Average Test F1 Scores:66.22

282/282  **11s** 28ms/step



	precision	recall	f1-score	support
beaver	0.83	0.88	0.85	90
dolphin	0.75	0.81	0.78	90
otter	0.53	0.59	0.56	90
seal	0.54	0.47	0.50	90
whale	0.38	0.42	0.40	90
aquarium fish	0.76	0.64	0.70	90
flatfish	0.57	0.64	0.61	90
ray	0.66	0.74	0.70	90
shark	0.83	0.77	0.80	90
trout	0.74	0.81	0.77	90
orchids	0.69	0.64	0.67	90
poppies	0.44	0.43	0.44	90
roses	0.78	0.67	0.72	90
sunflowers	0.58	0.58	0.58	90
tulips	0.78	0.60	0.68	90
bottles	0.66	0.59	0.62	90
bowls	0.76	0.82	0.79	90
cans	0.71	0.84	0.77	90
cups	0.54	0.67	0.59	90
plates	0.64	0.54	0.59	90
apples	0.80	0.87	0.83	90
mushrooms	0.76	0.81	0.78	90
oranges	0.78	0.81	0.80	90
pears	0.81	0.79	0.80	90
sweet peppers	0.84	0.83	0.84	90
clock	0.69	0.69	0.69	90
computer keyboard	0.49	0.47	0.48	90
lamp	0.45	0.44	0.45	90
telephone	0.82	0.79	0.80	90
television	0.68	0.63	0.66	90
bed	0.67	0.59	0.63	90
chair	0.70	0.69	0.69	90
couch	0.55	0.47	0.50	90
table	0.54	0.62	0.58	90
wardrobe	0.65	0.72	0.68	90
bee	0.34	0.27	0.30	90
beetle	0.71	0.69	0.70	90
butterfly	0.57	0.63	0.60	90
caterpillar	0.57	0.51	0.54	90
cockroach	0.94	0.84	0.89	90
bear	0.66	0.69	0.67	90
leopard	0.76	0.86	0.81	90
lion	0.64	0.60	0.62	90
tiger	0.71	0.69	0.70	90
wolf	0.44	0.54	0.49	90
bridge	0.50	0.52	0.51	90
castle	0.47	0.47	0.47	90
house	0.61	0.56	0.58	90
road	0.79	0.84	0.82	90
skyscraper	0.83	0.82	0.83	90
cloud	0.30	0.40	0.34	90
forest	0.76	0.74	0.75	90
mountain	0.61	0.69	0.65	90
plain	0.92	0.86	0.89	90
sea	0.70	0.74	0.72	90
camel	0.44	0.41	0.42	90
cattle	0.81	0.86	0.83	90
chimpanzee	0.77	0.79	0.78	90

elephant	0.89	0.72	0.80	90
kangaroo	0.66	0.54	0.60	90
fox	0.77	0.78	0.77	90
porcupine	0.65	0.74	0.69	90
possum	0.74	0.63	0.68	90
raccoon	0.70	0.62	0.66	90
skunk	0.50	0.42	0.46	90
crab	0.64	0.54	0.59	90
lobster	0.68	0.58	0.63	90
snail	0.61	0.59	0.60	90
spider	0.83	0.89	0.86	90
worm	0.80	0.88	0.84	90
baby	0.77	0.64	0.70	90
boy	0.76	0.78	0.77	90
girl	0.50	0.40	0.44	90
man	0.61	0.63	0.62	90
woman	0.41	0.34	0.37	90
crocodile	0.82	0.83	0.82	90
dinosaur	0.87	0.89	0.88	90
lizard	0.62	0.53	0.57	90
snake	0.76	0.61	0.68	90
turtle	0.69	0.58	0.63	90
hamster	0.38	0.41	0.40	90
mouse	0.50	0.57	0.53	90
rabbit	0.88	0.93	0.90	90
shrew	0.76	0.71	0.74	90
squirrel	0.66	0.58	0.62	90
maple	0.78	0.74	0.76	90
oak	0.84	0.79	0.81	90
palm	0.71	0.87	0.78	90
pine	0.65	0.60	0.62	90
willow	0.70	0.69	0.69	90
bicycle	0.62	0.62	0.62	90
bus	0.69	0.77	0.73	90
motorcycle	0.46	0.59	0.51	90
pickup truck	0.68	0.57	0.62	90
train	0.87	0.88	0.87	90
lawn-mower	0.62	0.76	0.68	90
rocket	0.48	0.67	0.56	90
streetcar	0.63	0.73	0.68	90
tank	0.43	0.44	0.43	90
tractor	0.75	0.77	0.76	90
accuracy			0.66	9000
macro avg	0.67	0.66	0.66	9000
weighted avg	0.67	0.66	0.66	9000

Model: "sequential_4"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None , 7 , 7 , 2048)
global_average_pooling2d_4 (GlobalAveragePooling2D)	(None , 2048)
dense_8 (Dense)	(None , 512)
dropout_2 (Dropout)	(None , 512)
dense_9 (Dense)	(None , 100)



Total params: [24,665,188](#) (94.09 MB)

Trainable params: [1,100,388](#) (4.20 MB)

Non-trainable params: [23,564,800](#) (89.89 MB)

Epoch 1/25
1313/1314 ————— 0s 26ms/step - accuracy: 0.8433 - f1_score: 0.8424
- loss: 0.5059 - precision: 0.9082
Epoch 1: val_loss improved from inf to 1.25277, saving model to best_model_fine_tune_v4.keras
1314/1314 ————— 55s 34ms/step - accuracy: 0.8432 - f1_score: 0.8424 - loss: 0.5060 - precision: 0.9082 - val_accuracy: 0.6617 - val_f1_score: 0.6607 - val_loss: 1.2528 - val_precision: 0.7727 - learning_rate: 1.0000e-05

Epoch 2/25
1313/1314 ————— 0s 24ms/step - accuracy: 0.8565 - f1_score: 0.8555
- loss: 0.4757 - precision: 0.9209
Epoch 2: val_loss improved from 1.25277 to 1.24883, saving model to best_model_fine_tune_v4.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.8564 - f1_score: 0.8555 - loss: 0.4758 - precision: 0.9209 - val_accuracy: 0.6637 - val_f1_score: 0.6627 - val_loss: 1.2488 - val_precision: 0.7736 - learning_rate: 1.0000e-05

Epoch 3/25
1312/1314 ————— 0s 24ms/step - accuracy: 0.8572 - f1_score: 0.8563
- loss: 0.4697 - precision: 0.9227
Epoch 3: val_loss improved from 1.24883 to 1.24670, saving model to best_model_fine_tune_v4.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.8572 - f1_score: 0.8563 - loss: 0.4699 - precision: 0.9227 - val_accuracy: 0.6648 - val_f1_score: 0.6640 - val_loss: 1.2467 - val_precision: 0.7735 - learning_rate: 1.0000e-05

Epoch 4/25
1314/1314 ————— 0s 24ms/step - accuracy: 0.8611 - f1_score: 0.8601
- loss: 0.4620 - precision: 0.9231
Epoch 4: val_loss improved from 1.24670 to 1.24532, saving model to best_model_fine_tune_v4.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.8611 - f1_score: 0.8600 - loss: 0.4621 - precision: 0.9231 - val_accuracy: 0.6652 - val_f1_score: 0.6644 - val_loss: 1.2453 - val_precision: 0.7726 - learning_rate: 1.0000e-05

Epoch 5/25
1313/1314 ————— 0s 24ms/step - accuracy: 0.8582 - f1_score: 0.8571
- loss: 0.4643 - precision: 0.9239
Epoch 5: val_loss improved from 1.24532 to 1.24322, saving model to best_model_fine_tune_v4.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.8581 - f1_score: 0.8570 - loss: 0.4644 - precision: 0.9239 - val_accuracy: 0.6654 - val_f1_score: 0.6645 - val_loss: 1.2432 - val_precision: 0.7764 - learning_rate: 1.0000e-05

Epoch 6/25
1314/1314 ————— 0s 24ms/step - accuracy: 0.8614 - f1_score: 0.8604
- loss: 0.4601 - precision: 0.9241
Epoch 6: val_loss did not improve from 1.24322
1314/1314 ————— 34s 26ms/step - accuracy: 0.8614 - f1_score: 0.8604 - loss: 0.4602 - precision: 0.9241 - val_accuracy: 0.6650 - val_f1_score: 0.6641 - val_loss: 1.2436 - val_precision: 0.7738 - learning_rate: 1.0000e-05

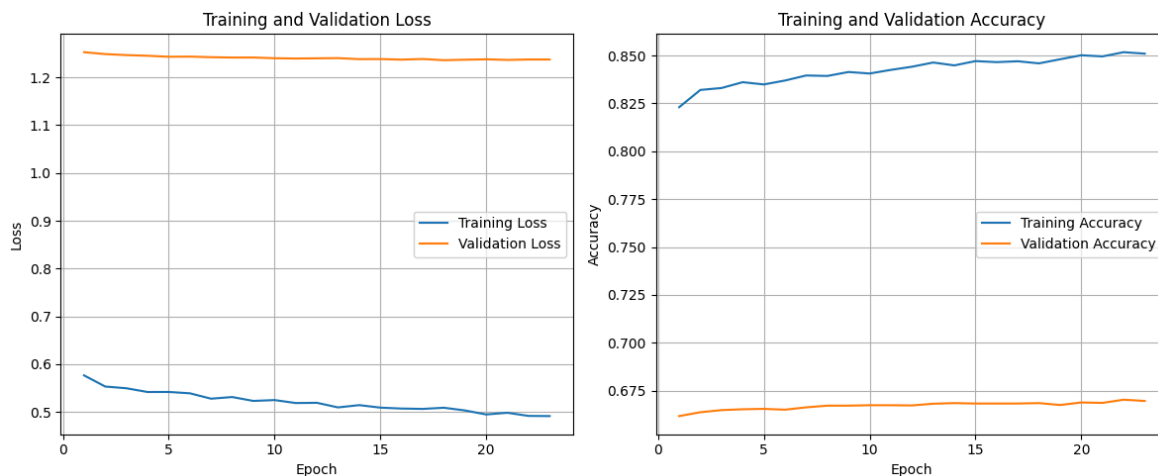
Epoch 7/25
1314/1314 ————— 0s 24ms/step - accuracy: 0.8642 - f1_score: 0.8633
- loss: 0.4516 - precision: 0.9260
Epoch 7: val_loss improved from 1.24322 to 1.24230, saving model to best_model_fine_tune_v4.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.8642 - f1_score: 0.8633 - loss: 0.4517 - precision: 0.9260 - val_accuracy: 0.6662 - val_f1_score: 0.6655 - val_loss: 1.2423 - val_precision: 0.7742 - learning_rate: 1.0000e-05

Epoch 8/25
1313/1314 ————— 0s 24ms/step - accuracy: 0.8632 - f1_score: 0.8623
- loss: 0.4513 - precision: 0.9240
Epoch 8: val_loss improved from 1.24230 to 1.24158, saving model to best_model_fine_tune_v4.keras

1314/1314 ————— **35s** 27ms/step - accuracy: 0.8632 - f1_score: 0.8623 - loss: 0.4514 - precision: 0.9240 - val_accuracy: 0.6671 - val_f1_score: 0.6664 - val_loss: 1.2416 - val_precision: 0.7747 - learning_rate: 1.0000e-05
Epoch 9/25
1312/1314 ————— **0s** 24ms/step - accuracy: 0.8636 - f1_score: 0.8628 - loss: 0.4482 - precision: 0.9275
Epoch 9: val_loss did not improve from 1.24158
1314/1314 ————— **35s** 27ms/step - accuracy: 0.8635 - f1_score: 0.8627 - loss: 0.4484 - precision: 0.9274 - val_accuracy: 0.6671 - val_f1_score: 0.6663 - val_loss: 1.2417 - val_precision: 0.7739 - learning_rate: 1.0000e-05
Epoch 10/25
1314/1314 ————— **0s** 24ms/step - accuracy: 0.8651 - f1_score: 0.8641 - loss: 0.4454 - precision: 0.9274
Epoch 10: val_loss improved from 1.24158 to 1.24013, saving model to best_model_fine_tune_v4.keras
1314/1314 ————— **36s** 27ms/step - accuracy: 0.8651 - f1_score: 0.8641 - loss: 0.4455 - precision: 0.9274 - val_accuracy: 0.6673 - val_f1_score: 0.6665 - val_loss: 1.2401 - val_precision: 0.7747 - learning_rate: 1.0000e-05
Epoch 11/25
1312/1314 ————— **0s** 24ms/step - accuracy: 0.8674 - f1_score: 0.8666 - loss: 0.4369 - precision: 0.9283
Epoch 11: val_loss improved from 1.24013 to 1.23952, saving model to best_model_fine_tune_v4.keras
1314/1314 ————— **35s** 27ms/step - accuracy: 0.8674 - f1_score: 0.8665 - loss: 0.4371 - precision: 0.9282 - val_accuracy: 0.6673 - val_f1_score: 0.6666 - val_loss: 1.2395 - val_precision: 0.7739 - learning_rate: 1.0000e-05
Epoch 12/25
1312/1314 ————— **0s** 24ms/step - accuracy: 0.8692 - f1_score: 0.8682 - loss: 0.4364 - precision: 0.9305
Epoch 12: val_loss did not improve from 1.23952
1314/1314 ————— **34s** 26ms/step - accuracy: 0.8691 - f1_score: 0.8682 - loss: 0.4366 - precision: 0.9305 - val_accuracy: 0.6672 - val_f1_score: 0.6663 - val_loss: 1.2400 - val_precision: 0.7736 - learning_rate: 1.0000e-05
Epoch 13/25
1313/1314 ————— **0s** 24ms/step - accuracy: 0.8720 - f1_score: 0.8711 - loss: 0.4322 - precision: 0.9305
Epoch 13: val_loss did not improve from 1.23952
1314/1314 ————— **34s** 26ms/step - accuracy: 0.8719 - f1_score: 0.8710 - loss: 0.4323 - precision: 0.9305 - val_accuracy: 0.6681 - val_f1_score: 0.6673 - val_loss: 1.2403 - val_precision: 0.7731 - learning_rate: 1.0000e-05
Epoch 14/25
1313/1314 ————— **0s** 24ms/step - accuracy: 0.8696 - f1_score: 0.8688 - loss: 0.4334 - precision: 0.9294
Epoch 14: val_loss improved from 1.23952 to 1.23822, saving model to best_model_fine_tune_v4.keras
1314/1314 ————— **36s** 27ms/step - accuracy: 0.8696 - f1_score: 0.8688 - loss: 0.4335 - precision: 0.9294 - val_accuracy: 0.6684 - val_f1_score: 0.6675 - val_loss: 1.2382 - val_precision: 0.7735 - learning_rate: 1.0000e-05
Epoch 15/25
1313/1314 ————— **0s** 24ms/step - accuracy: 0.8725 - f1_score: 0.8714 - loss: 0.4268 - precision: 0.9315
Epoch 15: val_loss did not improve from 1.23822
1314/1314 ————— **34s** 26ms/step - accuracy: 0.8725 - f1_score: 0.8714 - loss: 0.4269 - precision: 0.9315 - val_accuracy: 0.6682 - val_f1_score: 0.6674 - val_loss: 1.2384 - val_precision: 0.7747 - learning_rate: 1.0000e-05
Epoch 16/25
1312/1314 ————— **0s** 24ms/step - accuracy: 0.8711 - f1_score: 0.8701 - loss: 0.4287 - precision: 0.9293
Epoch 16: val_loss improved from 1.23822 to 1.23723, saving model to best_model_fine_tune_v4.keras

1314/1314 ————— **35s** 27ms/step - accuracy: 0.8710 - f1_score: 0.8700 - loss: 0.4289 - precision: 0.9292 - val_accuracy: 0.6682 - val_f1_score: 0.6676 - val_loss: 1.2372 - val_precision: 0.7737 - learning_rate: 1.0000e-05
Epoch 17/25
1314/1314 ————— **0s** 24ms/step - accuracy: 0.8736 - f1_score: 0.8727 - loss: 0.4235 - precision: 0.9325
Epoch 17: val_loss did not improve from 1.23723
1314/1314 ————— **34s** 26ms/step - accuracy: 0.8736 - f1_score: 0.8727 - loss: 0.4235 - precision: 0.9325 - val_accuracy: 0.6682 - val_f1_score: 0.6676 - val_loss: 1.2386 - val_precision: 0.7729 - learning_rate: 1.0000e-05
Epoch 18/25
1312/1314 ————— **0s** 24ms/step - accuracy: 0.8704 - f1_score: 0.8695 - loss: 0.4277 - precision: 0.9299
Epoch 18: val_loss improved from 1.23723 to 1.23600, saving model to best_model_fine_tune_v4.keras
1314/1314 ————— **35s** 27ms/step - accuracy: 0.8703 - f1_score: 0.8694 - loss: 0.4279 - precision: 0.9298 - val_accuracy: 0.6684 - val_f1_score: 0.6677 - val_loss: 1.2360 - val_precision: 0.7745 - learning_rate: 1.0000e-05
Epoch 19/25
1314/1314 ————— **0s** 24ms/step - accuracy: 0.8719 - f1_score: 0.8709 - loss: 0.4252 - precision: 0.9302
Epoch 19: val_loss did not improve from 1.23600
1314/1314 ————— **34s** 26ms/step - accuracy: 0.8719 - f1_score: 0.8709 - loss: 0.4253 - precision: 0.9302 - val_accuracy: 0.6674 - val_f1_score: 0.6666 - val_loss: 1.2371 - val_precision: 0.7738 - learning_rate: 1.0000e-05
Epoch 20/25
1314/1314 ————— **0s** 24ms/step - accuracy: 0.8762 - f1_score: 0.8755 - loss: 0.4145 - precision: 0.9338
Epoch 20: val_loss did not improve from 1.23600
1314/1314 ————— **34s** 26ms/step - accuracy: 0.8762 - f1_score: 0.8754 - loss: 0.4146 - precision: 0.9338 - val_accuracy: 0.6688 - val_f1_score: 0.6680 - val_loss: 1.2379 - val_precision: 0.7738 - learning_rate: 1.0000e-05
Epoch 21/25
1313/1314 ————— **0s** 24ms/step - accuracy: 0.8748 - f1_score: 0.8738 - loss: 0.4159 - precision: 0.9329
Epoch 21: val_loss did not improve from 1.23600

Epoch 21: ReduceLROnPlateau reducing learning rate to 4.999999873689376e-06.
1314/1314 ————— **34s** 26ms/step - accuracy: 0.8748 - f1_score: 0.8738 - loss: 0.4161 - precision: 0.9329 - val_accuracy: 0.6686 - val_f1_score: 0.6677 - val_loss: 1.2366 - val_precision: 0.7739 - learning_rate: 1.0000e-05
Epoch 22/25
1313/1314 ————— **0s** 24ms/step - accuracy: 0.8763 - f1_score: 0.8752 - loss: 0.4156 - precision: 0.9331
Epoch 22: val_loss did not improve from 1.23600
1314/1314 ————— **34s** 26ms/step - accuracy: 0.8763 - f1_score: 0.8752 - loss: 0.4157 - precision: 0.9330 - val_accuracy: 0.6702 - val_f1_score: 0.6695 - val_loss: 1.2375 - val_precision: 0.7740 - learning_rate: 5.0000e-06
Epoch 23/25
1312/1314 ————— **0s** 24ms/step - accuracy: 0.8754 - f1_score: 0.8743 - loss: 0.4153 - precision: 0.9323
Epoch 23: val_loss did not improve from 1.23600
1314/1314 ————— **34s** 26ms/step - accuracy: 0.8753 - f1_score: 0.8743 - loss: 0.4155 - precision: 0.9323 - val_accuracy: 0.6696 - val_f1_score: 0.6687 - val_loss: 1.2375 - val_precision: 0.7731 - learning_rate: 5.0000e-06



141/141 ————— **12s** 37ms/step - accuracy: 0.6623 - f1_score: 0.6541

- loss: 1.2552 - precision: 0.7754

Test Accuracy: 67.08%

Test Loss: 1.2318534851074219

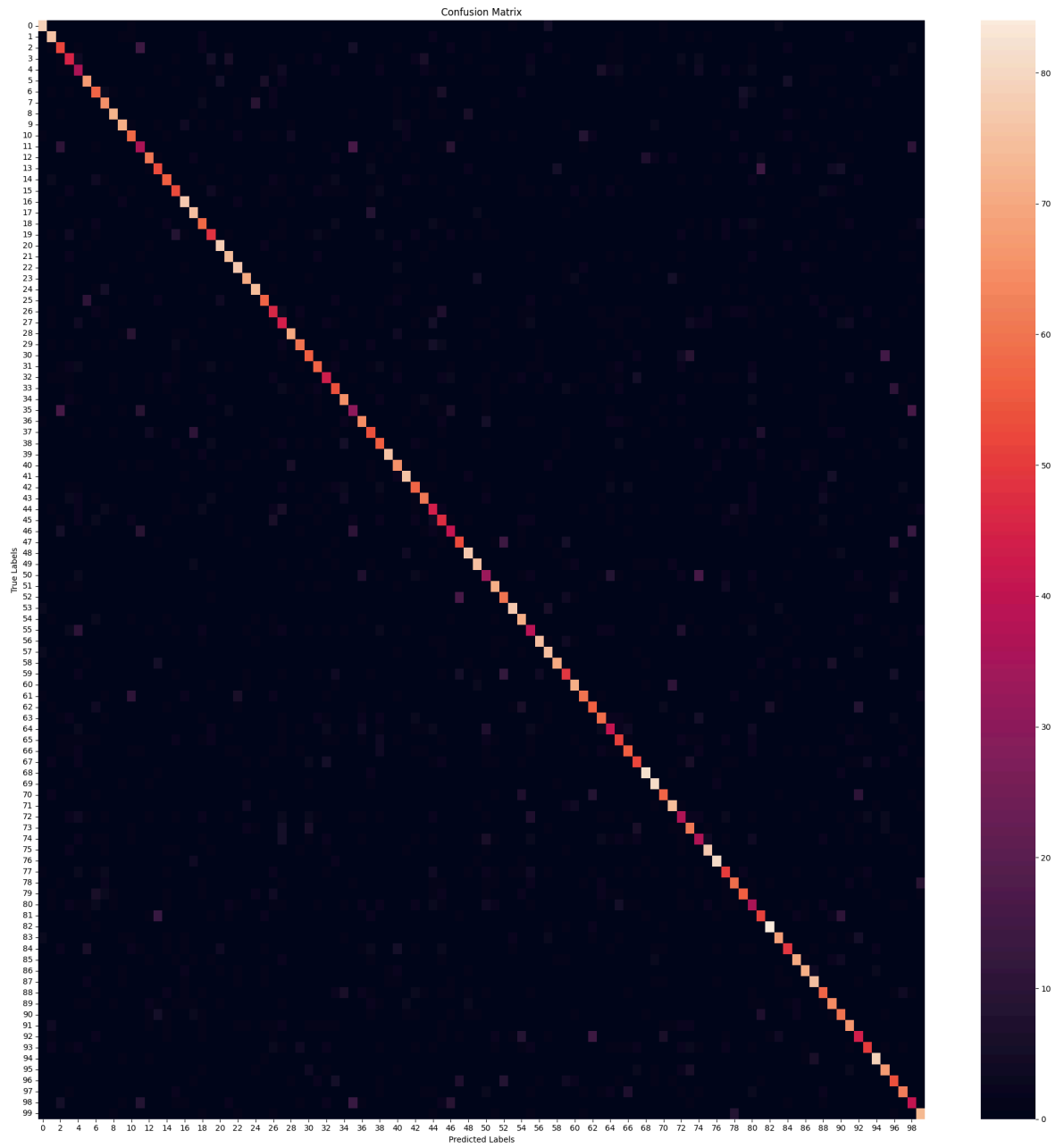
Test Precision: 78.07%

Test F1 Scores (Per Class): [86.81318 77.948715 54.639168 50.847454 38.91891 7
2.04301 60.31745

71.428566	82.75862	81.60919	64.44444	42.352932	70.93022	60.571426
67.46988	60.227264	82.35293	78.12499	63.387978	56.140347	83.87096
78.30688	81.481476	80.459755	82.22221	67.85713	51.685387	45.595848
79.54544	70.58823	65.49707	66.27907	51.497	59.66851	71.428566
34.883713	71.91011	60.33519	58.333324	89.41175	67.01031	82.16216
62.63736	70.520226	47.05882	50.267376	48.235287	60.919537	81.91489
81.52173	37.931026	77.173904	64.51612	87.49999	69.95074	44.571423
85.22727	77.31957	82.352936	56.97674	79.329605	71.00591	62.569828
65.55555	48.235287	56.66666	64.36781	61.176468	87.23404	84.81675
68.263466	79.14438	43.02325	64.210526	43.02325	83.69564	88.52458
57.142853	69.005844	64.739876	37.69633	55.737698	90.32257	76.83616
59.39393	77.34807	80.681816	78.947365	64.80447	67.72487	63.82978
70.588234	50.28571	61.349686	87.77777	72.82608	56.25	68.888885
44.808735	78.49462	]				

Average Test F1 Scores:66.96

282/282 ————— **12s** 28ms/step



	precision	recall	f1-score	support
beaver	0.86	0.88	0.87	90
dolphin	0.73	0.84	0.78	90
otter	0.50	0.58	0.54	90
seal	0.52	0.50	0.51	90
whale	0.38	0.40	0.39	90
aquarium fish	0.70	0.74	0.72	90
flatfish	0.58	0.63	0.60	90
ray	0.71	0.72	0.71	90
shark	0.86	0.80	0.83	90
trout	0.85	0.79	0.82	90
orchids	0.64	0.64	0.64	90
poppies	0.44	0.40	0.42	90
roses	0.74	0.68	0.71	90
sunflowers	0.62	0.59	0.61	90
tulips	0.74	0.62	0.67	90
bottles	0.62	0.59	0.60	90
bowls	0.79	0.86	0.82	90
cans	0.74	0.83	0.78	90
cups	0.62	0.64	0.63	90
plates	0.59	0.53	0.56	90
apples	0.81	0.87	0.84	90
mushrooms	0.75	0.82	0.78	90
oranges	0.78	0.86	0.81	90
pears	0.83	0.78	0.80	90
sweet peppers	0.82	0.82	0.82	90
clock	0.73	0.63	0.68	90
computer keyboard	0.52	0.51	0.51	90
lamp	0.43	0.49	0.46	90
telephone	0.81	0.78	0.80	90
television	0.75	0.67	0.71	90
bed	0.69	0.62	0.65	90
chair	0.70	0.63	0.66	90
couch	0.56	0.48	0.51	90
table	0.59	0.60	0.59	90
wardrobe	0.71	0.72	0.71	90
bee	0.37	0.33	0.35	90
beetle	0.73	0.71	0.72	90
butterfly	0.61	0.60	0.60	90
caterpillar	0.55	0.62	0.58	90
cockroach	0.95	0.84	0.89	90
bear	0.62	0.72	0.67	90
leopard	0.80	0.84	0.82	90
lion	0.62	0.63	0.63	90
tiger	0.73	0.68	0.71	90
wolf	0.45	0.49	0.47	90
bridge	0.48	0.52	0.50	90
castle	0.51	0.46	0.48	90
house	0.63	0.59	0.61	90
road	0.79	0.86	0.82	90
skyscraper	0.80	0.83	0.82	90
cloud	0.39	0.37	0.38	90
forest	0.76	0.79	0.77	90
mountain	0.62	0.67	0.65	90
plain	0.90	0.86	0.88	90
sea	0.63	0.79	0.70	90
camel	0.45	0.43	0.44	90
cattle	0.87	0.83	0.85	90
chimpanzee	0.72	0.83	0.77	90

elephant	0.88	0.78	0.82	90
kangaroo	0.60	0.54	0.57	90
fox	0.80	0.79	0.79	90
porcupine	0.76	0.67	0.71	90
possum	0.63	0.62	0.63	90
raccoon	0.66	0.66	0.66	90
skunk	0.51	0.46	0.48	90
crab	0.57	0.57	0.57	90
lobster	0.67	0.62	0.64	90
snail	0.65	0.58	0.61	90
spider	0.84	0.91	0.87	90
worm	0.80	0.90	0.85	90
baby	0.74	0.63	0.68	90
boy	0.76	0.82	0.79	90
girl	0.45	0.41	0.43	90
man	0.61	0.68	0.64	90
woman	0.45	0.41	0.43	90
crocodile	0.82	0.86	0.84	90
dinosaur	0.87	0.90	0.89	90
lizard	0.59	0.56	0.57	90
snake	0.73	0.66	0.69	90
turtle	0.67	0.62	0.65	90
hamster	0.36	0.40	0.38	90
mouse	0.55	0.57	0.56	90
rabbit	0.88	0.93	0.90	90
shrew	0.78	0.76	0.77	90
squirrel	0.65	0.54	0.59	90
maple	0.77	0.78	0.77	90
oak	0.83	0.79	0.81	90
palm	0.75	0.83	0.79	90
pine	0.65	0.63	0.64	90
willow	0.65	0.71	0.68	90
bicycle	0.61	0.67	0.64	90
bus	0.68	0.73	0.71	90
motorcycle	0.52	0.49	0.50	90
pickup truck	0.68	0.56	0.61	90
train	0.88	0.88	0.88	90
lawn-mower	0.71	0.74	0.73	90
rocket	0.53	0.60	0.56	90
streetcar	0.69	0.69	0.69	90
tank	0.44	0.46	0.45	90
tractor	0.76	0.81	0.78	90
accuracy			0.67	9000
macro avg	0.67	0.67	0.67	9000
weighted avg	0.67	0.67	0.67	9000

X_temp.shape: (8000, 32, 32, 3)

X_temp_val.shape: (4000, 32, 32, 3)

y_temp_val.shape: (4000, 1)

X_temp_test.shape: (4000, 32, 32, 3)

y_temp_test.shape: (4000, 1)

X_train.shape: (42000, 32, 32, 3)

y_train.shape: (42000, 1)

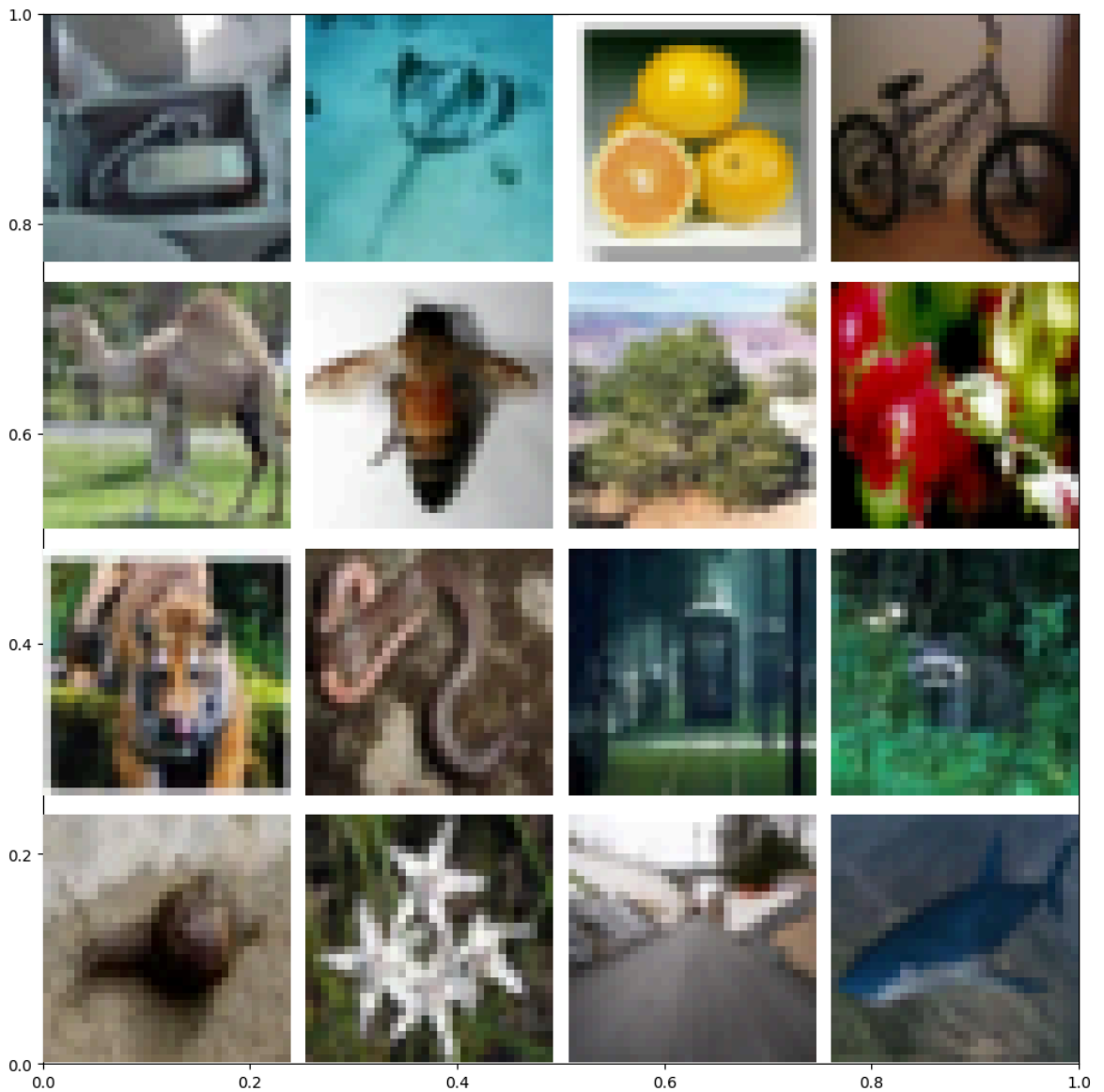
X_val.shape: (9000, 32, 32, 3)

y_val.shape: (9000, 1)

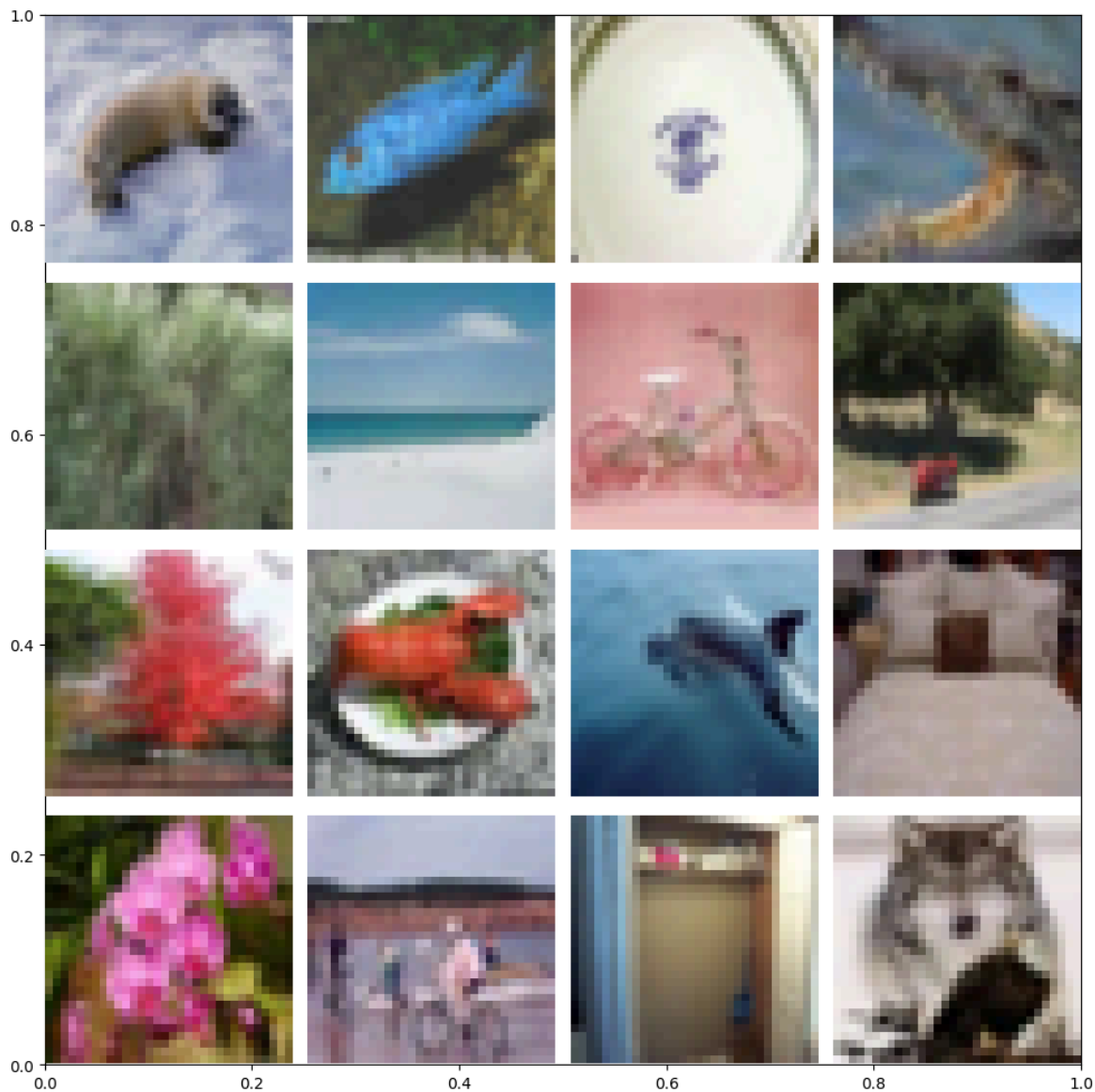
X_test.shape: (9000, 32, 32, 3)

y_test.shape: (9000, 1)

```
labels[10340].shape: (1,)  
imgs[10340].shape: (32, 32, 3)
```

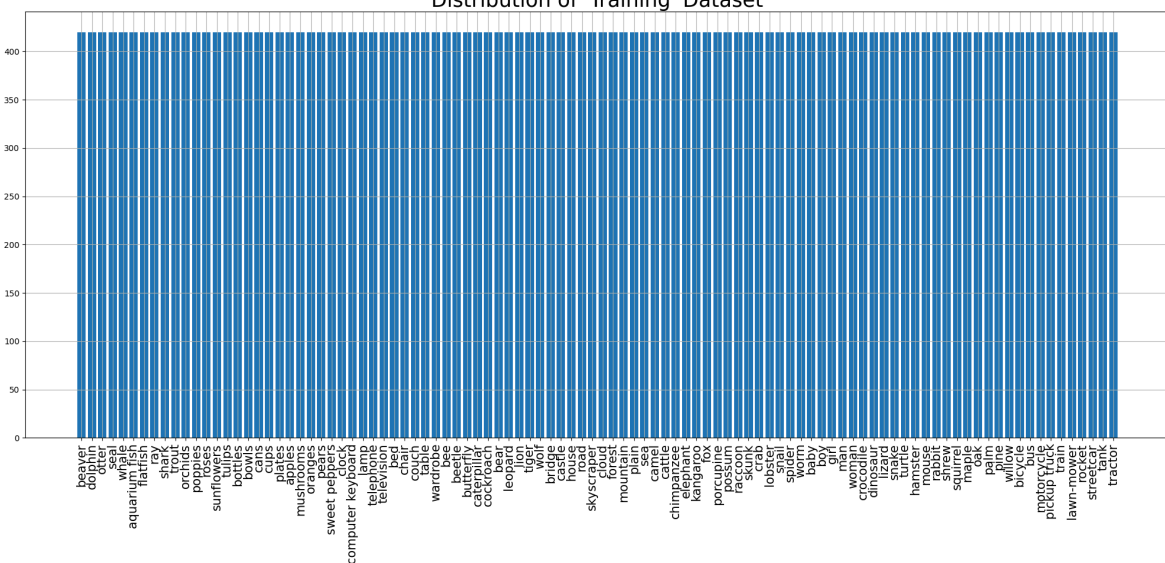


```
labels[34835].shape: (1,)  
imgs[34835].shape: (32, 32, 3)
```

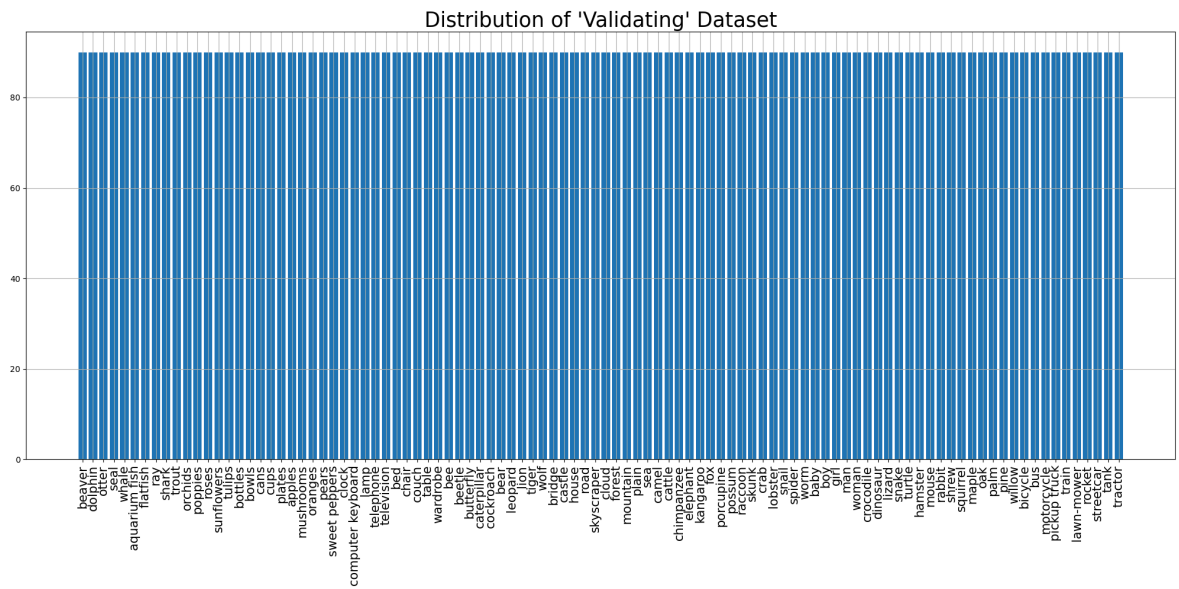


```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```

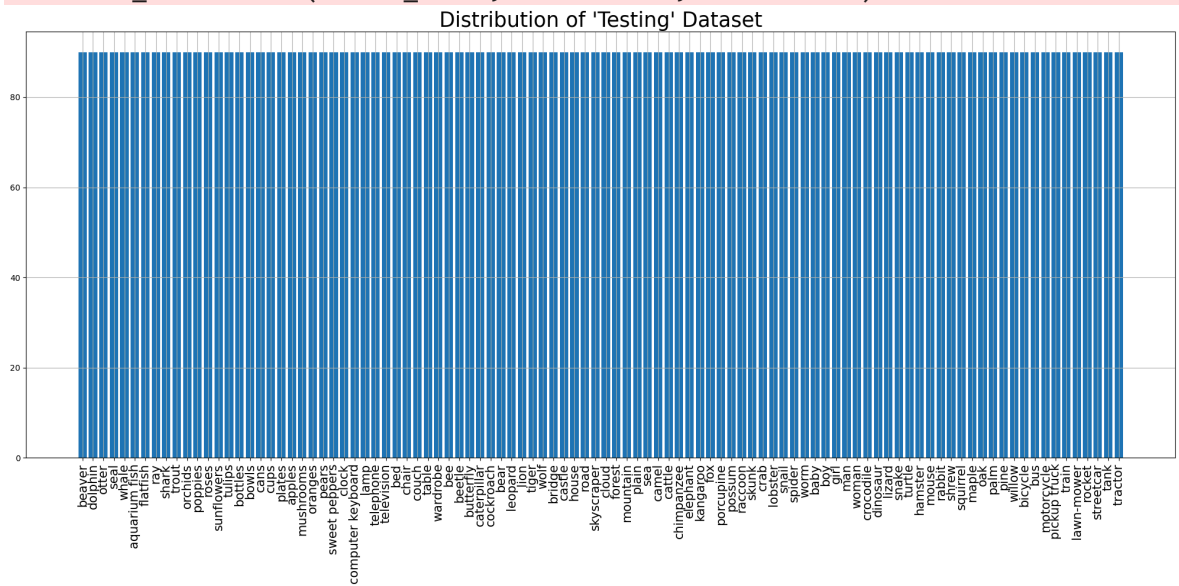
Distribution of 'Training' Dataset



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



Training dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
```

Validation dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
```

Testing dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
<class 'tuple'>
2
<class 'tensorflow.python.framework.ops.EagerTensor'> <class 'tensorflow.python.framework.ops.EagerTensor'>
(64, 224, 224, 3)
(64, 100)
Model input shape: (None, 224, 224, 3)
Model output shape: (None, 100)
2
<class 'numpy.ndarray'> <class 'numpy.ndarray'>
(64, 224, 224, 3) (64, 100)
```

Model: "sequential_5"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None , 7 , 7 , 2048)
global_average_pooling2d_5 (GlobalAveragePooling2D)	(None , 2048)
dense_10 (Dense)	(None , 512)
dropout_3 (Dropout)	(None , 512)
dense_11 (Dense)	(None , 100)

Total params: [24,665,188](#) (94.09 MB)

Trainable params: [1,100,388](#) (4.20 MB)

Non-trainable params: [23,564,800](#) (89.89 MB)

Epoch 1/25
657/657 ————— 0s 29ms/step - accuracy: 0.3549 - f1_score: 0.3505 - loss: 2.6885 - precision: 0.6847
Epoch 1: val_loss improved from inf to 1.47821, saving model to best_model_v4.keras
657/657 ————— 40s 45ms/step - accuracy: 0.3551 - f1_score: 0.3507 - loss: 2.6876 - precision: 0.6848 - val_accuracy: 0.5862 - val_f1_score: 0.5806 - val_loss: 1.4782 - val_precision: 0.7955 - learning_rate: 0.0010
Epoch 2/25
657/657 ————— 0s 24ms/step - accuracy: 0.5645 - f1_score: 0.5611 - loss: 1.5718 - precision: 0.7656
Epoch 2: val_loss improved from 1.47821 to 1.36866, saving model to best_model_v4.keras
657/657 ————— 20s 30ms/step - accuracy: 0.5645 - f1_score: 0.5611 - loss: 1.5717 - precision: 0.7656 - val_accuracy: 0.6181 - val_f1_score: 0.6139 - val_loss: 1.3687 - val_precision: 0.8056 - learning_rate: 0.0010
Epoch 3/25
655/657 ————— 0s 24ms/step - accuracy: 0.6068 - f1_score: 0.6032 - loss: 1.3731 - precision: 0.7843
Epoch 3: val_loss improved from 1.36866 to 1.34459, saving model to best_model_v4.keras
657/657 ————— 20s 30ms/step - accuracy: 0.6069 - f1_score: 0.6033 - loss: 1.3729 - precision: 0.7843 - val_accuracy: 0.6183 - val_f1_score: 0.6149 - val_loss: 1.3446 - val_precision: 0.7947 - learning_rate: 0.0010
Epoch 4/25
656/657 ————— 0s 24ms/step - accuracy: 0.6348 - f1_score: 0.6321 - loss: 1.2593 - precision: 0.7978
Epoch 4: val_loss improved from 1.34459 to 1.31512, saving model to best_model_v4.keras
657/657 ————— 20s 30ms/step - accuracy: 0.6348 - f1_score: 0.6322 - loss: 1.2592 - precision: 0.7978 - val_accuracy: 0.6289 - val_f1_score: 0.6260 - val_loss: 1.3151 - val_precision: 0.7881 - learning_rate: 0.0010
Epoch 5/25
655/657 ————— 0s 24ms/step - accuracy: 0.6574 - f1_score: 0.6546 - loss: 1.1526 - precision: 0.8071
Epoch 5: val_loss improved from 1.31512 to 1.29932, saving model to best_model_v4.keras
657/657 ————— 20s 30ms/step - accuracy: 0.6574 - f1_score: 0.6546 - loss: 1.1525 - precision: 0.8071 - val_accuracy: 0.6316 - val_f1_score: 0.6304 - val_loss: 1.2993 - val_precision: 0.7937 - learning_rate: 0.0010
Epoch 6/25
657/657 ————— 0s 24ms/step - accuracy: 0.6782 - f1_score: 0.6758 - loss: 1.0747 - precision: 0.8175
Epoch 6: val_loss did not improve from 1.29932
657/657 ————— 19s 28ms/step - accuracy: 0.6782 - f1_score: 0.6758 - loss: 1.0747 - precision: 0.8175 - val_accuracy: 0.6359 - val_f1_score: 0.6358 - val_loss: 1.3078 - val_precision: 0.7836 - learning_rate: 0.0010
Epoch 7/25
657/657 ————— 0s 24ms/step - accuracy: 0.6962 - f1_score: 0.6938 - loss: 1.0059 - precision: 0.8200
Epoch 7: val_loss did not improve from 1.29932
657/657 ————— 19s 28ms/step - accuracy: 0.6962 - f1_score: 0.6938 - loss: 1.0059 - precision: 0.8200 - val_accuracy: 0.6348 - val_f1_score: 0.6334 - val_loss: 1.3178 - val_precision: 0.7764 - learning_rate: 0.0010
Epoch 8/25
655/657 ————— 0s 23ms/step - accuracy: 0.7067 - f1_score: 0.7046 - loss: 0.9552 - precision: 0.8260
Epoch 8: val_loss did not improve from 1.29932

Epoch 8: ReduceLROnPlateau reducing learning rate to 0.0005000000237487257.

657/657 ————— **19s** 28ms/step - accuracy: 0.7067 - f1_score: 0.7046
- loss: 0.9550 - precision: 0.8260 - val_accuracy: 0.6404 - val_f1_score: 0.6373
- val_loss: 1.3385 - val_precision: 0.7701 - learning_rate: 0.0010
Epoch 9/25

657/657 ————— **0s** 24ms/step - accuracy: 0.7548 - f1_score: 0.7529 -
loss: 0.8000 - precision: 0.8589
Epoch 9: val_loss improved from 1.29932 to 1.28143, saving model to best_model_v
4.keras

657/657 ————— **20s** 30ms/step - accuracy: 0.7549 - f1_score: 0.7529
- loss: 0.8000 - precision: 0.8589 - val_accuracy: 0.6509 - val_f1_score: 0.6488
- val_loss: 1.2814 - val_precision: 0.7750 - learning_rate: 5.0000e-04
Epoch 10/25

657/657 ————— **0s** 23ms/step - accuracy: 0.7827 - f1_score: 0.7806 -
loss: 0.6912 - precision: 0.8777
Epoch 10: val_loss did not improve from 1.28143

657/657 ————— **19s** 28ms/step - accuracy: 0.7827 - f1_score: 0.7806
- loss: 0.6912 - precision: 0.8777 - val_accuracy: 0.6546 - val_f1_score: 0.6539
- val_loss: 1.2832 - val_precision: 0.7682 - learning_rate: 5.0000e-04
Epoch 11/25

656/657 ————— **0s** 24ms/step - accuracy: 0.7982 - f1_score: 0.7963 -
loss: 0.6413 - precision: 0.8862
Epoch 11: val_loss improved from 1.28143 to 1.27693, saving model to best_model_v
4.keras

657/657 ————— **20s** 30ms/step - accuracy: 0.7982 - f1_score: 0.7963
- loss: 0.6413 - precision: 0.8862 - val_accuracy: 0.6546 - val_f1_score: 0.6538
- val_loss: 1.2769 - val_precision: 0.7726 - learning_rate: 5.0000e-04
Epoch 12/25

657/657 ————— **0s** 24ms/step - accuracy: 0.8108 - f1_score: 0.8091 -
loss: 0.6009 - precision: 0.8910
Epoch 12: val_loss improved from 1.27693 to 1.27416, saving model to best_model_v
4.keras

657/657 ————— **20s** 30ms/step - accuracy: 0.8108 - f1_score: 0.8091
- loss: 0.6009 - precision: 0.8910 - val_accuracy: 0.6559 - val_f1_score: 0.6542
- val_loss: 1.2742 - val_precision: 0.7700 - learning_rate: 5.0000e-04
Epoch 13/25

656/657 ————— **0s** 24ms/step - accuracy: 0.8225 - f1_score: 0.8206 -
loss: 0.5658 - precision: 0.8995
Epoch 13: val_loss did not improve from 1.27416

657/657 ————— **19s** 29ms/step - accuracy: 0.8225 - f1_score: 0.8206
- loss: 0.5658 - precision: 0.8995 - val_accuracy: 0.6582 - val_f1_score: 0.6563
- val_loss: 1.2933 - val_precision: 0.7648 - learning_rate: 5.0000e-04
Epoch 14/25

656/657 ————— **0s** 24ms/step - accuracy: 0.8288 - f1_score: 0.8271 -
loss: 0.5379 - precision: 0.9008
Epoch 14: val_loss did not improve from 1.27416

657/657 ————— **19s** 29ms/step - accuracy: 0.8289 - f1_score: 0.8271
- loss: 0.5378 - precision: 0.9008 - val_accuracy: 0.6544 - val_f1_score: 0.6537
- val_loss: 1.2957 - val_precision: 0.7656 - learning_rate: 5.0000e-04
Epoch 15/25

655/657 ————— **0s** 24ms/step - accuracy: 0.8382 - f1_score: 0.8363 -
loss: 0.5127 - precision: 0.9073
Epoch 15: val_loss did not improve from 1.27416

Epoch 15: ReduceLROnPlateau reducing learning rate to 0.0002500000118743628.

657/657 ————— **19s** 29ms/step - accuracy: 0.8382 - f1_score: 0.8363
- loss: 0.5126 - precision: 0.9073 - val_accuracy: 0.6523 - val_f1_score: 0.6507
- val_loss: 1.3055 - val_precision: 0.7596 - learning_rate: 5.0000e-04
Epoch 16/25

655/657 ————— **0s** 24ms/step - accuracy: 0.8598 - f1_score: 0.8580 -
loss: 0.4527 - precision: 0.9184

Epoch 16: val_loss did not improve from 1.27416

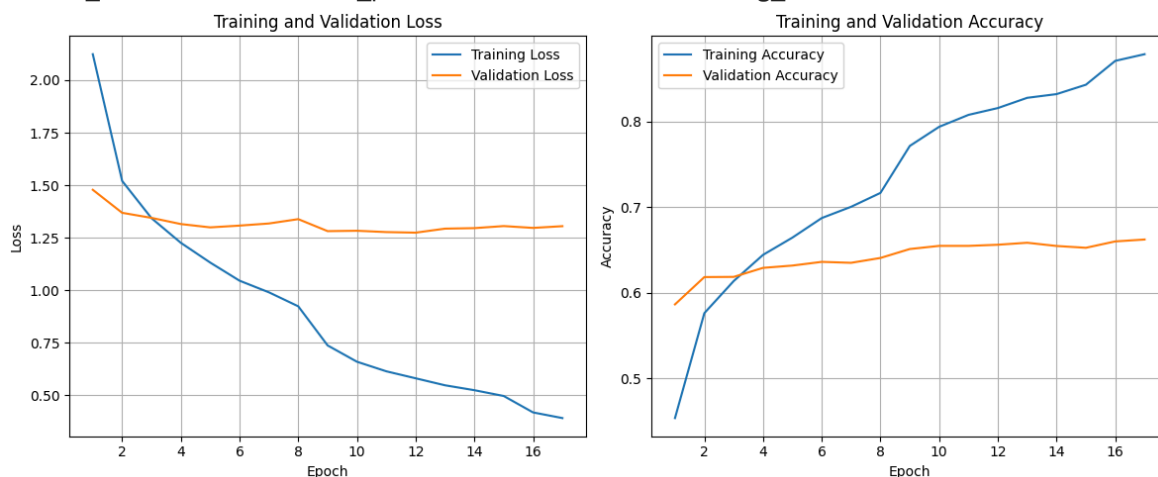
657/657 ————— **19s** 28ms/step - accuracy: 0.8599 - f1_score: 0.8580
 - loss: 0.4526 - precision: 0.9185 - val_accuracy: 0.6598 - val_f1_score: 0.6589
 - val_loss: 1.2968 - val_precision: 0.7574 - learning_rate: 2.5000e-04

Epoch 17/25

656/657 ————— **0s** 24ms/step - accuracy: 0.8717 - f1_score: 0.8698 -
 loss: 0.4088 - precision: 0.9236

Epoch 17: val_loss did not improve from 1.27416

657/657 ————— **19s** 28ms/step - accuracy: 0.8717 - f1_score: 0.8698
 - loss: 0.4087 - precision: 0.9236 - val_accuracy: 0.6620 - val_f1_score: 0.6613
 - val_loss: 1.3050 - val_precision: 0.7583 - learning_rate: 2.5000e-04



141/141 ————— **12s** 37ms/step - accuracy: 0.6479 - f1_score: 0.6398
 - loss: 1.2926 - precision: 0.7621

Test Accuracy: 65.99%

Test Loss: 1.2699257135391235

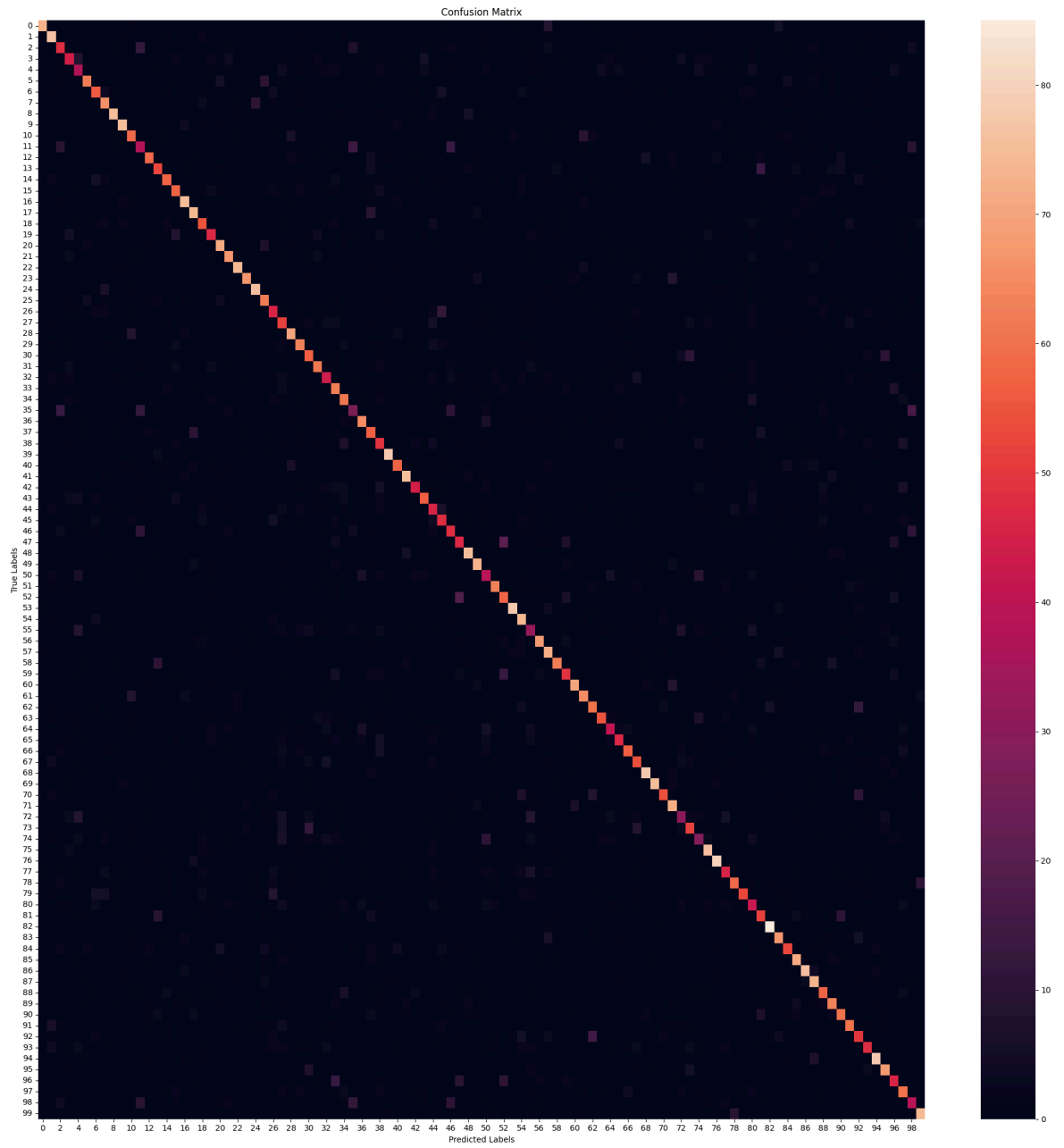
Test Precision: 77.05%

Test F1 Scores (Per Class): [87.42514 75.49018 53.038673 50.86705 40.659336 7
 2.94117 62.569828

68.39378	81.96721	83.33333	65.55555	42.937847	71.95121	60.227264
67.83625	64.44444	79.787224	77.08333	62.499992	56.969696	78.02197
74.444435	82.68156	79.761894	83.51647	65.60846	49.197857	49.0566
76.50272	69.23077	61.290314	68.53932	49.162006	57.142853	62.5641
32.298134	71.428566	59.067356	50.51546	90.17341	71.16564	83.798874
58.441555	69.090904	47.66839	51.336895	50.526314	53.40909	82.872925
79.347824	42.391296	72.41379	58.883244	88.63635	70.14217	35.555546
78.61271	70.243904	76.543205	57.309933	80.92485	72.222206	64.51612
67.07316	51.572315	58.385086	70.80744	58.064507	86.18784	86.20689
68.35443	74.6114	36.363632	59.428566	35.22012	79.58114	85.56149
55.75757	69.41176	67.08861	42.857136	56.830597	91.891884	72.82608
62.72189	75.53191	82.16216	73.36682	64.088394	68.1081	64.86485
68.965515	53.475933	59.999996	86.18784	72.727264	52.272724	61.855663
43.575413	78.723404					

Average Test F1 Scores:65.95

282/282 ————— **11s** 28ms/step



	precision	recall	f1-score	support
beaver	0.95	0.81	0.87	90
dolphin	0.68	0.86	0.75	90
otter	0.53	0.53	0.53	90
seal	0.54	0.50	0.52	90
whale	0.40	0.41	0.41	90
aquarium fish	0.78	0.69	0.73	90
flatfish	0.63	0.62	0.63	90
ray	0.64	0.73	0.68	90
shark	0.82	0.83	0.82	90
trout	0.83	0.83	0.83	90
orchids	0.66	0.66	0.66	90
poppies	0.43	0.42	0.43	90
roses	0.80	0.66	0.72	90
sunflowers	0.61	0.59	0.60	90
tulips	0.72	0.64	0.68	90
bottles	0.64	0.64	0.64	90
bowls	0.77	0.83	0.80	90
cans	0.73	0.82	0.77	90
cups	0.63	0.61	0.62	90
plates	0.63	0.52	0.57	90
apples	0.77	0.79	0.78	90
mushrooms	0.74	0.74	0.74	90
oranges	0.83	0.82	0.83	90
pears	0.86	0.74	0.80	90
sweet peppers	0.83	0.84	0.84	90
clock	0.63	0.69	0.66	90
computer keyboard	0.48	0.51	0.49	90
lamp	0.43	0.58	0.49	90
telephone	0.75	0.78	0.77	90
television	0.68	0.70	0.69	90
bed	0.59	0.63	0.61	90
chair	0.69	0.68	0.69	90
couch	0.49	0.49	0.49	90
table	0.49	0.69	0.57	90
wardrobe	0.58	0.68	0.63	90
bee	0.37	0.29	0.32	90
beetle	0.71	0.72	0.71	90
butterfly	0.55	0.63	0.59	90
caterpillar	0.47	0.54	0.51	90
cockroach	0.94	0.87	0.90	90
bear	0.79	0.64	0.71	90
leopard	0.84	0.83	0.84	90
lion	0.70	0.50	0.58	90
tiger	0.76	0.63	0.69	90
wolf	0.45	0.51	0.48	90
bridge	0.49	0.53	0.51	90
castle	0.48	0.53	0.51	90
house	0.55	0.52	0.53	90
road	0.82	0.83	0.83	90
skyscraper	0.78	0.81	0.79	90
cloud	0.41	0.43	0.42	90
forest	0.75	0.70	0.72	90
mountain	0.55	0.64	0.59	90
plain	0.91	0.87	0.89	90
sea	0.61	0.82	0.70	90
camel	0.36	0.36	0.36	90
cattle	0.82	0.76	0.79	90
chimpanzee	0.63	0.80	0.70	90

elephant	0.86	0.69	0.77	90
kangaroo	0.60	0.54	0.57	90
fox	0.84	0.78	0.81	90
porcupine	0.72	0.72	0.72	90
possum	0.62	0.67	0.65	90
raccoon	0.74	0.61	0.67	90
skunk	0.59	0.46	0.52	90
crab	0.66	0.52	0.58	90
lobster	0.80	0.63	0.71	90
snail	0.56	0.60	0.58	90
spider	0.86	0.87	0.86	90
worm	0.89	0.83	0.86	90
baby	0.79	0.60	0.68	90
boy	0.70	0.80	0.75	90
girl	0.40	0.33	0.36	90
man	0.61	0.58	0.59	90
woman	0.41	0.31	0.35	90
crocodile	0.75	0.84	0.80	90
dinosaur	0.82	0.89	0.86	90
lizard	0.61	0.51	0.56	90
snake	0.74	0.66	0.69	90
turtle	0.78	0.59	0.67	90
hamster	0.40	0.47	0.43	90
mouse	0.56	0.58	0.57	90
rabbit	0.89	0.94	0.91	90
shrew	0.71	0.74	0.73	90
squirrel	0.67	0.59	0.63	90
maple	0.72	0.79	0.76	90
oak	0.80	0.84	0.82	90
palm	0.67	0.81	0.73	90
pine	0.64	0.64	0.64	90
willow	0.66	0.70	0.68	90
bicycle	0.63	0.67	0.65	90
bus	0.71	0.67	0.69	90
motorcycle	0.52	0.56	0.53	90
pickup truck	0.69	0.53	0.60	90
train	0.86	0.87	0.86	90
lawn-mower	0.70	0.76	0.73	90
rocket	0.53	0.51	0.52	90
streetcar	0.58	0.67	0.62	90
tank	0.44	0.43	0.44	90
tractor	0.76	0.82	0.79	90
accuracy			0.66	9000
macro avg	0.67	0.66	0.66	9000
weighted avg	0.67	0.66	0.66	9000

Model: "sequential_5"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None , 7 , 7 , 2048)
global_average_pooling2d_5 (GlobalAveragePooling2D)	(None , 2048)
dense_10 (Dense)	(None , 512)
dropout_3 (Dropout)	(None , 512)
dense_11 (Dense)	(None , 100)



Total params: [24,665,188](#) (94.09 MB)

Trainable params: [1,100,388](#) (4.20 MB)

Non-trainable params: [23,564,800](#) (89.89 MB)

Epoch 1/25
1314/1314 ————— 0s 26ms/step - accuracy: 0.8151 - f1_score: 0.8140
- loss: 0.5919 - precision: 0.8934
Epoch 1: val_loss improved from inf to 1.24255, saving model to best_model_fine_tune_v4.keras
1314/1314 ————— 55s 34ms/step - accuracy: 0.8151 - f1_score: 0.8140
0 - loss: 0.5919 - precision: 0.8934 - val_accuracy: 0.6620 - val_f1_score: 0.6606
6 - val_loss: 1.2426 - val_precision: 0.7788 - learning_rate: 1.0000e-05

Epoch 2/25
1313/1314 ————— 0s 24ms/step - accuracy: 0.8356 - f1_score: 0.8347
- loss: 0.5408 - precision: 0.9078
Epoch 2: val_loss improved from 1.24255 to 1.23747, saving model to best_model_fine_tune_v4.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.8356 - f1_score: 0.8347
7 - loss: 0.5409 - precision: 0.9078 - val_accuracy: 0.6644 - val_f1_score: 0.6629
9 - val_loss: 1.2375 - val_precision: 0.7812 - learning_rate: 1.0000e-05

Epoch 3/25
1313/1314 ————— 0s 24ms/step - accuracy: 0.8391 - f1_score: 0.8380
- loss: 0.5291 - precision: 0.9113
Epoch 3: val_loss improved from 1.23747 to 1.23559, saving model to best_model_fine_tune_v4.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.8391 - f1_score: 0.8380
0 - loss: 0.5292 - precision: 0.9113 - val_accuracy: 0.6641 - val_f1_score: 0.6629
9 - val_loss: 1.2356 - val_precision: 0.7790 - learning_rate: 1.0000e-05

Epoch 4/25
1313/1314 ————— 0s 23ms/step - accuracy: 0.8401 - f1_score: 0.8391
- loss: 0.5218 - precision: 0.9118
Epoch 4: val_loss improved from 1.23559 to 1.23187, saving model to best_model_fine_tune_v4.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.8401 - f1_score: 0.8390
0 - loss: 0.5219 - precision: 0.9118 - val_accuracy: 0.6647 - val_f1_score: 0.6634
4 - val_loss: 1.2319 - val_precision: 0.7820 - learning_rate: 1.0000e-05

Epoch 5/25
1312/1314 ————— 0s 23ms/step - accuracy: 0.8421 - f1_score: 0.8410
- loss: 0.5173 - precision: 0.9142
Epoch 5: val_loss improved from 1.23187 to 1.23006, saving model to best_model_fine_tune_v4.keras
1314/1314 ————— 35s 26ms/step - accuracy: 0.8421 - f1_score: 0.8409
9 - loss: 0.5175 - precision: 0.9142 - val_accuracy: 0.6656 - val_f1_score: 0.6643
3 - val_loss: 1.2301 - val_precision: 0.7804 - learning_rate: 1.0000e-05

Epoch 6/25
1313/1314 ————— 0s 23ms/step - accuracy: 0.8463 - f1_score: 0.8452
- loss: 0.5062 - precision: 0.9161
Epoch 6: val_loss did not improve from 1.23006
1314/1314 ————— 34s 26ms/step - accuracy: 0.8463 - f1_score: 0.8451
1 - loss: 0.5063 - precision: 0.9161 - val_accuracy: 0.6666 - val_f1_score: 0.6651
1 - val_loss: 1.2310 - val_precision: 0.7806 - learning_rate: 1.0000e-05

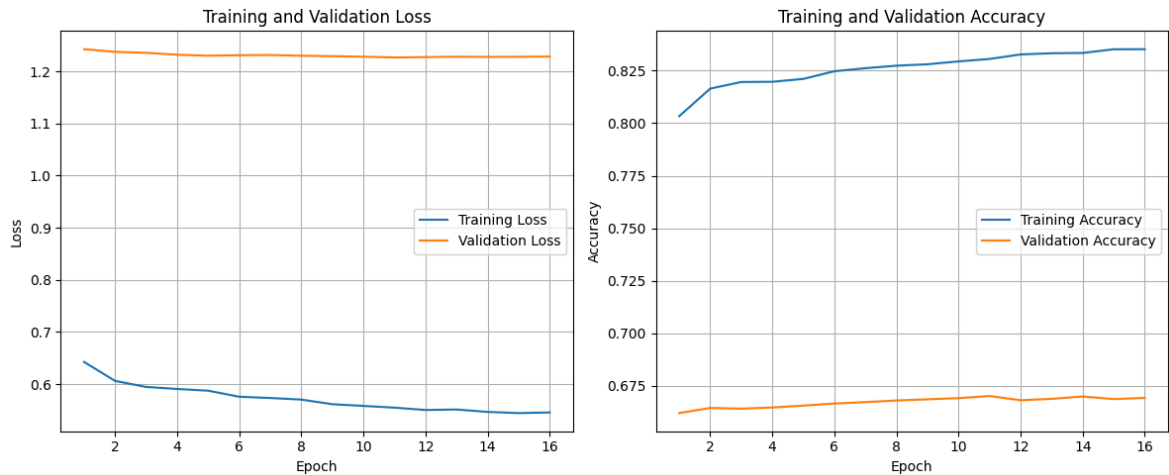
Epoch 7/25
1312/1314 ————— 0s 23ms/step - accuracy: 0.8492 - f1_score: 0.8482
- loss: 0.5010 - precision: 0.9177
Epoch 7: val_loss did not improve from 1.23006
1314/1314 ————— 34s 26ms/step - accuracy: 0.8492 - f1_score: 0.8482
2 - loss: 0.5011 - precision: 0.9177 - val_accuracy: 0.6672 - val_f1_score: 0.6660
0 - val_loss: 1.2315 - val_precision: 0.7809 - learning_rate: 1.0000e-05

Epoch 8/25
1312/1314 ————— 0s 24ms/step - accuracy: 0.8485 - f1_score: 0.8476
- loss: 0.4963 - precision: 0.9187
Epoch 8: val_loss improved from 1.23006 to 1.22991, saving model to best_model_fine_tune_v4.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.8485 - f1_score: 0.847

6 - loss: 0.4965 - precision: 0.9186 - val_accuracy: 0.6680 - val_f1_score: 0.666
5 - val_loss: 1.2299 - val_precision: 0.7811 - learning_rate: 1.0000e-05
Epoch 9/25
1314/1314 ————— 0s 24ms/step - accuracy: 0.8512 - f1_score: 0.8501
- loss: 0.4864 - precision: 0.9207
Epoch 9: val_loss improved from 1.22991 to 1.22914, saving model to best_model_fine_tune_v4.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.8512 - f1_score: 0.850
1 - loss: 0.4865 - precision: 0.9207 - val_accuracy: 0.6686 - val_f1_score: 0.667
3 - val_loss: 1.2291 - val_precision: 0.7825 - learning_rate: 1.0000e-05
Epoch 10/25
1314/1314 ————— 0s 24ms/step - accuracy: 0.8503 - f1_score: 0.8494
- loss: 0.4880 - precision: 0.9183
Epoch 10: val_loss improved from 1.22914 to 1.22813, saving model to best_model_fine_tune_v4.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.8503 - f1_score: 0.849
4 - loss: 0.4881 - precision: 0.9183 - val_accuracy: 0.6691 - val_f1_score: 0.667
8 - val_loss: 1.2281 - val_precision: 0.7824 - learning_rate: 1.0000e-05
Epoch 11/25
1313/1314 ————— 0s 24ms/step - accuracy: 0.8533 - f1_score: 0.8524
- loss: 0.4833 - precision: 0.9204
Epoch 11: val_loss improved from 1.22813 to 1.22664, saving model to best_model_fine_tune_v4.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.8533 - f1_score: 0.852
4 - loss: 0.4834 - precision: 0.9204 - val_accuracy: 0.6701 - val_f1_score: 0.668
8 - val_loss: 1.2266 - val_precision: 0.7818 - learning_rate: 1.0000e-05
Epoch 12/25
1314/1314 ————— 0s 24ms/step - accuracy: 0.8548 - f1_score: 0.8538
- loss: 0.4792 - precision: 0.9219
Epoch 12: val_loss did not improve from 1.22664
1314/1314 ————— 34s 26ms/step - accuracy: 0.8548 - f1_score: 0.853
8 - loss: 0.4793 - precision: 0.9219 - val_accuracy: 0.6681 - val_f1_score: 0.666
7 - val_loss: 1.2274 - val_precision: 0.7795 - learning_rate: 1.0000e-05
Epoch 13/25
1312/1314 ————— 0s 24ms/step - accuracy: 0.8560 - f1_score: 0.8548
- loss: 0.4771 - precision: 0.9235
Epoch 13: val_loss did not improve from 1.22664
1314/1314 ————— 34s 26ms/step - accuracy: 0.8559 - f1_score: 0.854
8 - loss: 0.4772 - precision: 0.9235 - val_accuracy: 0.6688 - val_f1_score: 0.667
4 - val_loss: 1.2281 - val_precision: 0.7807 - learning_rate: 1.0000e-05
Epoch 14/25
1312/1314 ————— 0s 24ms/step - accuracy: 0.8562 - f1_score: 0.8551
- loss: 0.4724 - precision: 0.9222
Epoch 14: val_loss did not improve from 1.22664

Epoch 14: ReduceLROnPlateau reducing learning rate to 4.999999873689376e-06.
1314/1314 ————— 34s 26ms/step - accuracy: 0.8561 - f1_score: 0.855
0 - loss: 0.4726 - precision: 0.9222 - val_accuracy: 0.6699 - val_f1_score: 0.668
7 - val_loss: 1.2277 - val_precision: 0.7802 - learning_rate: 1.0000e-05
Epoch 15/25
1312/1314 ————— 0s 24ms/step - accuracy: 0.8578 - f1_score: 0.8567
- loss: 0.4707 - precision: 0.9239
Epoch 15: val_loss did not improve from 1.22664
1314/1314 ————— 34s 26ms/step - accuracy: 0.8577 - f1_score: 0.856
6 - loss: 0.4709 - precision: 0.9239 - val_accuracy: 0.6687 - val_f1_score: 0.667
4 - val_loss: 1.2280 - val_precision: 0.7804 - learning_rate: 5.0000e-06
Epoch 16/25
1314/1314 ————— 0s 24ms/step - accuracy: 0.8572 - f1_score: 0.8563
- loss: 0.4732 - precision: 0.9223
Epoch 16: val_loss did not improve from 1.22664

1314/1314 ————— **34s** 26ms/step - accuracy: 0.8572 - f1_score: 0.856
 3 - loss: 0.4733 - precision: 0.9223 - val_accuracy: 0.6692 - val_f1_score: 0.668
 0 - val_loss: 1.2285 - val_precision: 0.7806 - learning_rate: 5.0000e-06



141/141 ————— **11s** 37ms/step - accuracy: 0.6638 - f1_score: 0.6552
 - loss: 1.2445 - precision: 0.7743

Test Accuracy: 67.17%

Test Loss: 1.2226710319519043

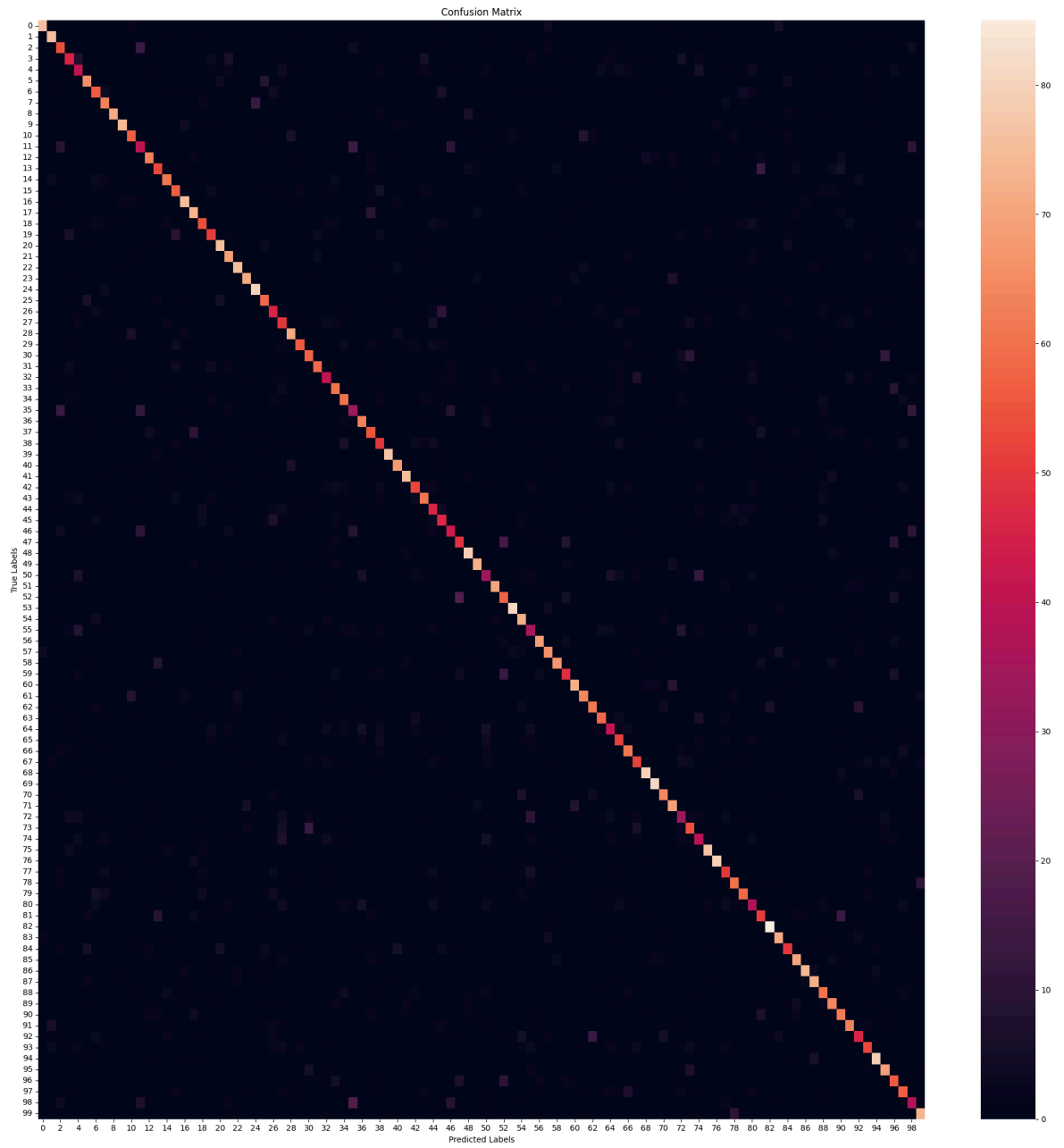
Test Precision: 78.19%

Test F1 Scores (Per Class): [86.36363 79.99999 57.14285 53.179188 43.478252 7
2.92817 61.538452

69.61325	82.558136	84.090904	64.77273	44.44444	71.99999	62.352936
69.71428	62.63736	80.64516	76.041664	61.01694	58.620686	80.64516
73.40425	81.52173	80.22598	84.21052	67.04545	50.549446	50.74626
77.59562	66.666664	64.44444	71.51515	50.61728	62.5	65.57377
37.714283	70.39106	61.111103	54.05405	89.534874	73.513504	84.74575
61.6279	68.15642	48.93617	51.366108	50.588226	56.64739	84.491974
80.	38.596485	75.268814	61.052628	89.99999	72.3618	38.043476
81.17647	71.73912	78.362564	56.470577	80.89887	73.988434	68.53932
69.005844	48.520706	56.830597	66.304344	58.757053	87.91208	86.17021
73.14285	73.79678	38.150284	60.33519	43.575413	84.61538	88.39778
57.471264	68.965515	68.208084	40.883972	56.98323	90.90908	75.93583
58.823524	74.8663	82.68156	78.68852	64.86485	68.449196	64.2487
70.786514	54.43787	61.6279	86.33879	74.59459	57.731956	64.77273
42.222218	78.07486					

Average Test F1 Scores:67.08

282/282 ————— **11s** 27ms/step



	precision	recall	f1-score	support
beaver	0.88	0.84	0.86	90
dolphin	0.76	0.84	0.80	90
otter	0.55	0.60	0.57	90
seal	0.55	0.51	0.53	90
whale	0.42	0.44	0.43	90
aquarium fish	0.73	0.73	0.73	90
flatfish	0.61	0.62	0.62	90
ray	0.69	0.70	0.70	90
shark	0.87	0.79	0.83	90
trout	0.86	0.82	0.84	90
orchids	0.66	0.63	0.65	90
poppies	0.44	0.44	0.44	90
roses	0.74	0.70	0.72	90
sunflowers	0.66	0.59	0.62	90
tulips	0.72	0.68	0.70	90
bottles	0.62	0.63	0.63	90
bowls	0.78	0.83	0.81	90
cans	0.72	0.81	0.76	90
cups	0.62	0.60	0.61	90
plates	0.61	0.57	0.59	90
apples	0.78	0.83	0.81	90
mushrooms	0.70	0.77	0.73	90
oranges	0.80	0.83	0.82	90
pears	0.82	0.79	0.80	90
sweet peppers	0.80	0.89	0.84	90
clock	0.69	0.66	0.67	90
computer keyboard	0.50	0.51	0.51	90
lamp	0.45	0.56	0.50	90
telephone	0.76	0.79	0.78	90
television	0.72	0.62	0.67	90
bed	0.64	0.64	0.64	90
chair	0.79	0.66	0.72	90
couch	0.57	0.46	0.51	90
table	0.59	0.67	0.62	90
wardrobe	0.65	0.67	0.66	90
bee	0.39	0.37	0.38	90
beetle	0.71	0.70	0.70	90
butterfly	0.61	0.61	0.61	90
caterpillar	0.53	0.56	0.54	90
cockroach	0.94	0.86	0.90	90
bear	0.72	0.76	0.74	90
leopard	0.86	0.83	0.85	90
lion	0.65	0.59	0.62	90
tiger	0.69	0.68	0.68	90
wolf	0.47	0.51	0.49	90
bridge	0.51	0.52	0.51	90
castle	0.54	0.48	0.51	90
house	0.59	0.54	0.57	90
road	0.81	0.88	0.84	90
skyscraper	0.80	0.80	0.80	90
cloud	0.41	0.37	0.39	90
forest	0.73	0.78	0.75	90
mountain	0.58	0.64	0.61	90
plain	0.90	0.90	0.90	90
sea	0.66	0.80	0.72	90
camel	0.37	0.39	0.38	90
cattle	0.86	0.77	0.81	90
chimpanzee	0.70	0.73	0.72	90

elephant	0.83	0.74	0.78	90
kangaroo	0.60	0.53	0.56	90
fox	0.82	0.80	0.81	90
porcupine	0.77	0.71	0.74	90
possum	0.69	0.68	0.69	90
raccoon	0.73	0.66	0.69	90
skunk	0.52	0.46	0.49	90
crab	0.56	0.58	0.57	90
lobster	0.65	0.68	0.66	90
snail	0.60	0.58	0.59	90
spider	0.87	0.89	0.88	90
worm	0.83	0.90	0.86	90
baby	0.75	0.71	0.73	90
boy	0.71	0.77	0.74	90
girl	0.40	0.37	0.38	90
man	0.61	0.60	0.60	90
woman	0.44	0.43	0.44	90
crocodile	0.84	0.86	0.85	90
dinosaur	0.88	0.89	0.88	90
lizard	0.60	0.56	0.57	90
snake	0.71	0.67	0.69	90
turtle	0.71	0.66	0.68	90
hamster	0.41	0.41	0.41	90
mouse	0.57	0.57	0.57	90
rabbit	0.88	0.94	0.91	90
shrew	0.73	0.79	0.76	90
squirrel	0.62	0.56	0.59	90
maple	0.72	0.78	0.75	90
oak	0.83	0.82	0.83	90
palm	0.77	0.80	0.79	90
pine	0.63	0.67	0.65	90
willow	0.66	0.71	0.68	90
bicycle	0.60	0.69	0.64	90
bus	0.72	0.70	0.71	90
motorcycle	0.58	0.51	0.54	90
pickup truck	0.65	0.59	0.62	90
train	0.85	0.88	0.86	90
lawn-mower	0.73	0.77	0.75	90
rocket	0.54	0.62	0.58	90
streetcar	0.66	0.63	0.65	90
tank	0.42	0.42	0.42	90
tractor	0.75	0.81	0.78	90
accuracy			0.67	9000
macro avg	0.67	0.67	0.67	9000
weighted avg	0.67	0.67	0.67	9000